

javaSE

一、java概念

1.什么是程序?

程序：程序□是一组计算机能识别和执行的指令，用于指导计算机执行特定任务或解决特定问题。程序通常由程序员使用某种程序设计语言编写，这些语言包括但不限于Java□、C/C++□、C#□等。编写程序的过程涉及到将算法和数据处理逻辑转换为计算机能够理解的指令序列。这些指令被存储在计算机的存储介质上，如硬盘□或内存□中，当程序被执行时，计算机会按照这些指令的顺序执行，从而完成特定的功能或任务。

2.什么是java?

Java是一门面向对象的编程语言，不仅吸收了C++语言的各种优点，还摒弃了C++里难以理解的多继承、指针等概念，因此Java语言具有功能强大和简单易用两个特征。Java语言作为静态面向对象编程语言的代表，极好地实现了面向对象理论，允许程序员以优雅的思维方式进行复杂的编程。

Java具有简单性、面向对象、分布式、健壮性、安全性、平台独立与可移植性、多线程、动态性等特点。Java可以编写桌面应用程序、Web应用程序、分布式系统和嵌入式系统应用程序等。

3.java三大版本

javaSE:标准版（桌面程序，控制台开发....）

javaME:嵌入式开发（手机小家电....）

javaEE:E企业级开发（web端，服务器开发....）

4.java软件介绍

1.Java 核心机制-Java 虚拟机 [JVM java virtual machine]

JVM：JVM 是一个虚拟的计算机，具有指令集并使用不同的存储区域。负责执行指令，管理数据、内存、寄存器，包含在JDK中。Java 虚拟机机制屏蔽了底层运行平台的差别，实现了“一次编译，到处运行”。

2. JDK和JRE

JDK：JDK 的全称(Java Development Kit Java 开发工具包)，JDK = JRE+java的开发工具[java,javac,javadoc,javap等]。JDK 是提供给 Java 开发人员使用的，其中包含了 java 的开发工具，也包括了 JRE。所以安装了 JDK，就不用在单独安装 JRE 了。

JRE：JRE(Java Runtime Environment Java 运行环境)JRE = JVM + Java 的核心类库[类] 包括 Java 虚拟机(JVM Java Virtual Machine)和 Java 程序所需的核心类库等，如果想要运行一个开发好的 Java 程序，计算机中只需要安装 JRE 即可。

二、java开发环境搭建

1.windows下载及安装jdk

下载地址：<https://www.oracle.com/java/technologies/downloads/archive/>

名称	修改日期	类型	大小
jdk-8u131-windows-x64.exe	2024/5/2 19:53	应用程序	202,784 KB
jdk-11.0.8_windows-x64_bin.exe	2024/5/2 19:52	应用程序	155,377 KB

双击进行安装

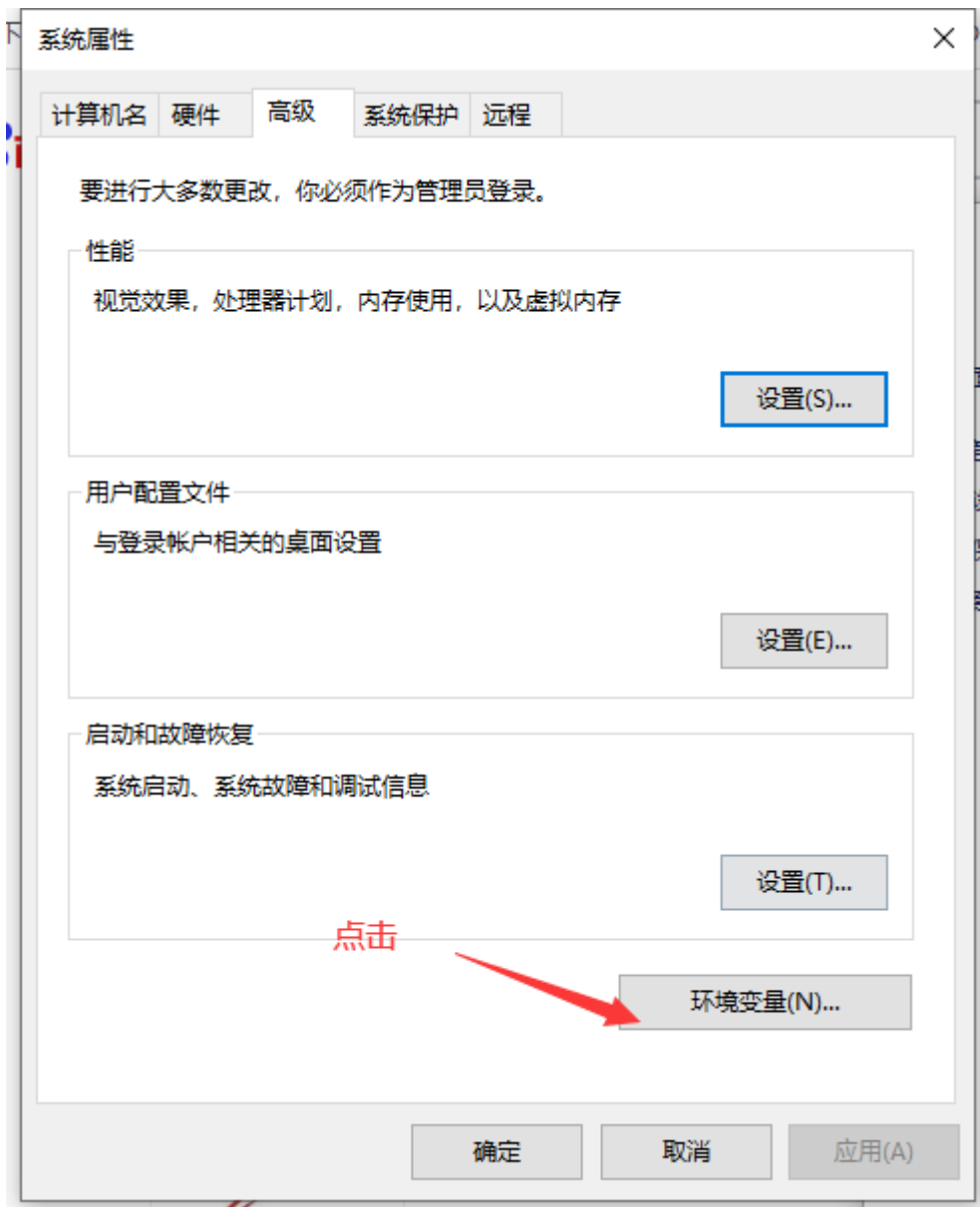
我的安装路径：D:\APP\java\jdk11



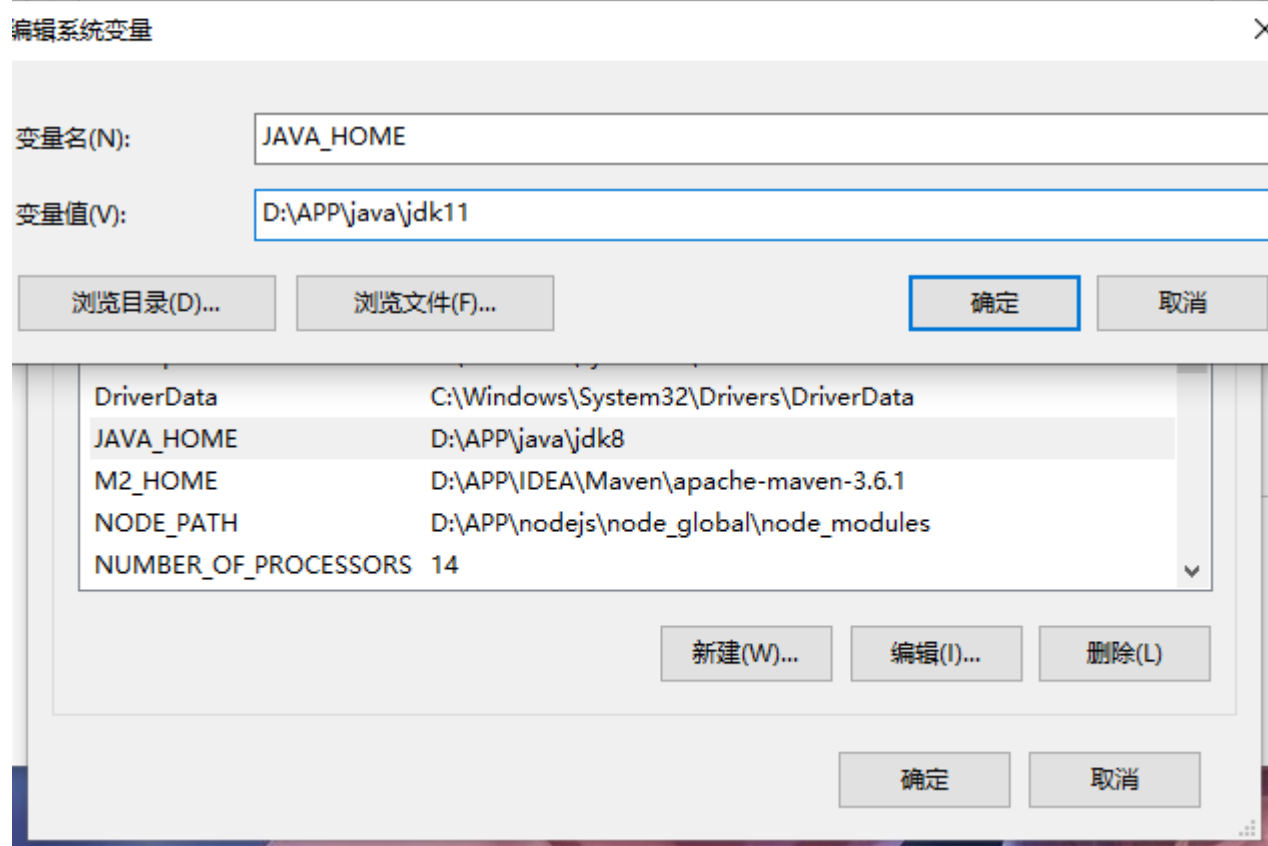
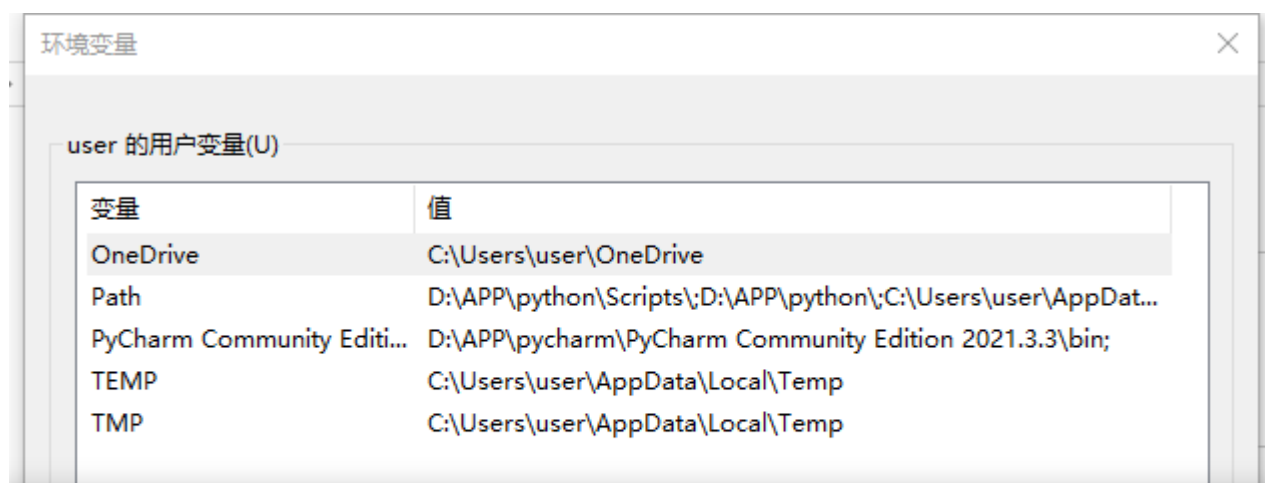
2.环境变量配置

右击我的电脑-----》属性-----》高级系统设置-----》环境变量

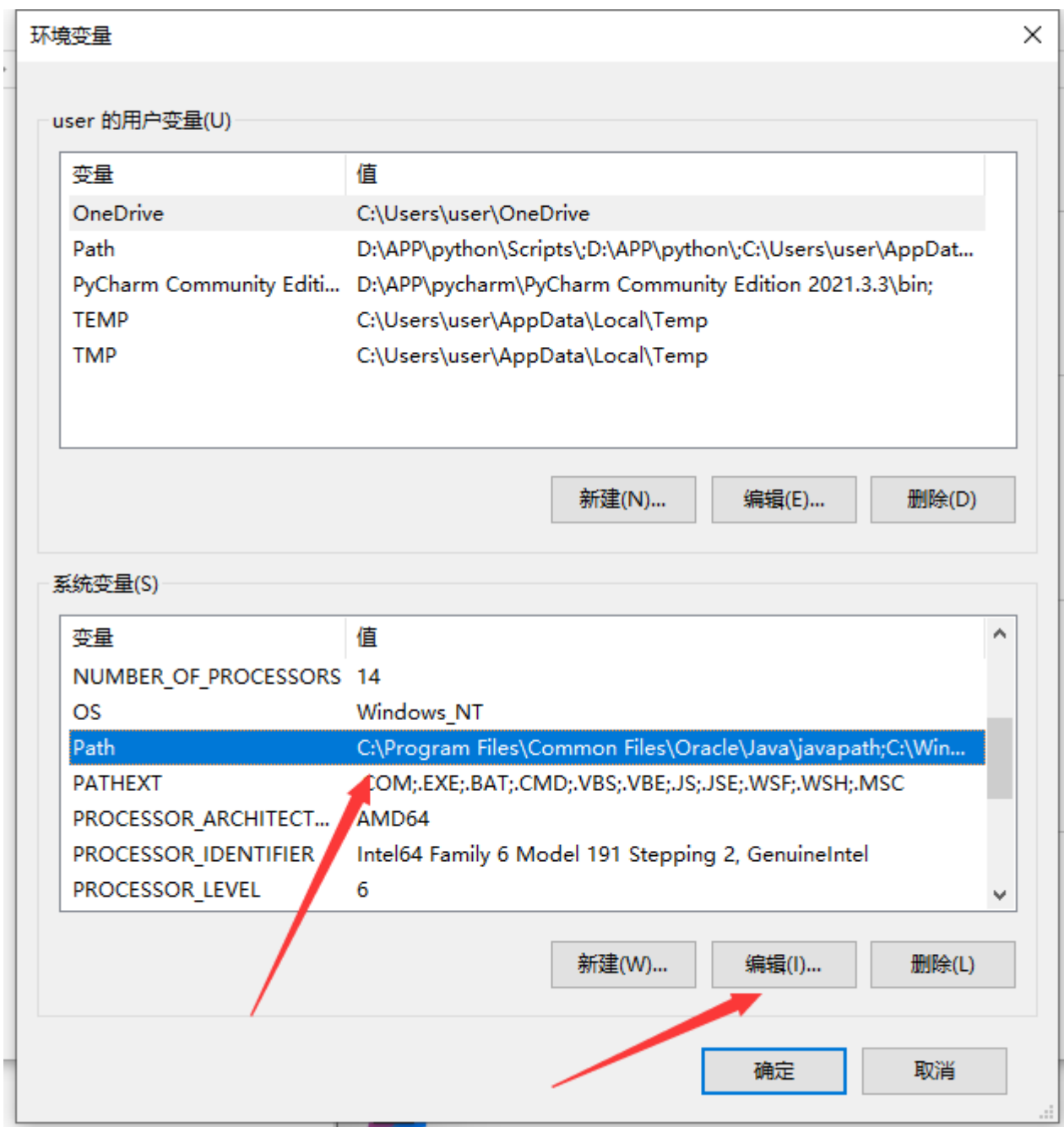




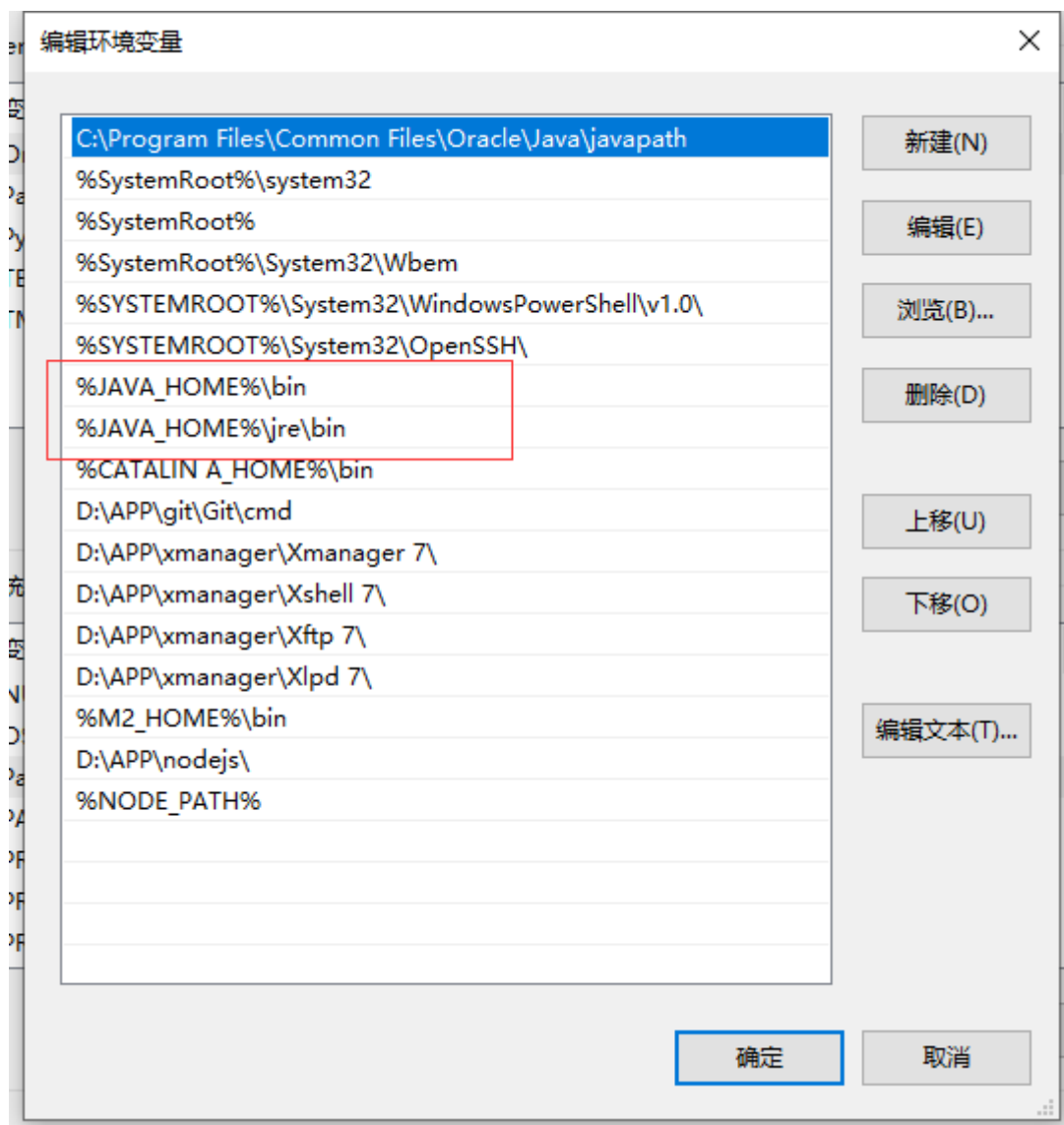
新建系统变量变量名: JAVA_HOME, 变量值: java安装的路径



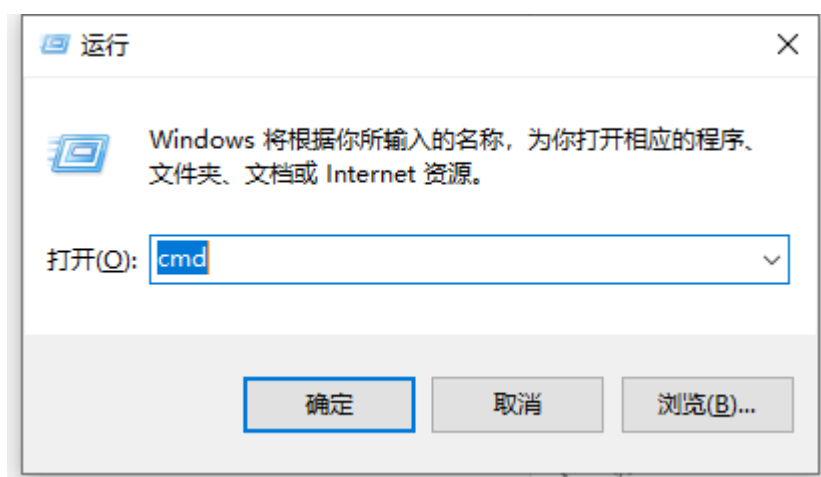
点击path----》编辑----》新建



新加: %JAVA_HOME%\bin和%JAVA_HOME%\jre\bin



最后win+r,输入cmd回车后输入java -version 进行验证



出现以下代表已经安装完成

```
C:\Windows\system32\cmd.exe
Microsoft Windows [版本 10.0.18362.30]
(c) 2019 Microsoft Corporation。保留所有权利。

C:\Users\user>java -version
java version "11.0.8" 2020-07-14 LTS
Java(TM) SE Runtime Environment 18.9 (build 11.0.8+10-LTS)
Java HotSpot(TM) 64-Bit Server VM 18.9 (build 11.0.8+10-LTS, mixed mode)

C:\Users\user>
```

三、第一个java程序

1.如何编写java程序？

1.创建文件夹

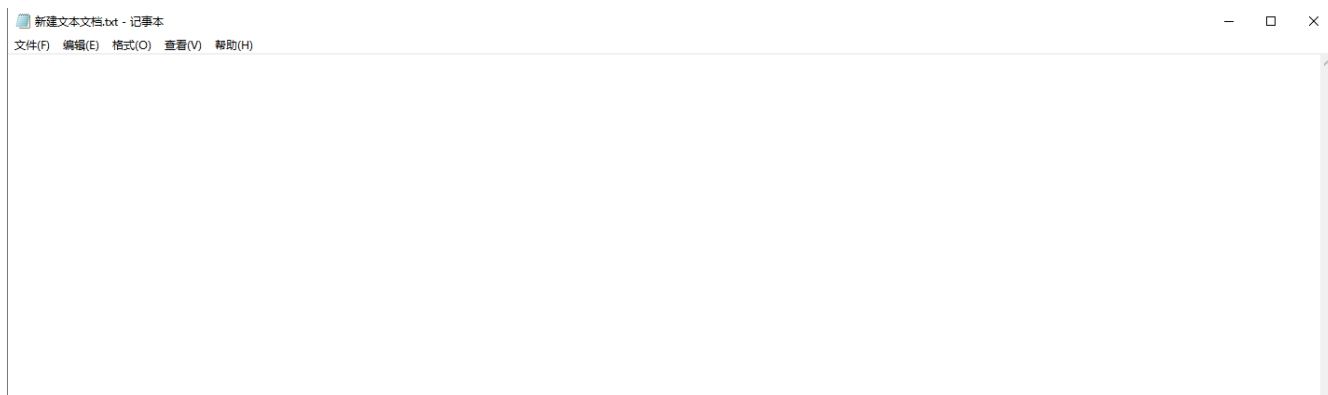
随便找一个磁盘，创建一个文件夹，我在这里在E盘创建一个java_project文件夹



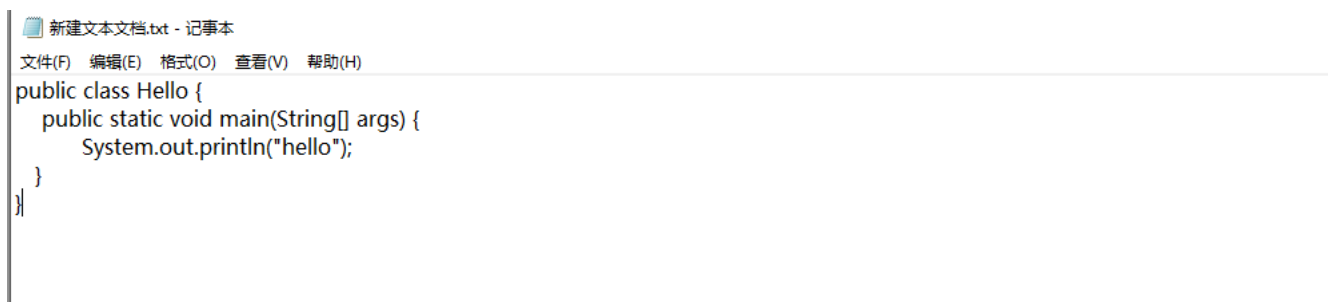
2.在文件夹中创建一个文本文档



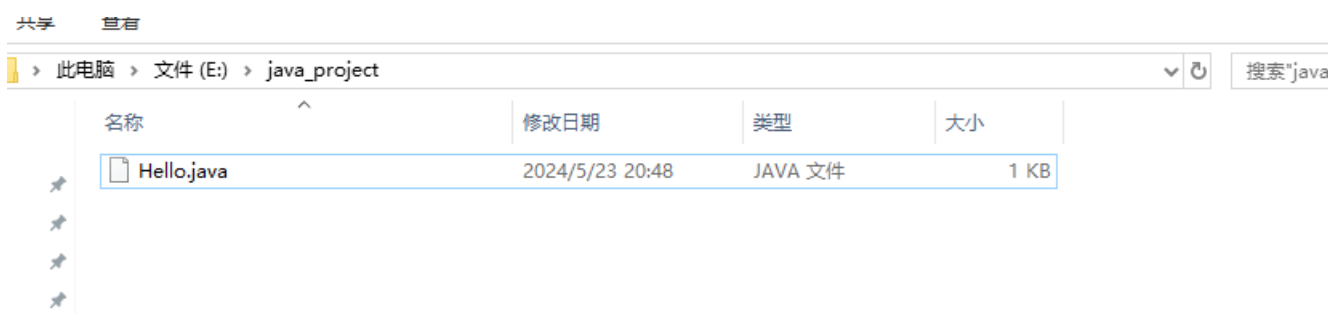
3.双击文本打开记事本



4.编写java代码

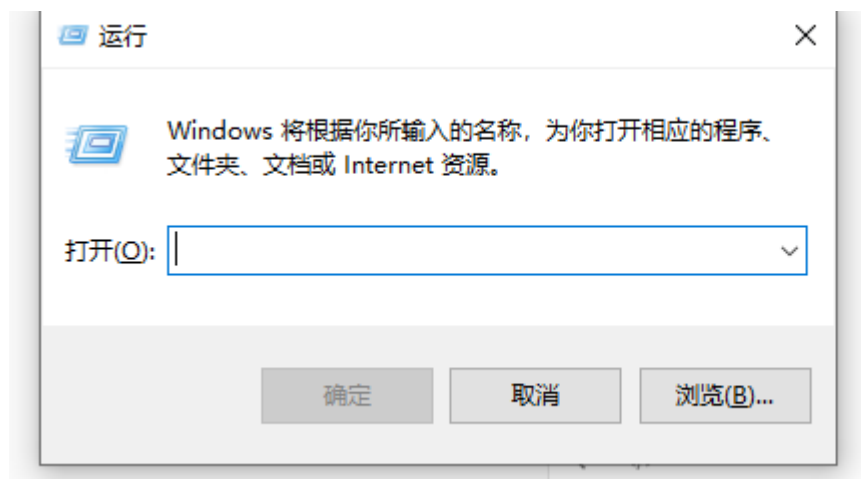


5.保存并修改文件名和文件后缀名,此时的这个文本就变成了java程序

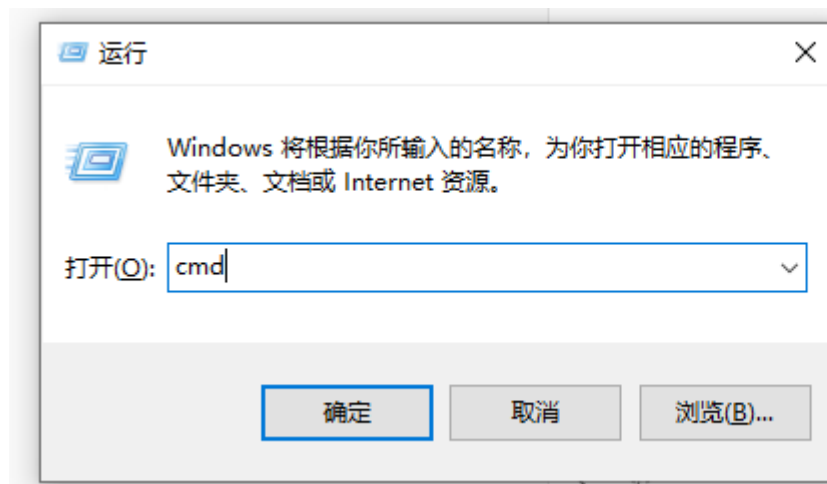


2.如何运行Java?

1.win + r打开运行框



2.输入cmd并回车



3.切换到java文件的所在目录



4.使用javac进行编译。因为java属于编译性语言, 所以需要编译之后才能进行运行

```
管理员: 命令提示符
Microsoft Windows [版本 10.0.18362.30]
(c) 2019 Microsoft Corporation。保留所有权利。

C:\Windows\system32>cd E:\java_project
C:\Windows\system32>e:
E:\java_project>javac Hello.java
E:\java_project>
```

编译完成后会在文件夹里面出现一个.class的文件，这个就是java编译后的文件

此电脑 > 文件 (E:) > java_project				
名称	修改日期	类型	大小	
Hello.class	2024/5/23 21:01	CLASS 文件	1 KB	
Hello.java	2024/5/23 20:48	JAVA 文件	1 KB	

5.运行java

```
管理员: 命令提示符
Microsoft Windows [版本 10.0.18362.30]
(c) 2019 Microsoft Corporation。保留所有权利。

C:\Windows\system32>cd E:\java_project
C:\Windows\system32>e:
E:\java_project>javac Hello.java
E:\java_project>java Hello
hello
E:\java_project>
```

3.语法介绍

```
//1. public class Hello 表示 Hello 是一个类,是一个 public 公有的类
//2. Hello{ } 表示一个类的开始和结束
//3. public static void main(String[] args) 表示一个主方法,即我们程序的入口
//4. main() {} 表示方法的开始和结束
//5. System.out.println("hello,world~"); 表示输出"hello,world~"到屏幕
//6. ;表示语句结束
public class Hello {
    public static void main(String[] args) {
        system.out.println("hello");
    }
}
```

四、idea

1.什么是IDEA?

IDEA：IDEA 全称 IntelliJ IDEA，在业界被公认为最好的 Java 开发工具，IDEA 是 JetBrains 公司的产品，除了支持 Java 开发，还支持 HTML，CSS，PHP，MySQL，Python 等。

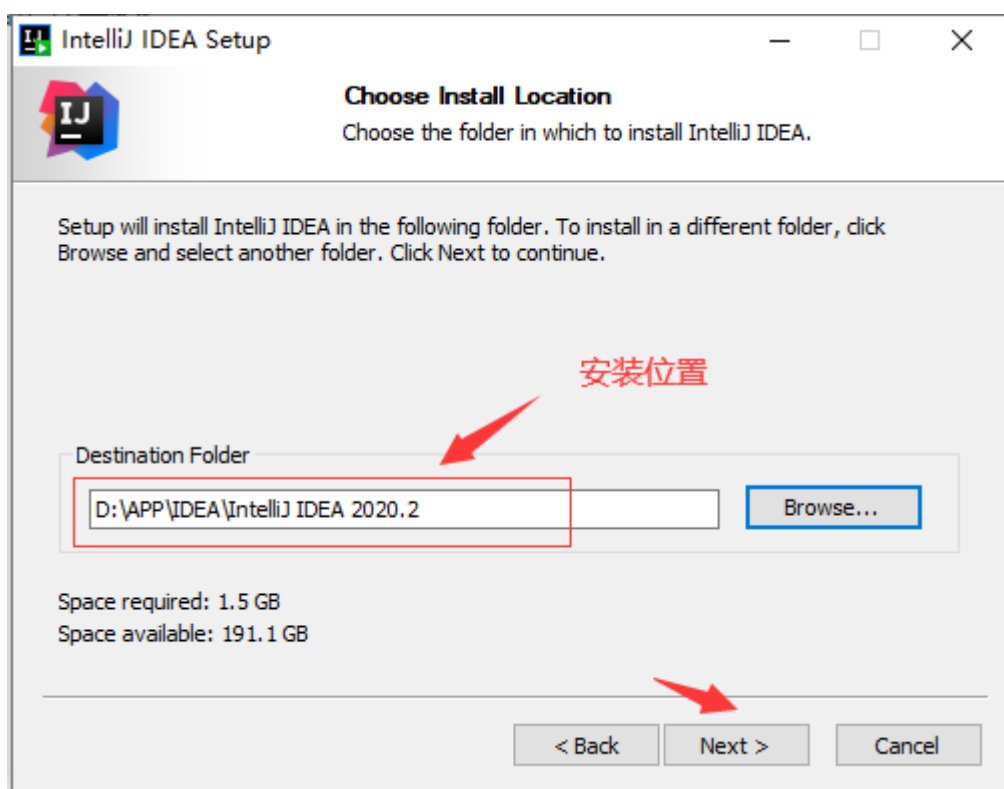
官网: <https://www.jetbrains.com/>

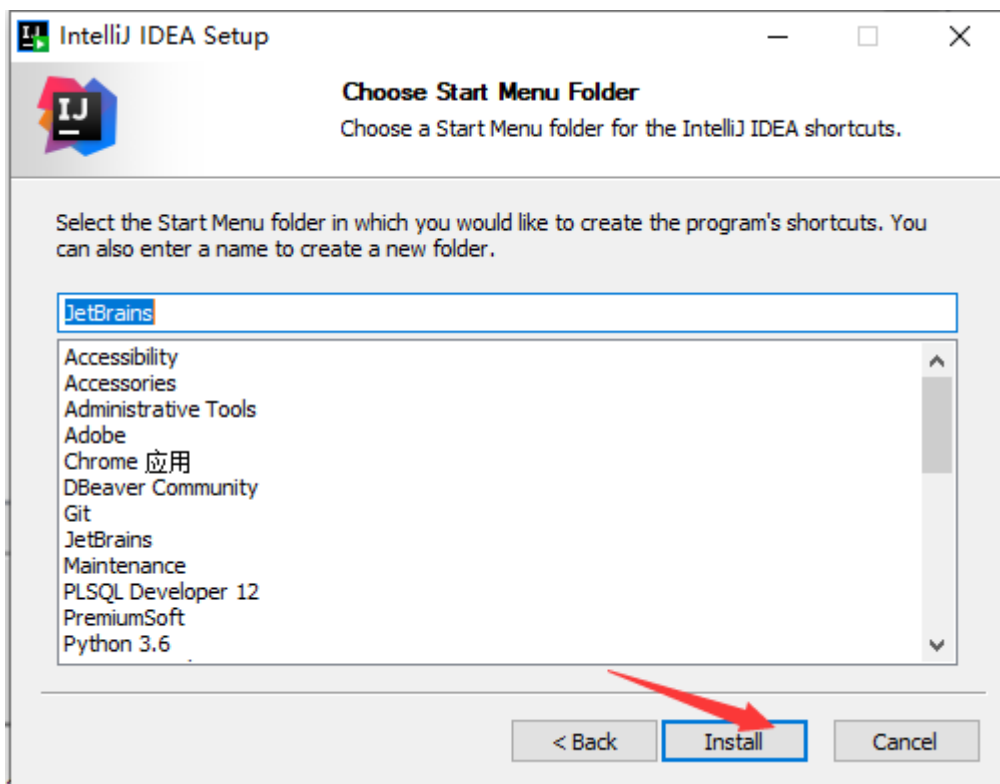
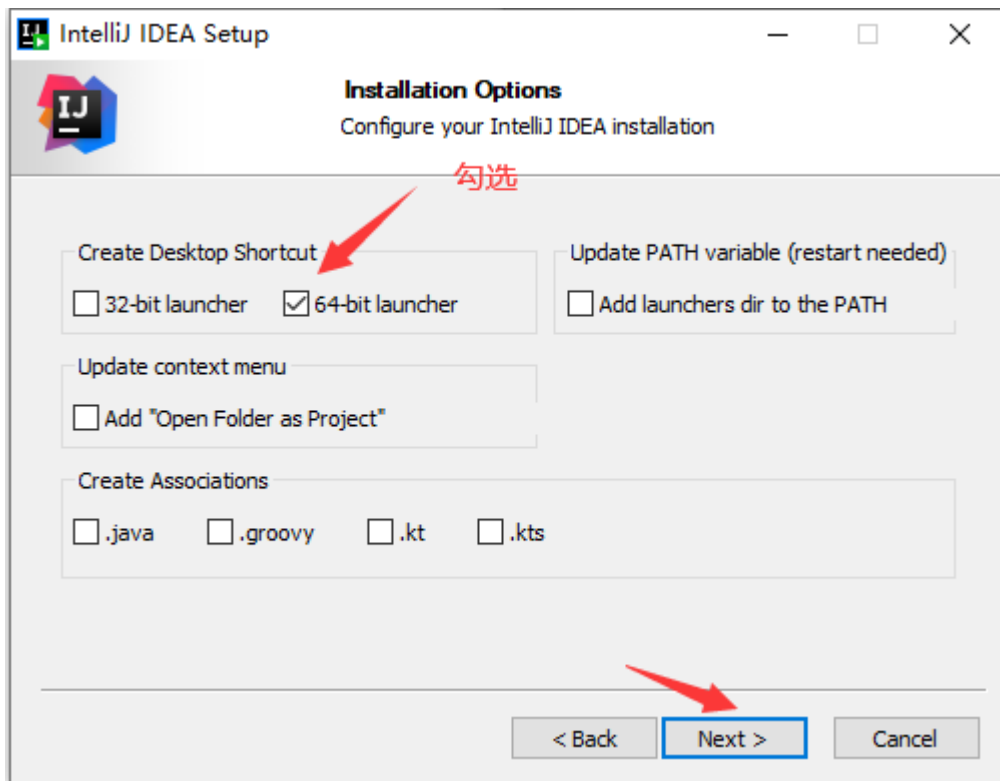
2.idea安装

名称	修改日期	类型	大小
ideaIU-2020.2.exe	2024/4/10 20:23	应用程序	678,636 KB
IDEA永久激活补丁.zip	2024/4/10 20:32	WinRAR ZIP 压缩...	3,924 KB

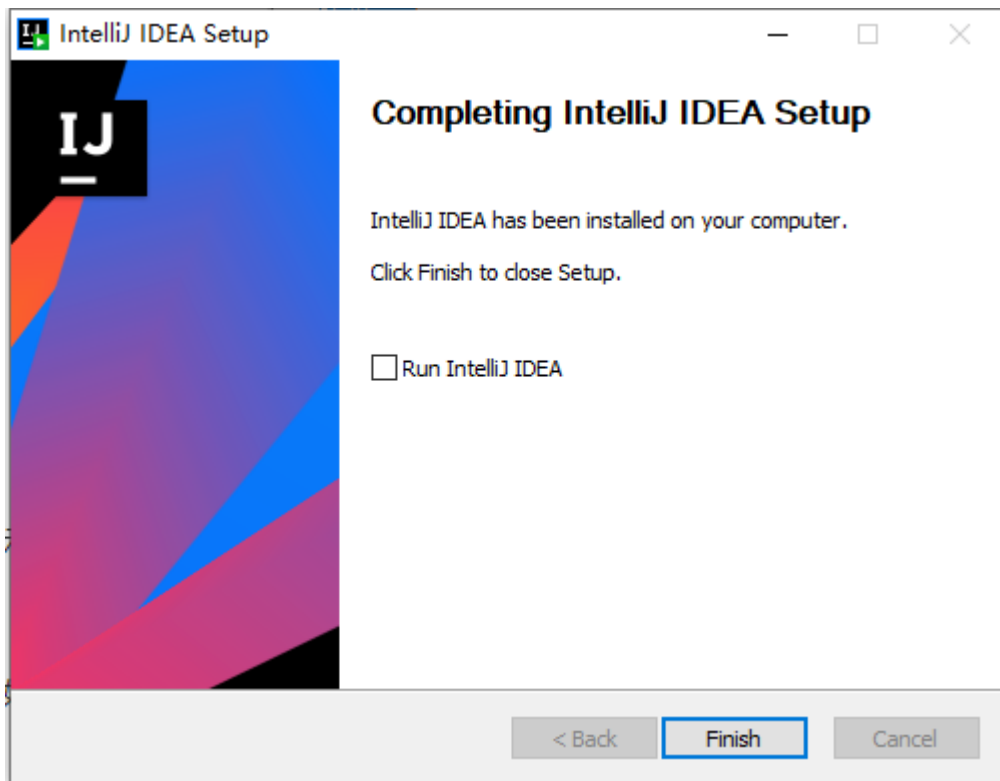
双击安装

指定安装路径

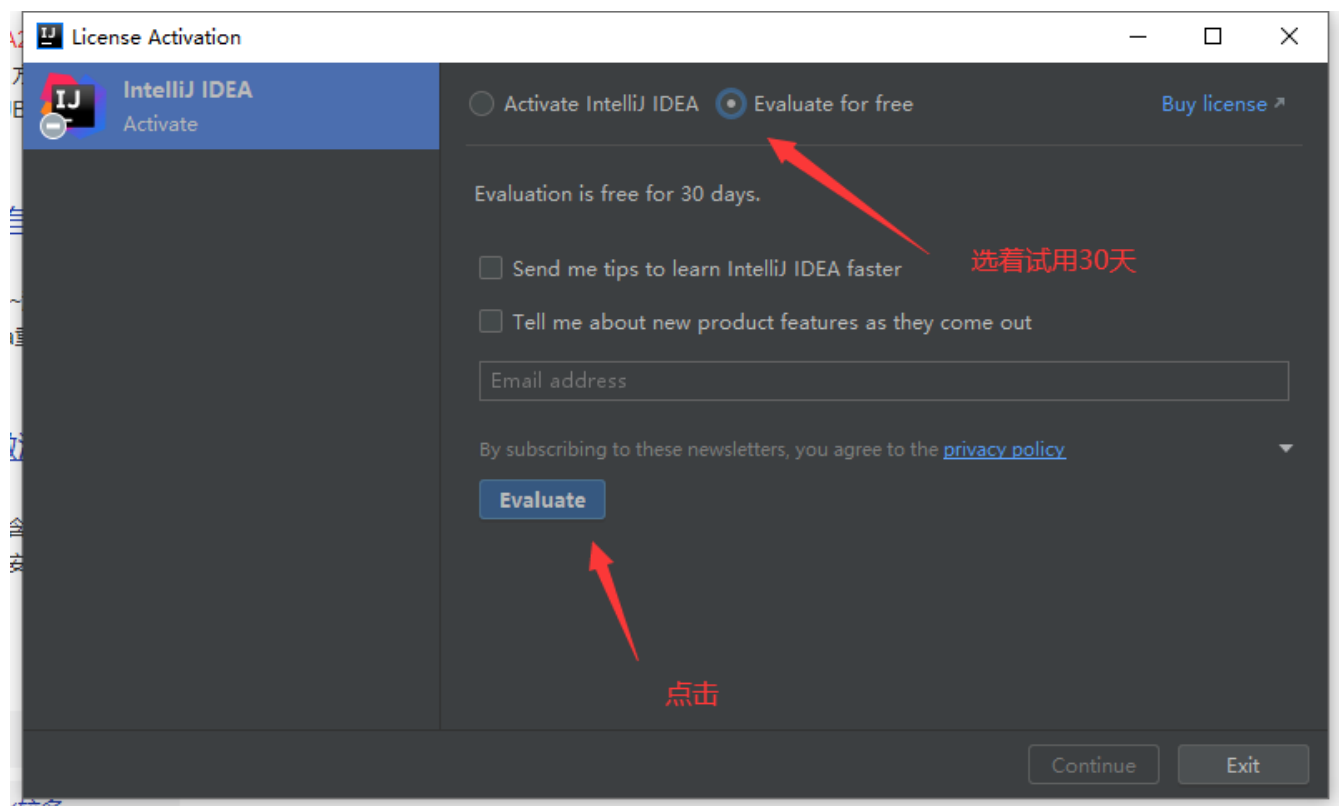




提示已经安装完成，完成后运行idea



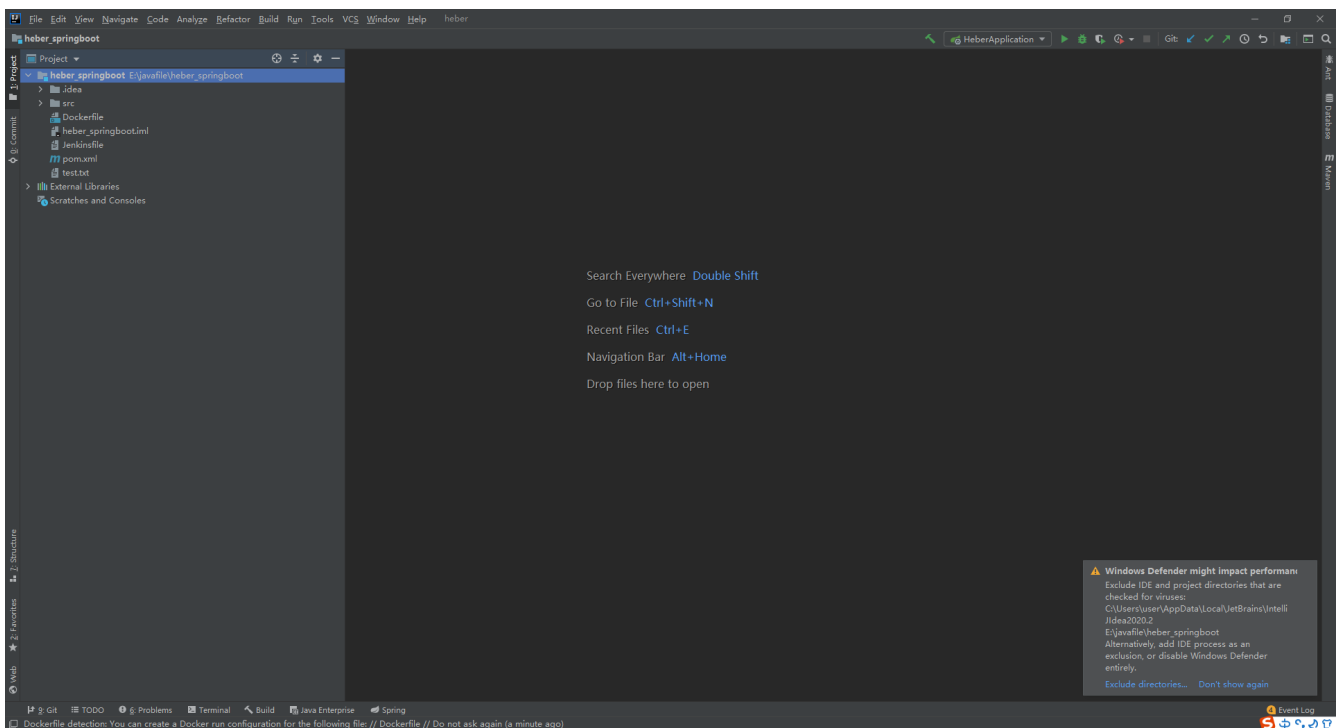
3.激活idea



到了这个界面先打开一个项目，或者新建一个项目



打开后解压破解包



打开解压后的文件

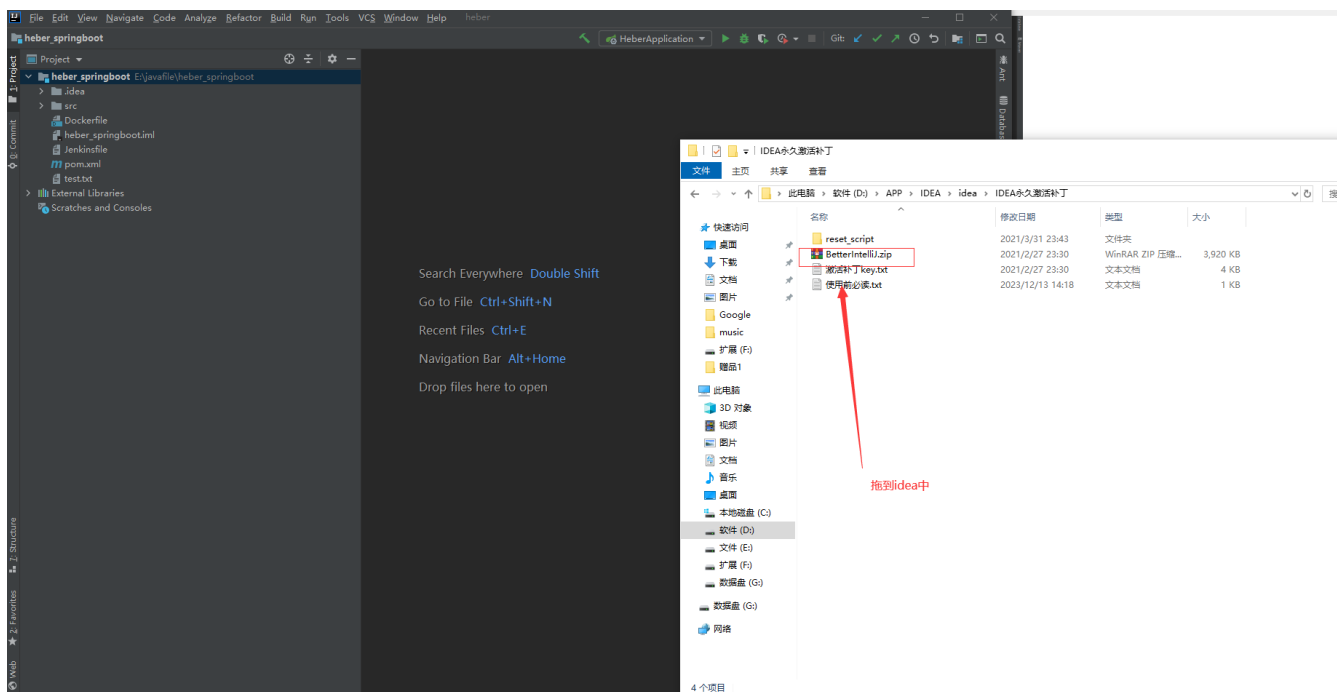
名称	修改日期	类型	大小
IDEA永久激活补丁	2023/12/13 14:06	文件夹	
ideaU-2020.2.exe	2024/4/10 20:23	应用程序	678,636 KB
IDEA永久激活补丁.zip	2024/4/10 20:32	WinRAR ZIP 压缩...	3,924 KB

解压

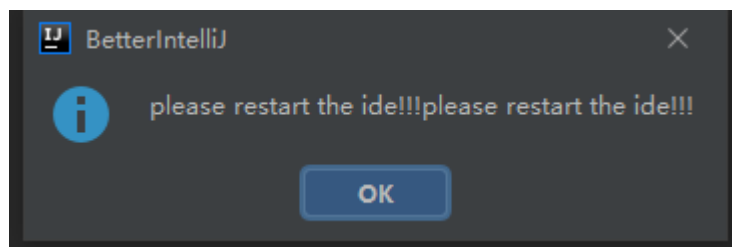
解压后的文件

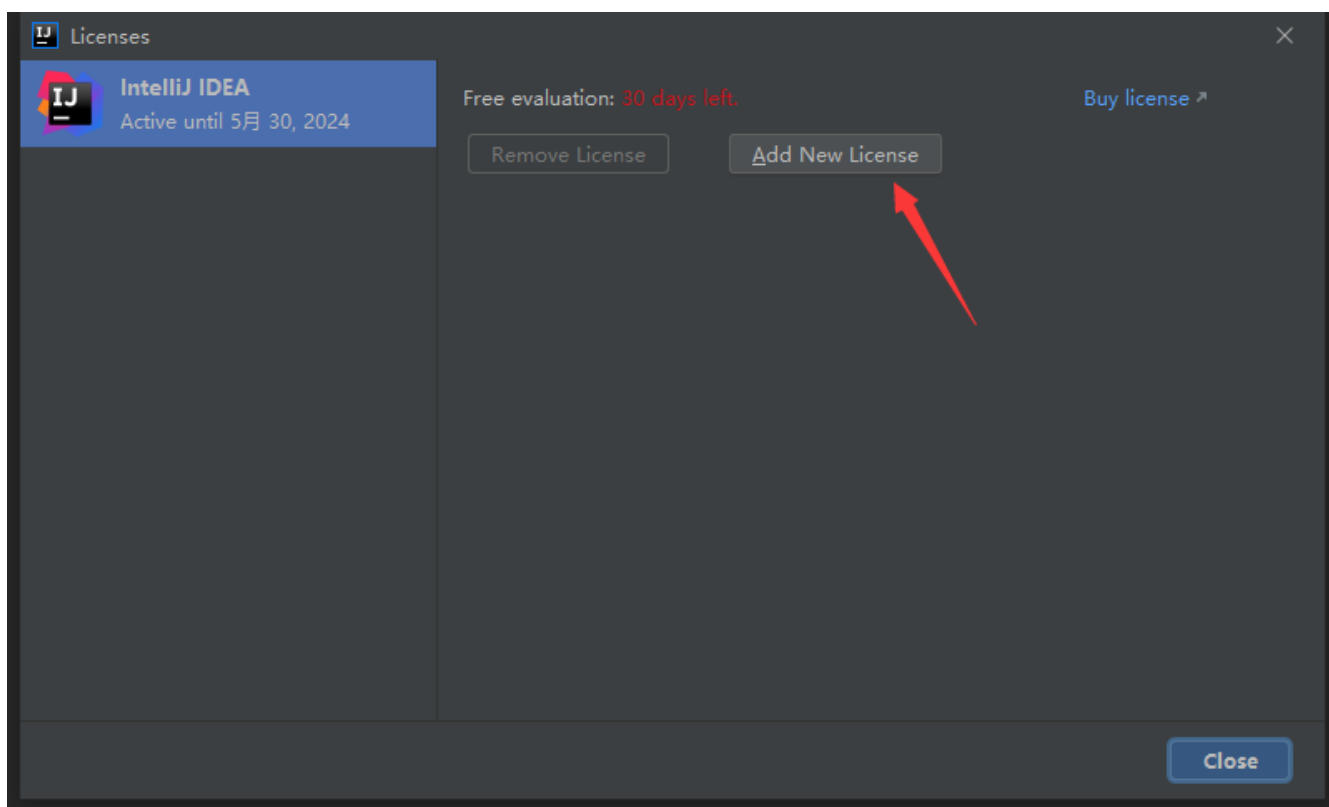
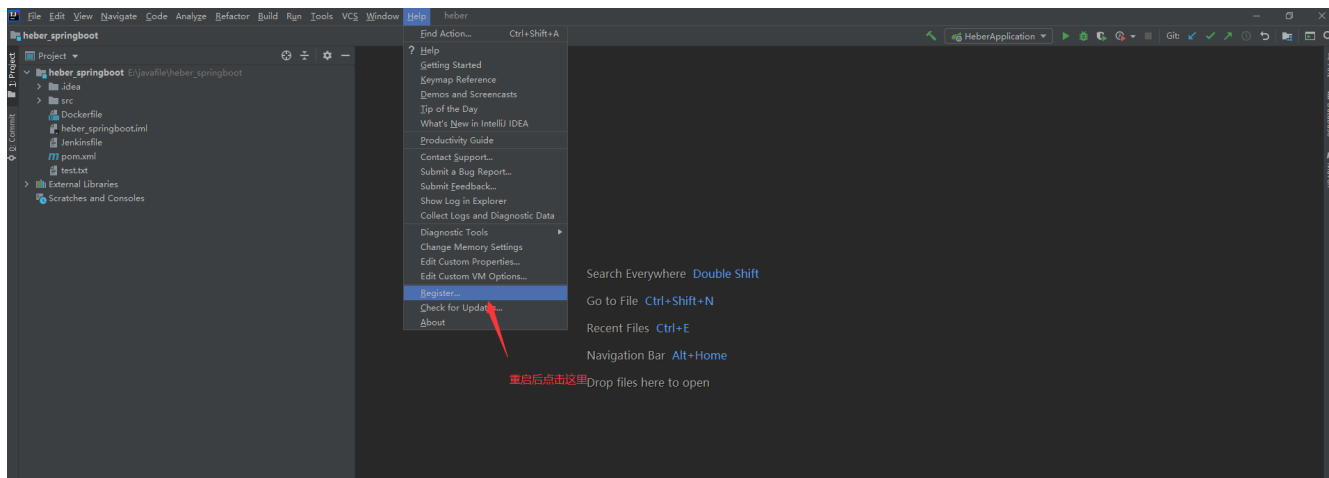
reset_script	2021/3/31 23:43	文件夹	
BetterIntelliJ.zip	2021/2/27 23:30	WinRAR ZIP 压缩...	3,920 KB
激活补丁key.txt	2021/2/27 23:30	文本文档	4 KB
使用前必读.txt	2023/12/13 14:18	文本文档	1 KB

将破解包拖到idea中

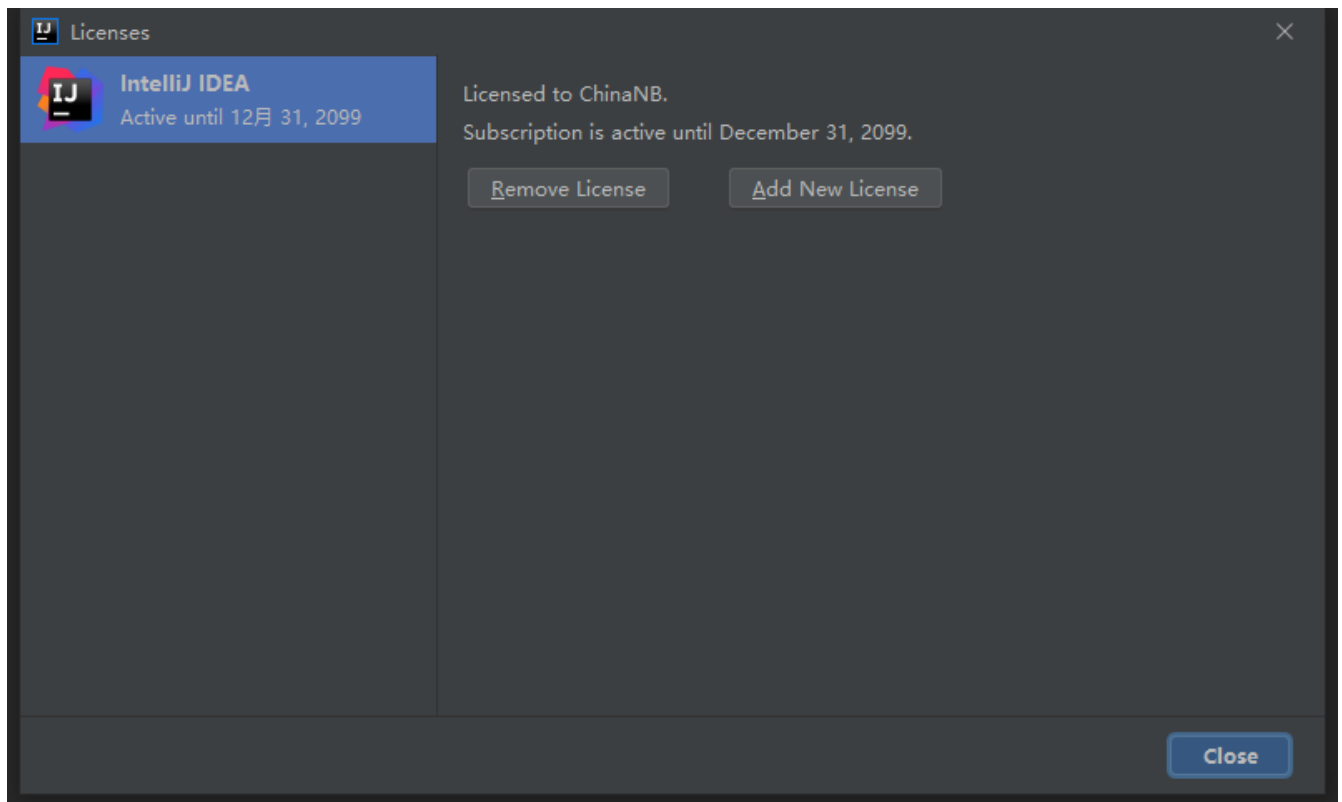


拖进去后会弹出，点击OK并重启idea





激活完成

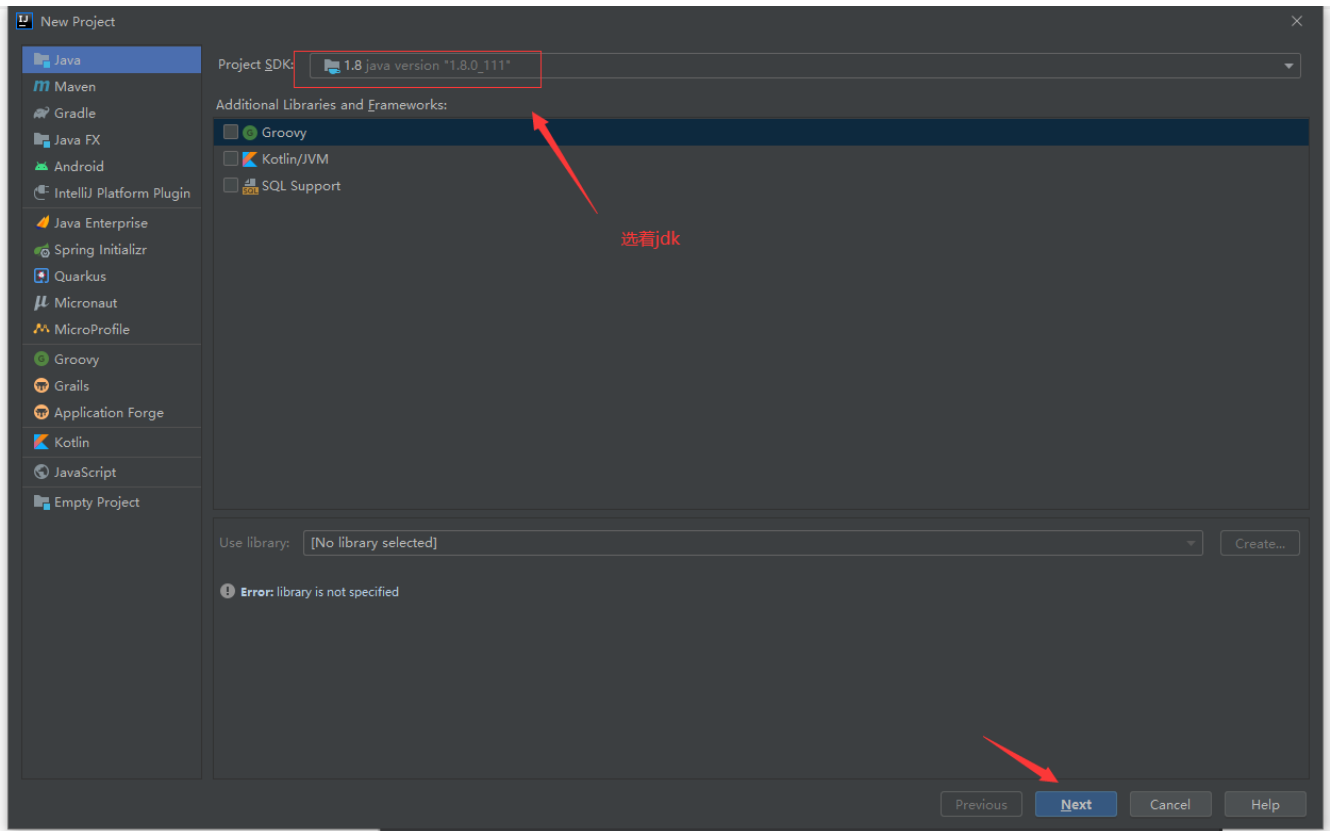


4.使用IDEA创建一个java项目

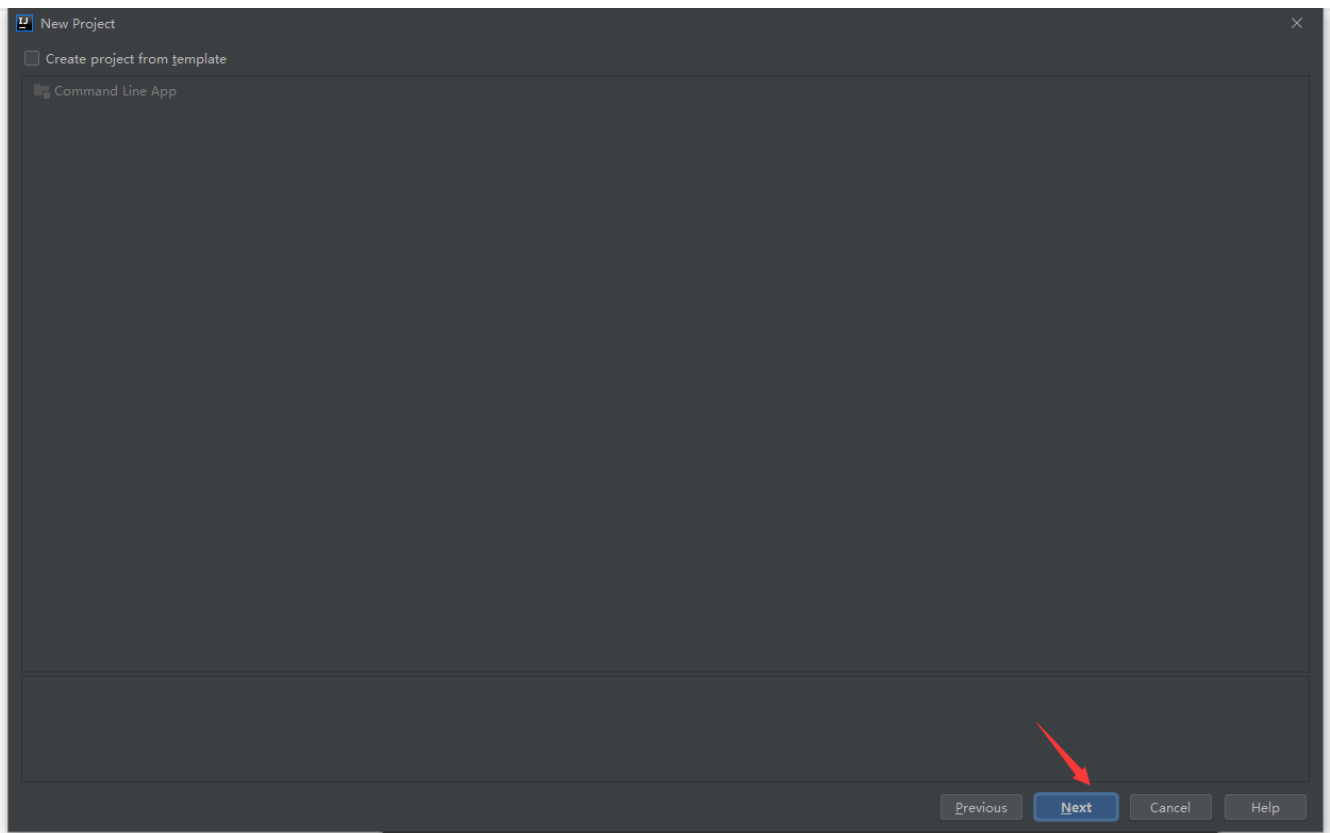
1.双击打开IDEA，如果没有java项目就点击新建



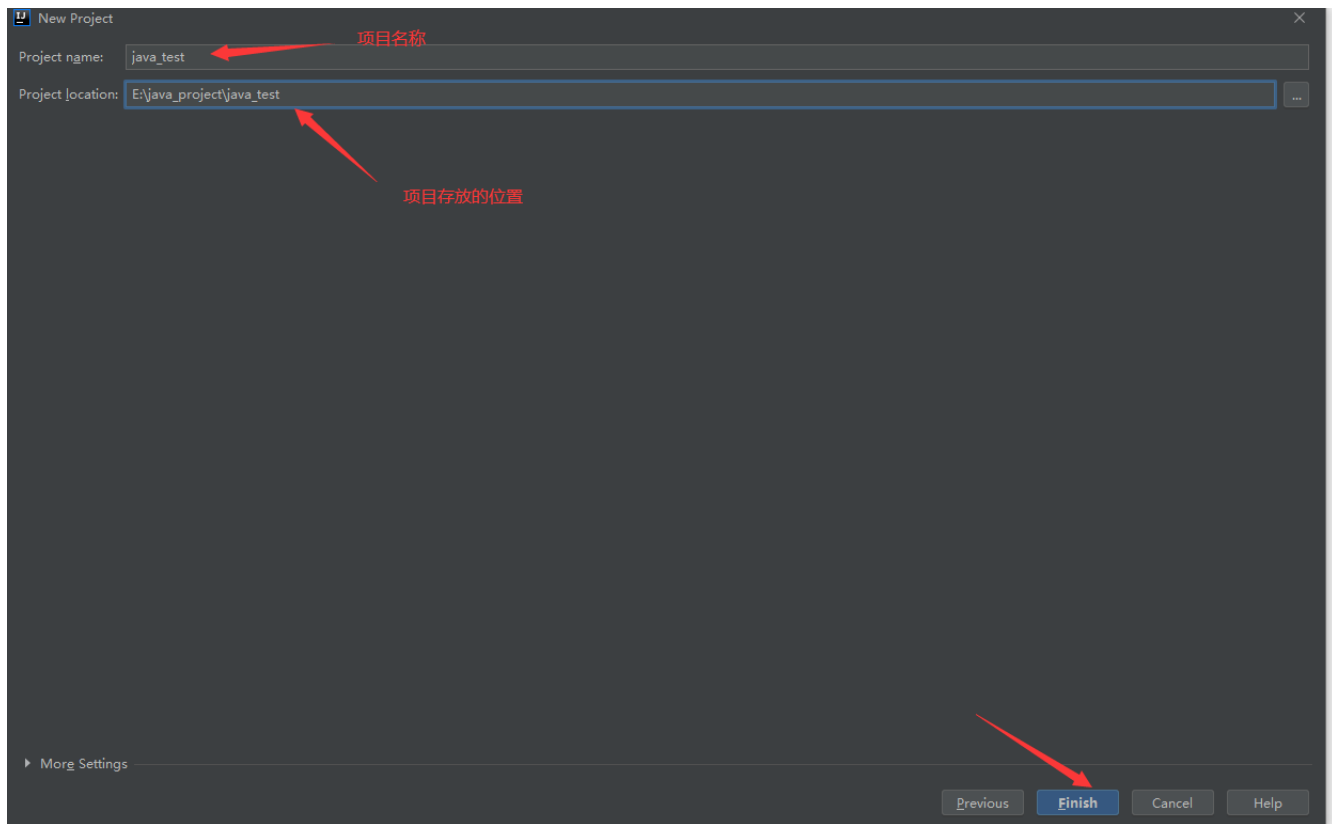
2.选择你安装的jdk版本，点击下一步



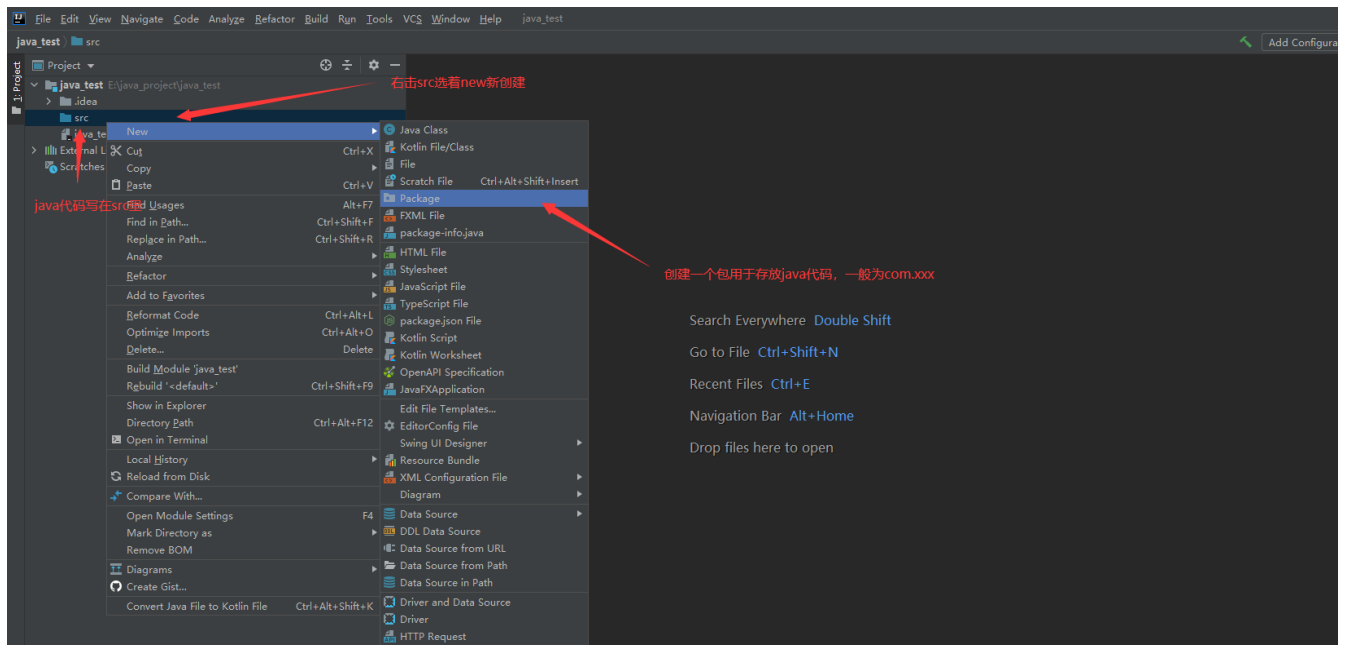
直接下一步



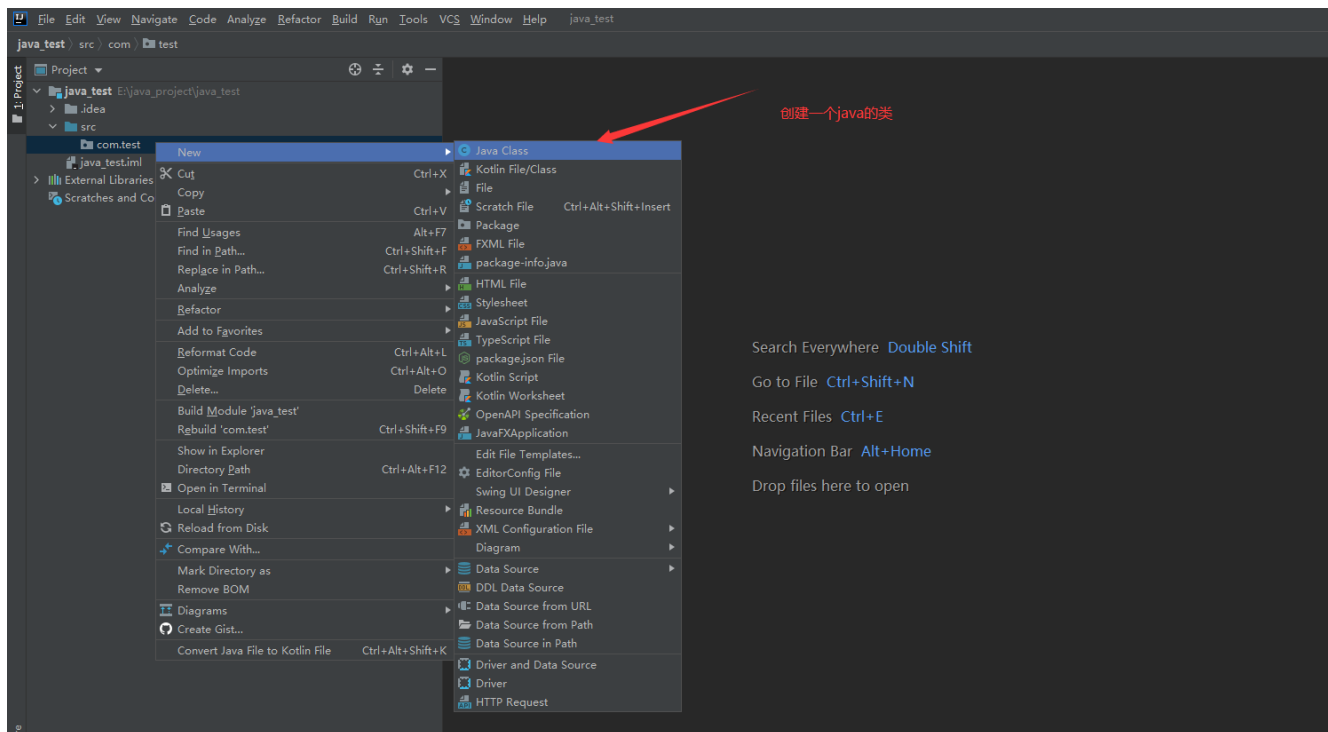
3.填写java项目名称和路径



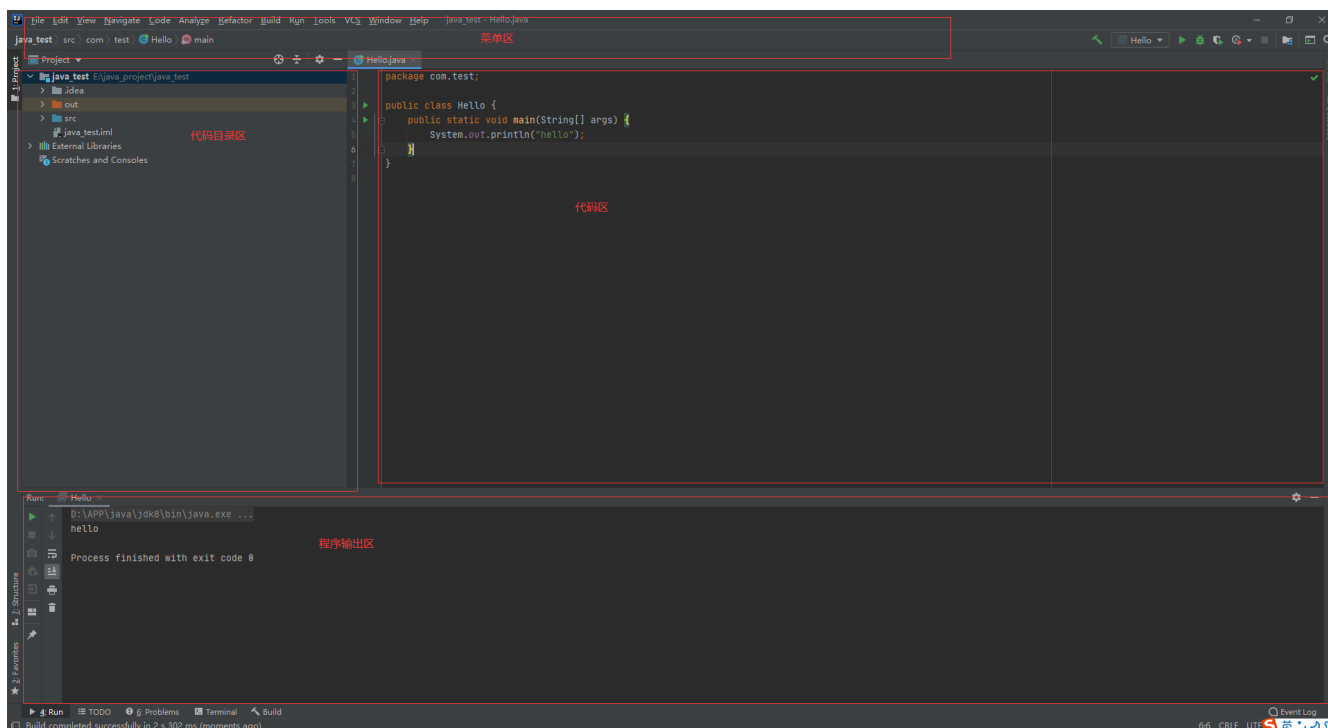
4. 创建java包



5. 创建Java类



5.功能区介绍

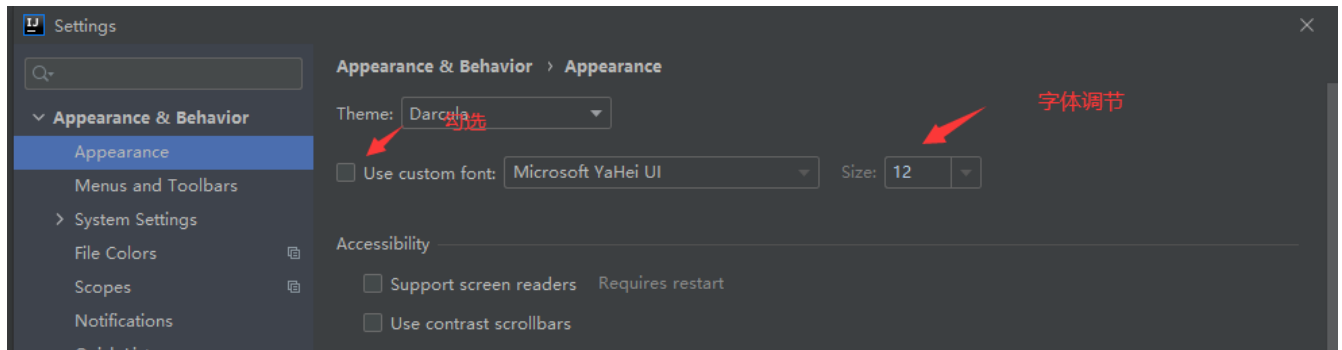


6.IDEA配置

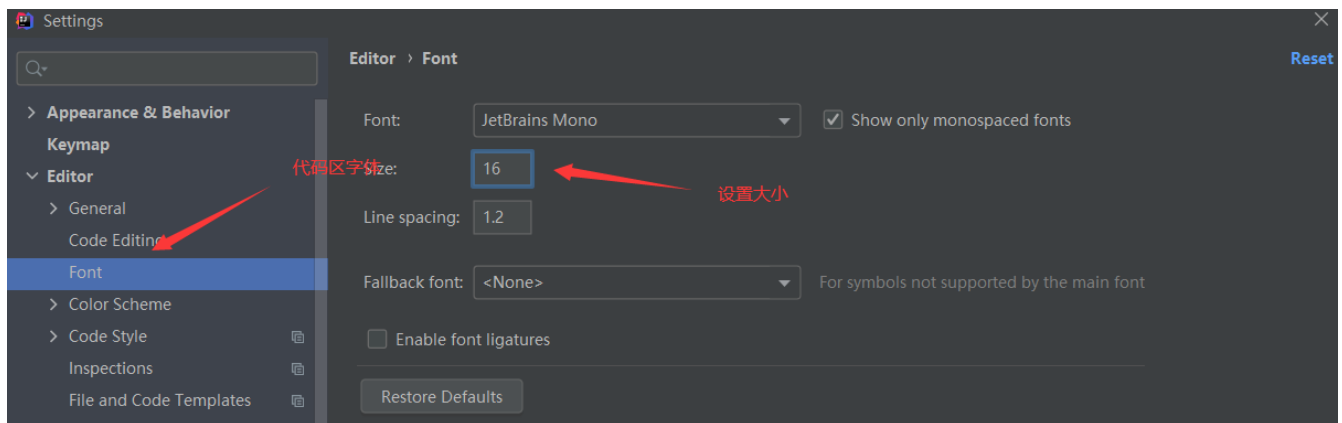
1.设置字体

菜单 file -> settings

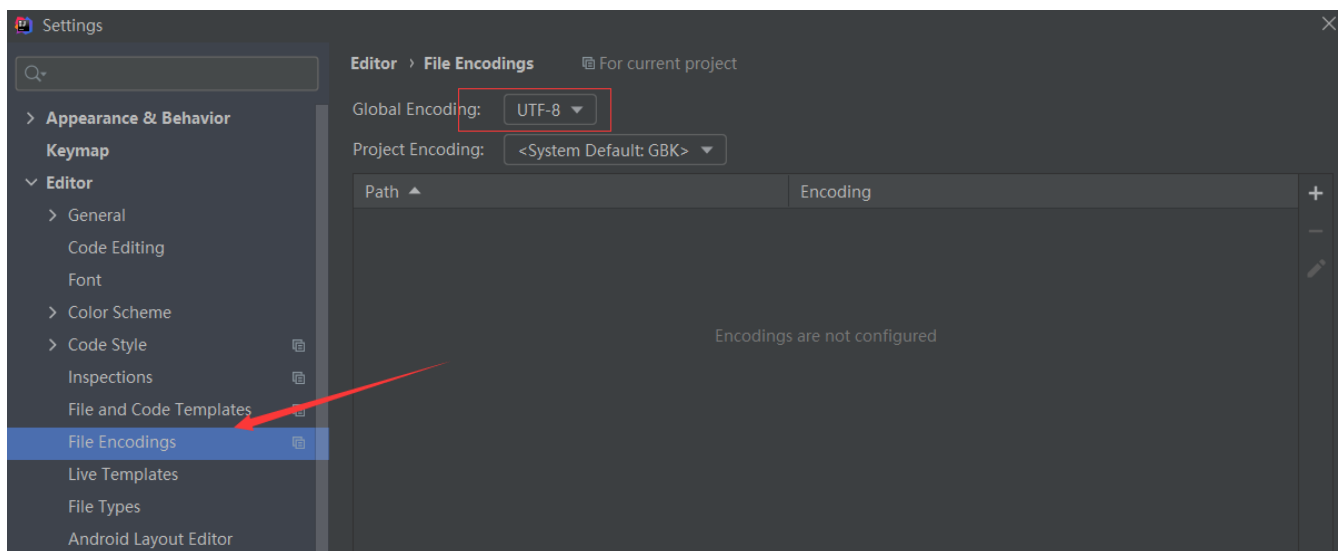
菜单栏的字体设置



代码区字体设置



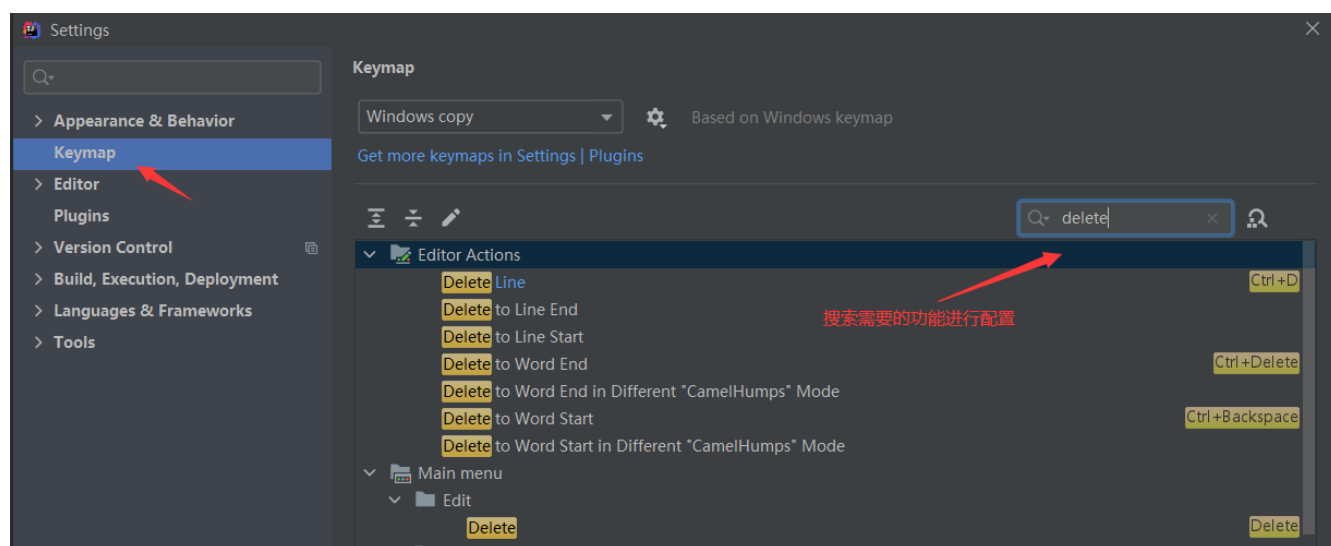
2. 编码设置



3. IDEA常用快捷键

- 1) 删除当前行，默认是 `ctrl + Y` 自己配置 `ctrl + d`
- 2) 复制当前行，自己配置 `ctrl + alt + 向下光标`
- 3) 补全代码 `alt + /`
- 4) 添加注释和取消注释 `ctrl + /` 【第一次是添加注释，第二次是取消注释】
- 5) 导入该行需要的类 先配置 `auto import`，然后使用 `alt+enter` 即可
- 6) 快速格式化代码 `ctrl + alt + L`
- 7) 快速运行程序 自己定义 `alt + R`
- 8) 生成构造器等 `alt + insert` [提高开发效率]
- 9) 查看一个类的层级关系 `ctrl + H` [学习继承后，非常有用]
- 10) 将光标放在一个方法上，输入 `ctrl + B`，可以定位到方法 [学继承后，非常有用]

快捷键设置



五、注释

1.什么是注释？

- 1.平时我们编写代码，在代码量比较少的时候，我们还可以看懂自己写的，但是当项目结构一旦复杂起来，我们就需要用到注释了
- 2.注释并不会被执行，是给我们写代码的人看的

2.Java中的三种注释

1.单行注释

```
//双斜杠代表Java的单行注释
```

2.多行注释

```
/*  
*多行注释  
*/
```

3.文档注释


```
/**
 *文档注释
 */
```

六、数据类型

每一种数据都定义了明确的数据类型，在内存中分配了不同大小的内存空间(字节)。

1.数据类型分类

Java中数据类型分为两大类：基本数据类型，引用数据类型



基本数据类型：

byte, short, int, long; float, double; char; boolean

```
public class JavaBase1 {
    public static void main(String[] args) {
        byte a=20;///占1个字节 -128~127
        short b=300;///占2个字节 -32768~32767
        int c=32800;///占4个字节
        long d=125325996665655555L;///占8个字节，要在数字后加个L
        float e=5.6F;///占4个字节 要在后面加个F
        double f=4.2;///占8个字节
        char g='a';///占两个字节
        //String s="name";不是关键字，是一个类
        boolean h=true;///占1个字节
    }
}
```

2.数据类型扩展

```
public class JavaBase2 {
    public static void main(String[] args) {
        //整数扩展
        int num1=10;///十进制
        int num2=0b11;///二进制 0b
        int num3=010;///八进制0
        int num4=0x1F;///十六进制0x
        System.out.println(num1);
    }
}
```

```

        System.out.println(num2);
        System.out.println(num3);
        System.out.println(num4);
        System.out.println("=====");
        //浮点数扩展
        float ft=0.1f;
        double db=1.0/10;
        System.out.println(ft==db);//false
        System.out.println(ft);
        System.out.println(db);
        System.out.println("=====");
        float d1=55656565656565f;
        float d2=d1+1;
        System.out.println(d1);
        System.out.println(d2);
        System.out.println(d1==d2);//true
        //float 有限 离散 舍入误差 大约 接近但不等于
        //最好完全避免使用浮点数进行比较
        System.out.println("=====");
        //字符类扩展
        char ch1='a';
        char ch2='中';
        System.out.println(ch1);
        System.out.println((int)ch1);
        System.out.println(ch2);
        System.out.println((int)ch2);
        //所有的字符本质还是数字
        //编码 unicode
        char ch3='\u0064';
        System.out.println(ch3);
        //转移字符 \t制表符 \n换行
        String s1=new String("hello");
        String s2=new String("hello");
        String s3="hello";
        String s4="hello";
        System.out.println(s1==s2);//false
        System.out.println(s3==s4);//true
        System.out.println("=====");
        //boolean类扩展
        boolean flag=true;
        if(flag==true){}
        if(flag){}
    }
}

```

3.数据类型转换

类型转换： 强制类型转换和自动类型转换。前者是低到高，后者是高到低。

从低到高为：

```
byte, short, char=>int=>long=>float=>double
```

注意

1.不能对Boolean值进行转换 2.不能把对象类型转换为不相干的类型 3.把高容量转换到低容量的时候强制转换 4.转换的时候可能存在内存溢出，或是精度问题

七、变量，常量，作用域

1.变量

在Java中，变量是程序中最基本的存储单元，用于存储数据。变量由三个基本元素组成：变量类型、变量名和变量值

```
public class Var1 {
    public static void main(String[] args) {
        //记录人的信息
        int age = 26;
        double score = 60;
        char gender = '男';
        String name = "heber";
        System.out.println("人的信息如下:");
        System.out.println(name);
        System.out.println(age);
        System.out.println(score);
        System.out.println(gender);
    }
}
```

2.常量

常量(Constant)：初始化(Initialize)后不能再改变值！不会变动的值。

```
final 常量名 = 值;
final double PI = 3.14;
```

3.作用域

作用域：类变量，实例变量，局部变量。

```
public class Var2 {
    //属性：变量
    //类变量 static
    static double salary = 5500;
    //实例变量：从属于对象；如果不自行初始化，这个类型默认值0 0.0
    //布尔值：默认false
    //除了基本类型，其他默认都是null
    String name;
    int age;
    //main方法
    public static void main(String[] args) {
        //局部变量：必须声明和初始化值
        int i = 10;
        System.out.println(i);
        //变量类型 变量名字 = new variable()
```

```

        variable variable = new variable();
        System.out.println(variable.name);
        System.out.println(variable.age);
        System.out.println(salary);
    }
}

```

八、运算符

1.运算符介绍

运算符是一种特殊的符号，用以表示数据的运算、赋值和比较等。运算符分为

1) 算术运算符 2) 赋值运算符 3) 关系运算符 [比较运算符] 4) 逻辑运算符 5) 位运算符 [需要二进制基础] 6) 三元运算符

算术运算符：+、-、*、/、%、++、- 。“%”运算符叫模运算（取余数）

赋值运算符=。例：a=10,意思为把10赋值给a。

关系运算符：>、<、>=、<=、!=instanceof 。注意：java的等于需要用两个等于号表示。“!=”符号的意思是不等于。

逻辑运算符：&&、||、!（与、或、非）

位运算符：&、|、^、~、>>、<<、>>>（了解掌握）

条件运算符：? :

扩展赋值运算符：+=、-=、*=、/=

2.算术运算符

算术运算符是对数值类型的变量进行运算的，在 Java 程序中使用的非常多。

```

package com.heber.operator_;
public class Operator1 {
    public static void main(String[] args) {
        int a = 10;
        int b = 20;
        //算术运算符：+、-、*、/、%、++、- 。“%”运算符叫模运算（取余数）
        System.out.println(a + b);
        System.out.println(a - b);
        System.out.println(a * b);
        System.out.println(a / b);
        System.out.println(a % b);
    }
}

```

3.赋值运算符

赋值运算符就是将某个运算后的值，赋给指定的变量。

```

//基本赋值运算符 =
int a = 10;

```

4.关系运算符

关系运算符的结果都是 boolean 型，也就是要么是 true，要么是 false

```

package com.heber.operator_;
public class Operator1 {
    public static void main(String[] args) {
        int a = 9;
        int b = 8;
        System.out.println(a > b); //T
        System.out.println(a >= b); //T
        System.out.println(a <= b); //F
        System.out.println(a < b); //F
    }
}

```

5.逻辑运算符

用于连接多个条件（多个关系表达式），最终的结果也是一个 boolean 值。

- 1) 短路与 &&，短路或 ||，取反 !
- 2) 逻辑与 &，逻辑或 |，^ 逻辑异或

案例演示

```

package com.heber.operator_;
public class Operator1 {
    public static void main(String[] args) {
        //短路与 和 & 案例演示
        int age = 50;
        if(age > 20 && age < 90) {
            System.out.println("ok");
        }
        // &逻辑与使用
        if(age > 20 & age < 90) {
            System.out.println("ok");
        }
    }
}

```

6.三元运算符

条件表达式 ? 表达式 1: 表达式 2; 运算规则:

1. 如果条件表达式为 true，运算后的结果是表达式 1;
2. 如果条件表达式为 false，运算后的结果是表达式 2;

```

package com.heber.operator_;
public class Operator1 {
    public static void main(String[] args) {
        int a = 10;
        int b = 99;
        // 解读
        // 1. a > b 为 false
        // 2. 返回 b--, 先返回 b 的值,然后在 b-1
        // 3. 返回的结果是 99
    }
}

```

```

        int result = a > b ? a++ : b--;
        System.out.println("result=" + result);
        System.out.println("a=" + a);
        System.out.println("b=" + b);
    }
}

```

九、流程控制

在程序中，程序运行的流程控制决定程序是如何执行的，是必须掌握的。

主要有三大流程控制语句：顺序控制；分支控制；循环控制。

本章需掌握内容：

1. 用户交互Scanner
2. 顺序控制
3. 分支控制
4. 循环控制
5. break & continue

1. 用户交互Scanner

Scanner：Scanner是一个对象，可以通过Scanner类获取用户的输入。

```

//基本语法
Scanner scanner = new Scanner(System.in);

```

使用Scanner获取用户输入实例

```

package com.heber.scanner_;
import java.util.Scanner;
public class Scanner1 {
    public static void main(String[] args) {
        //通过 Scanner类 的 next() 与 nextLine() 方法获取输入的字符串，在读取前我们一般需要使用
        hasNext() 与hasNextLine() 判断是否还有输入的数据。
        //创建实例对象，接收键盘输入
        Scanner scanner = new Scanner(System.in);
        System.out.println("使用next方法接收");
        /**
         * 1.一定要读取到有效字符后才可以结束输入
         * 2.对输入有效字符之前遇到的空白，next()方法会自动将其去掉
         * 3.只有输入有效字符后才能将其后面输入的空白作为分隔符或者结束符
         * 4.next()不能得到带有空格的字符串
         */
        if(scanner.hasNext()){
            String next = scanner.next();
            System.out.println("已经接收到数据:" + next);
        }
        /**
         * 1.以Enter为结束符，也就是说 nextLine() 方法返回的是输入回车之前的所有字符
         * 2.可以获得空白
         */
    }
}

```

```

        */
        if(scanner.hasNext()){
            String next1 = scanner.nextLine();
            System.out.println("已经接收到数据:" + next1);
        }
        //用完关闭
        scanner.close();
    }
}

```

2.顺序控制

java的执行顺序就是从上往下执行，除非特别指定。否则按顺序一句一句执行

3.分支控制

分支控制有以下几种：

- 1) 单分支 if
- 2) 双分支 if-else
- 3) 多分支 if-else if -....-else
- 4) switch多选择

1.if分支

单分支语法：

```

if(表达式){
    //如果为真需要执行的语句
}

```

双分支语法：

```

if(表达式){
    //如果为真需要执行的语句
}else{
    //否则就执行什么
}

```

多分支语法：

```

if(表达式){
    //如果为真需要执行的语句
}else if(表达式){
    //如果为真需要执行的语句
}else{
    //否则就执行什么
}

```

2.switch分支

switch语法：

```

switch(exp){
    case value:
        //执行语句
        break;//可选
    case value:
        //执行语句
        break;//可选
    default://默认执行
        //语句
}

```

4.循环控制

循环就是可以让代码可以循环的执行，循环分类：

```

//for循环：
for (int i = 0; i < 循环次数; i++) {
    //循环体
}
//while循环：
while (布尔表达式){
    //循环体
}
//do while循环：
do{
    //循环体
}while()

```

十、方法

Java方法是语句的集合，它们在一起执行一个功能。

System.out.println()，到底是个什么？

答：解释为调用系统System的标准输出对象out中的方法println

1.方法的定义

Java的方法类似于其他语言的函数，是一段用来完成特定功能的代码片段。

方法包含一个方法和一个方法体，下面是一个方法的所有部分：

修饰符：修饰符，可选，告诉编译器如何调用方法，定义该方法的访问类型。

返回值类型：方法可能会返回值。returnValueType是方法返回值的数据类型，有些方法执行所需的操作，但没有返回值，在这种情况下，returnValueType的关键字是void。

方法名：是方法的实际名称，方法和参数表共同构成方法签名。

参数类型：参数是一个占位符，当方法被调用时，传递值给参数，这个值被称为参数列表是指方法的参数类型、顺序和参数的个数，参数是可选的，

形参：在方法被调用时用于接收外界的数据。

实参：被调用方法时实际传给方法的数据。。

方法体：方法体包含具体的语句，定义该方法的功能。

方法的格式

```
修饰符 返回值类型 方法名(参数类型 参数名){  
    方法体  
    return 返回值;    //返回值要与返回值的类型对应  
}
```

2.方法的重载

- 重载就是在一个类中，有相同的函数名称，但形参不同的函数。
- 方法的重载规则：
 - 方法的名称必须相同。
 - 参数列表必须不同（个数不同、类型不同、顺序不同）。
 - 方法的返回类型可以相同，也可以不相同。
 - 仅返回类型不同不足以成为方法的重载。
- 实现理论：
 - 方法名称相同时，编译器会根据调用方法的参数的个数、参数类型等去逐个匹配，以选择对应的方法，如果匹配失败，则编译器报错。
- 语法格式

```
public static int add(double a, double b){ return a+b; }  
public static int add(int a, int b){ return a+b; }  
public static int add(int a, int b, int c){return a+b+c;}
```

3.递归

- 递归就是自己调用自己。
- 递归策略只需少量的程序就可描述出解题过程所需要的多次重复计算，大大地减少了程序的代码量。递归的能力在于用有限的语句来定义对象的无限集合。
- 递归结构包括两个部分：
 - 递归头：什么时候不调用自身方法。如果没有头，将陷入死循环。
 - 递归体：什么时候需要调用自身方法。
- 注意点：
 1. 适用于小计算。
 2. 能不使用递归尽量不使用。
 3. 自己调用自己。

十一、数组

数组可以存放多个同一类型的数据。数组也是一种数据类型，是引用类型。

1.数组声明

```
dataType[] arrayRefVar; //首选的方法  
或  
dataType arrayRefVar[]; //效果相同
```

Java语言使用new操作符来创建数组，语法如下：

```
dataType[] arrayRefVar=new dataType[arraySize];
```

```
package com.heber.array_;
public class Array2 {
    //变量的类型  变量的名字=变量的值
    //数组类型
    public static void main(String[] args) {
        int[] nums;//1.定义
        int nums2[];
        nums=new int[10];//这里面可以存放10个int类型的数字
        //3.给数组中元素赋值
        nums[0]=1;
        nums[1]=2;
        nums[2]=3;
        nums[3]=4;
        nums[4]=5;
        nums[5]=6;
        nums[6]=7;
        nums[7]=8;
        nums[8]=9;
        nums[9]=10;
        //计算所有元素的和
        int sum=0;
        //获取数组长度: arrays.length
        for (int i=0;i<nums.length;i++){
            sum+=nums[i];
        }
        System.out.println("总和为:"+sum);
    }
}
```

2.内存分析

Java内存

- 堆
存放new的对象和数组
可以被所有的线程共享，不会存放别的对象引用
- 栈
存放基本变量类型(会包含这个基本类型的具体数值)
引用对象的变量(会存放这个引用在堆里面的具体地址)
- 方法区
可以被所有的线程共享
包含了所有的class和static变量

3.三种初始化

静态初始化:

```
int[] a={1,2,3};
Man[] mans={new Man(1,1),new man(2,2)}
```

动态初始化:

```
int[] a=new int[2];
a[0]=1;
a[1]=2;
```

数组默认初始化:

数组是引用类型，它的元素相当于类的实例变量，因此数组一经分配空间，其中的每个元素也被按照实例变量相同的方式被隐式初始化。

```
package com.heber.array_;
public class Array2 {
    public static void main(String[] args) {
        //静态初始化:创建+赋值
        int[] a={1,2,3,4,5,6,7,8};
        System.out.println(a[0]);
        //Man [] mans={new Man(),new Man()};
        //动态初始化:包含默认初始化(int初始化为0,String初始化为null)
        int[] b=new int[10];
        b[0]=10;
        System.out.println(b[0]);
        System.out.println(b[8]);
    }
}
```

十二、类与对象

Java是面向对象语言，对象则是面向对象程序设计的核心，而对象与实体又是一一对应的，那么他们之间的关系是什么，又怎么理解呢？

对象：对象是指客观存在且能相互区分的个体，比如这辆车、这个人、这间房子、这顶帽子等等等等。简而言之——万物皆对象。

类：类是指一组相关的属性和行为的集合，可以看成一类事物的模板，橡皮泥的模具大家都见过吧，我们就可以把类想象成是这个模具，而通过模具刻画出来的橡皮泥则对应着这个类的对象。

对象与类之间的关系：类是对象的抽象，对象是类的实例化。

1.类

声明类变量：类名 对象名；

类对象的创建：对象名= new 构造器（参数列表）；

对象属性的赋值：属性：对象名.属性名、对象名.属性名=数据；

属性和方法的调用：对象名.属性名；对象名.方法名（）；

```

package com.heber.class_;
public class Students {
    public static void main(String[] args) {
        //类:实例化
        Student student = new Student();
        student.name = "heber";
        student.age = 20;
        System.out.println(student.name + student.age);
    }
}
class Student {
    //属性字段
    String name;
    int age;
    //方法
    public void study() {
        System.out.println("学习");
    }
}

```

2.构造方法

构造方法：

- 1.实例在创建时通过new操作符会调用其对应的构造方法，构造方法用于初始化实例；
- 2.没有定义构造方法时，编译器会自动创建一个默认的空参构造；
- 3.可以定义多个构造方法，编译器根据参数自动判断；
- 4.可以在一个构造方法内部调用另一个构造方法，便于代码复用。

构造方法的作用：

给对象的属性进行初始化

构造方法的格式：

- 构造方法的方法名必须与类名相同
- 构造方法没有返回类型，也不能定义为void
- 没有具体的返回值

```

package com.heber.class_;
public class Students {
    public static void main(String[] args) {
        //类:实例化有参构造，对象
        Student student = new Student("heber");
        System.out.println(student.name);
    }
}
class Student {
    //属性字段
    String name;
    //一个类就算什么都不写也会有一个空参构造方法
    //构建空参构造
}

```

```

    public Student() {
    }
    //构建有参构造方法
    public Student(String name) {
        /**
         * this关键字
         * this关键字是特殊的引用，指向当前对象
         * this关键字的使用方法
         * 如果发生局部变量和成员变量命名冲突时，可以通过this.成员变量名的方式区分成员变量和局部变量
         */
        this.name = name;
    }

    //方法
    public void study() {
        System.out.println("学习");
    }
}

```

十三、封装，继承，多态

1.封装

- 封装是指隐藏对象的属性和实现细节，仅对外提供公共访问方式
- 好处
 - 隐藏实现细节，提供公共的访问方式
 - 提高了代码的复用性
 - 提高安全性
- 原则：将不需要对外提供的内容都隐藏起来，提供公共方法对其访问。

```

package com.heber.class_;
public class Package1 {
    public static void main(String[] args) {
        Student1 student1 = new Student1();
        //由于私有化,所有不能直接调用,需要设置get和set
        //System.out.println(student1.name);
        //私有化后可以使用set进行赋值
        student1.setName("heber");
        //设置get和set后需要使用get来进行获取
        String name = student1.getName();
        System.out.println(name);
    }
}
class Student1 {
    /**
     * Java 修饰符 用于控制类、属性、方法和构造函数的访问权限和行为。它们可以分为两组:
     * 访问修饰符:
     * public: 意味着代码对所有类可访问。
     * private: 意味着代码只能在声明的类内部访问。
     * default: 意味着代码只能在同一包中访问。
     * protected: 意味着代码在同一包和子类中可访问。
     * 非访问修饰符:
    */
}

```

```

    * final: 意味着类不能被继承, 属性和方法不能被重写。
    * static: 意味着属性和方法属于类, 而不属于对象。
    * abstract: 意味着类不能用于创建对象, 方法没有主体, 必须由子类提供。
    * transient: 意味着在序列化包含它们的对象时, 属性和方法将被跳过。
    * synchronized: 意味着方法一次只能由一个线程访问。
    * volatile: 意味着属性的值不会在本地线程缓存, 总是从“主内存”读取。
    */
//属性字段,私有化后就不能直接进行调用
private String name;
//构建无参构造
public Student1() {
}
//构建有参构造方法
public Student1(String name) {
    this.name = name;
}
//设置set和get
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
}

```

2.继承

在Java中, 可以从一个类继承属性和方法到另一个类, 继承就是子类继承父类的特征和行为, 使得子类对象(实例)具有父类的实例域和方法, 或子类从父类继承方法, 使得子类具有父类相同的行为。要从一个类继承, 使用 `extends` 关键字

演示分别创建一个人类和一个学生类

人类

```

package com.heber.class_;
public class Person {
    public String name;
}

```

学生类

```

package com.heber.class_;
public class Student2 extends Person{
    //使用继承之后子类就能使用父类属性和方法
}

```

super

1. super关键字: 我们可以通过super关键字来实现对父类成员的访问, 用来引用当前对象的父类。

- super 表示使用它的类的父类。super 可用于:
- 调用父类的构造方法;

- 调用父类的方法（子类覆盖了父类的方法时）；
- 访问父类的数据域（可以这样用但没有必要这样用）。

演示：父类

```
package com.heber.class_;
public class Person {
    public String name = "heber";
}
```

子类

```
package com.heber.class_;
public class Student2 extends Person{
    public String name = "heber";
    public void test(){
        //这个就会直接调用本类的
        System.out.println(name);
        //使用了super就会调用父类的name
        System.out.println(super.name);
    }
}
```

重写

重载和重写的区别 重载：发生在同一个类中，方法名相同，参数类型、参数个数、顺序可以不同 重写：发生在父子类中，方法名，参数必须相同，返回值类型小于等于父类，抛出的异常范围小于等于父类，访问修饰符范围大于等于父类，如果父类方法访问修饰符为private，则子类不可重载该方法。

父类

```
package com.heber.class_;
public class Person {
    public static void test(){
        System.out.println("重写是针对方法的重写");
    }
}
```

子类

```
package com.heber.class_;
public class Student2 extends Person{
    public static void test(){
        System.out.println("重写是针对方法的重写,与属性无关");
    }
}
```

3.多态

多态是同一个行为具有多个不同表现形式的或形态的能力。多态是对象多种表现形式的体现。

多态存在的三个必要条件:

- 继承
- 重写
- 父类的引用指向子类对象

父类:

```
package com.heber.class_;
public class Person {
    public String name;
}
```

子类:

```
package com.heber.class_;
public class Student2 extends Person{
    public String name;
}
```

main:

```
package com.heber.class_;
public class main1 {
    public static void main(String[] args) {
        //new的是学生，但是指向的是Person
        Person student2 = new Student2();
    }
}
```

十四、接口

接口技术用于描述类具有什么功能，但并不给出具体实现，当某个类要使用接口时，再去实现接口中的这些方法。类需要遵从接口中描述的统一规则进行定义，所以，接口是对外提供的一组规则，标准。

```
//①定义接口要用到关键字interface，格式如下：
interface 接口名 {
}
//②类和接口之间不再是继承关系，而是实现关系，用implements关键字表示。如下：
class 类名 implements 接口名 {
}
```

十五、异常

在 Java 代码执行期间，可能会发生各种错误，包括程序员编码错误、用户输入错误以及其他不可预料的情况。

当错误发生时，Java 通常会停止并生成错误消息，这个过程称为抛出异常。

1.异常 - Try...Catch

try 语句允许您定义一段代码块，并在其中测试是否发生错误。

catch 语句允许您定义一段代码块，当 try 块中发生错误时执行该代码块。

try 和 catch 关键字成对使用，语法如下：

```
try {
    // 要尝试的代码块
} catch (Exception e) {
    // 处理错误的代码块
}
```

2. finally 语句

finally 语句允许您在 try...catch 语句之后执行一段代码，无论 try 块中是否发生错误。

```
public class Error1 {
    public static void main(String[] args) {
        try {
            int[] myNumbers = {1, 2, 3};
            System.out.println(myNumbers[10]); // 数组越界错误
        } catch (Exception e) {
            System.out.println("Something went wrong.");
        } finally {
            System.out.println("The 'try catch' is finished.");
        }
    }
}
```

3. throw 关键字

throw 关键字允许您创建自定义异常。

throw 关键字通常与异常类型一起使用。Java 中提供了许多异常类型，例如：

- ArithmeticException
- FileNotFoundException
- ArrayIndexOutOfBoundsException
- SecurityException

```
public class Error1 {
    static void checkAge(int age) {
        if (age < 18) {
            throw new ArithmeticException("Access denied - You must be at least 18 years old.");
        } else {
            System.out.println("Access granted - You are old enough!");
        }
    }
    public static void main(String[] args) {
        checkAge(15); // 年龄小于 18 岁
    }
}
```

总结：

- try...catch 语句用于处理 Java 代码中的异常。

- finally 语句用于确保代码在任何情况下都会执行。
- throw 关键字用于创建自定义异常。

十六、多线程

在程序中同时运行多个线程

Java中的多线程指的是在程序中同时运行多个线程，每个线程都是独立运行的，具有自己的执行路径、栈、寄存器等资源，并且可以同步地访问共享数据。这种机制的主要优势在于能够充分利用多核处理器，同时提升程序的并发性和响应速度。多线程允许在一个程序中同时执行多个任务，从而提高系统的资源利用率和响应性能。在传统的单线程编程模型中，程序按照顺序执行，一次只处理一个任务，这种方式在某些情况下可能会导致效率低下或者无法满足需求。而多线程通过将任务拆分为多个子任务，并且在不同的线程上同时执行，从而实现并发处理。¹²

Java中实现多线程的方式主要有两种：

1. **继承Thread类**：通过自定义类继承Thread类后，该类就成为一个线程类，线程代码必须编写在run()方法中。要开启新的线程需要调用Thread类的start()方法，该方法自动开启一个新的线程并自动执行run方法中的内容。
2. **实现Runnable接口**：通过实现Runnable接口并重写run方法，然后创建对象并调用start()方法启动线程。这种方式要通用一些，可以避免单继承的限制，方便共享资源。

多线程的启动顺序不一定是线程执行的顺序，各个线程之间是抢占CPU资源执行的，有可能出现与启动顺序不一致的情况。CPU的调用策略由操作系统来决定，但操作系统只能决定CPU的使用策略不能控制实际获得CPU执行权的程序。

1.继承Thread类

```
package com.heber.thread_;
//创建线程方式一：继承Thread类，重写run方法，调用start启动线程
public class Thread1 extends Thread{
    //重写run方法
    @Override
    public void run() {
        for (int i = 0; i < 20; i++) {
            System.out.println("=====" + i);
        }
    }
    public static void main(String[] args) {
        //打开另外一个线程
        Thread1 thread1 = new Thread1();
        thread1.start();
        //main方法的主线程
        for (int i = 0; i < 20; i++) {
            System.out.println("-----" + i);
        }
    }
}
```

2.实现Runnable接口

```
package com.heber.thread_;
/**
 * 实现Runnable接口
 * (1) 重写run方法，执行线程需要丢入runnable接口实现类。调用start()方法
```

```

* (2) 推荐使用：很好的避免了单继承的局限性，灵活方便，同一个对象被多个线程使用
* (3) 代码展示
*/
public class Runnable1 implements Runnable{
    //重写run方法
    @Override
    public void run() {
        for (int i = 0; i < 20; i++) {
            System.out.println("======" + i);
        }
    }
    public static void main(String[] args) {
        //创建Runnable实现类
        Runnable1 runnable1 = new Runnable1();
        //创建线程对象
        Thread thread = new Thread(runnable1);
        thread.start();
        //main方法的主线程
        for (int i = 0; i < 20; i++) {
            System.out.println("-----" + i);
        }
    }
}

```

3.线程状态

五大状态:

- 创建状态
- 就绪状态
- 运行状态
- 阻塞状态
- 死亡状态

```

package com.heber.thread_;
public class Runnable1 implements Runnable{
    // 设置标志位
    private boolean stop = true;
    @Override
    public void run() {
        int i = 0;
        while (stop) {
            System.out.println("Run...." + i);
        }
    }
    // 写一个标志位，用来停止某个线程
    public void stop() {
        this.stop = false;
    }
    public static void main(String[] args) {
        Runnable1 runnable1 = new Runnable1();
        new Thread(runnable1).start();
        for (int i = 0; i < 500; i++) {

```

```

        System.out.println("main...." + i);
        if (i == 490) {
            // 调用stop方法停止线程
            runnable1.stop();
            System.out.println("run线程停止了");
        }
    }
}
}
}

```

十七、注解，反射

1.注解的定义

Java注解又称Java标注，是在JDK5时引入的新特性，注解（也被称为元数据）。

Java注解它提供了一种安全的类似注释的机制，用来将任何的信息或元数据（metadata）与程序元素（类、方法、成员变量等）进行关联。

Java注解是附加在代码中的一些元信息，用于一些工具在编译、运行时进行解析和使用，起到说明、配置的功能。

2.注解与注释的区别

注解是对代码的解释和说明，其目的是提高程序代码的可读性。注解是可以被编译器打包进入class文件。

注释只存在于java源代码中，对于编译和运行没有任何作用，也不会被编译到class文件中。注释是给程序员看的，编译器与JVM都不需要知道它。

3.注解的应用

- 1.生成文档这是最常见的，也是java 最早提供的注解;
- 2.在编译时进行格式检查，如@Override放在方法前，如果你这个方法并不是覆盖了超类方法，则编译时就能检查出;
- 3.跟踪代码依赖性，实现替代配置文件功能，比较常见的是spring 2.5 开始的基于注解配置,作用就是减少配置;
- 4.在反射的 Class, Method, Field 等函数中，有许多于 Annotation 相关的接口,可以在反射中解析并使用 Annotation。

```

package com.heber.thread_;
public class Zhujie extends Object{
    // @Override他就是注解,这里标记这是一个重写的方法
    @Override
    public String toString() {
        return super.toString();
    }
}

```

4.反射

JAVA反射机制是在运行状态中，对于任意一个类，都能够知道这个类的所有属性和方法；对于任意一个对象，都能够调用它的任意一个方法和属性；这种动态获取的信息以及动态调用对象的方法的功能称为java语言的反射机制。要想解剖一个类,必须先要获取到该类的字节码文件对象。而解剖使用的就是Class类中的方法,所以先要获取到每一个字节码文件对应的Class类型的对象。

反射就是把java类中的各种成分映射成一个个的Java对象

javaweb

一、基本概念

1.前言

web: 网页的意思，比如百度www.baidu.com

web分为:

1. 静态web: html, css (提供给所有人看的数据，始终不会发生变化)
2. 动态web: Servlet/JSP, ASP, PHP, 百度等几乎所有的网站

2.web应用程序

web应用程序: 可以提供浏览器访问的程序

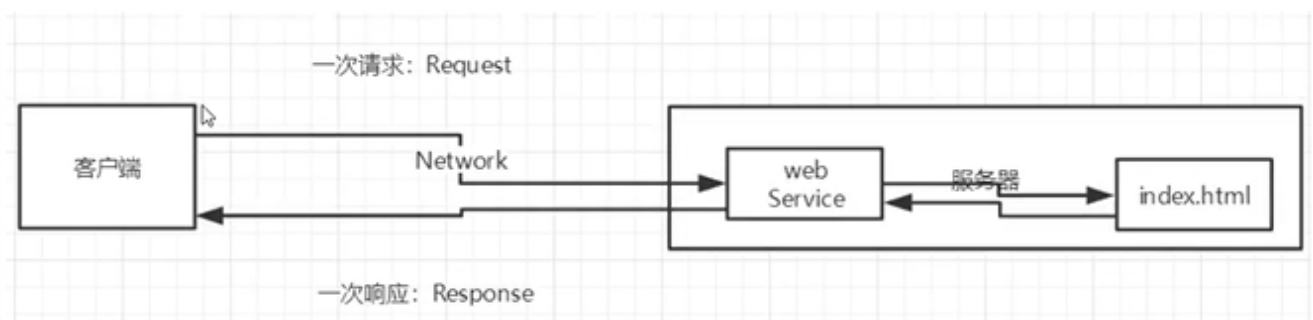
- a.html、b.html等多个web资源，这些web资源可以被外界访问，对外界提供服务。
- 任何一个页面或者资源，都存在于这个世界的某一个角落的计算机上。
- URL: 统一资源定位符。
- web资源会被放在同一个文件夹下，web应用程序，这个不能直接使用需要依赖—>Tomcat: 服务器。
- web应用程序编写完毕后，若想提供给外界访问: 需要一个服务器来统一管理。

一个web应用由多部分组成 (静态web, 动态web) html, css, js jsp, servlet Java程序 jar包 配置文件 (Properties) web应用程序编写完毕后，若想提供给外界访问: 需要一个服务器来统一管理;

3.静态web

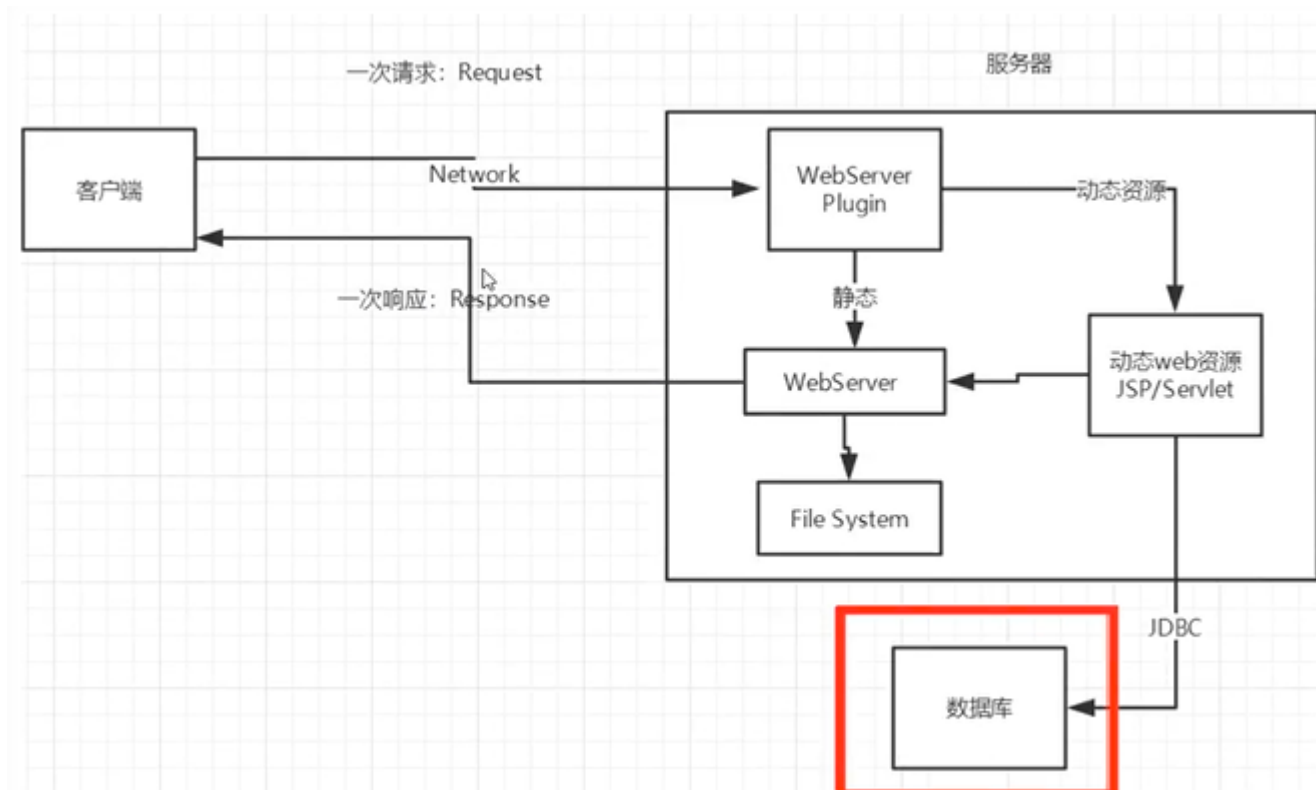
.htm, .html,这些都是网页的后缀，如果服务器上一直存在这些东西，我们就可以直接进行读取。通路;

静态web存在的缺点 Web页面无法动态更新，所有用户看到都是同一个页面 轮播图，点击特效: 伪动态 JavaScript [实际开发中，它用的最多] VBScript 它无法和数据库交互 (数据无法持久化，用户无法交互)



4.动态web

缺点: 假如服务器的动态web资源出现了错误，我们需要重新编写我们的后台程序,重新发布; 停机维护 优点: Web页面可以动态更新，所有用户看到都不一定是同一个页面 它可以与数据库交互 (数据持久化: 注册，商品信息，用户信息.....)



二、web服务器

1.什么是web服务器

web服务器是一种被动的操作，用来处理用户的一些请求和给用户一些响应信息。

常见的web服务器：

nginx, iis, http, Tomcat等。

JSP/Servlet: B/S：浏览和服务

C/S：客户端和服务

- sun公司主推的B/S架构
- 基于Java语言的（所有的大公司，或者一些开源的组件，都是用java写的）
- 可以承载三高问题带来的影响
- 语法像ASP，ASP->JSP，加强市场强度

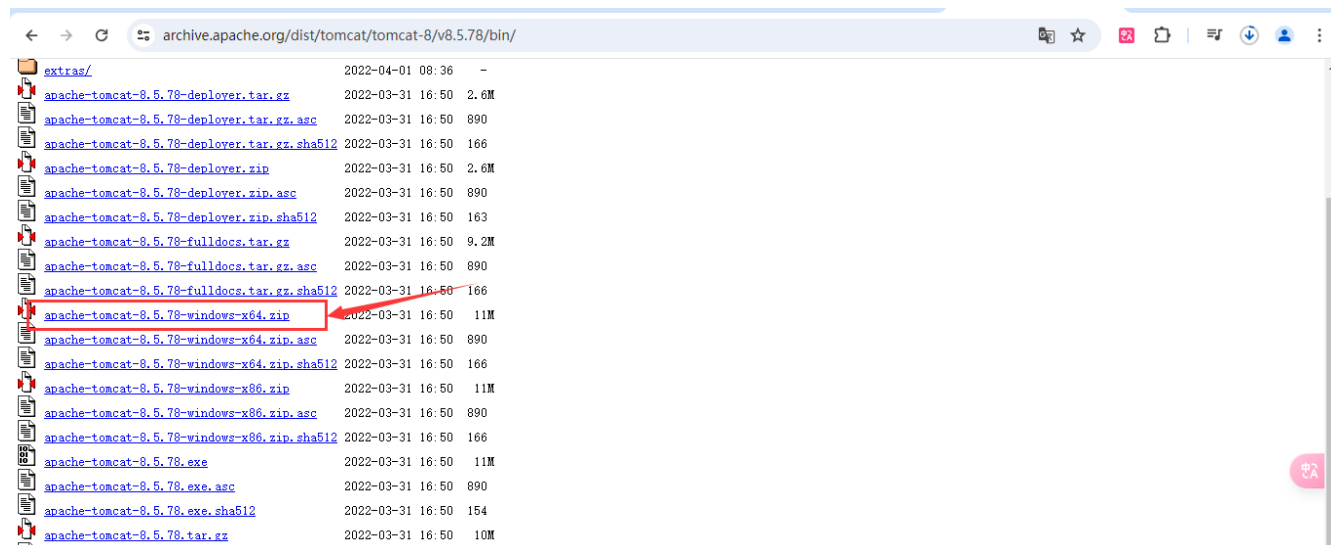
2.tomcat

Tomcat是Apache软件基金会（Apache Software Foundation）的Jakarta 项目中的一个核心项目，由Apache、Sun和其他一些公司及个人共同开发而成。由于有了Sun 的参与和支持，最新的Servlet 和JSP规范总是能在Tomcat 中得到体现，Tomcat 5支持最新的Servlet 2.4和JSP2.0规范。Tomcat服务器是一个免费的开放源代码的Web应用服务器，属于轻量级应用服务器，在中小型系统和并发访问用户不是很多的场合下被普遍使用，是开发和调试JSP程序的首选。对于一个初学者来说，可以这样认为，当在一台机器上配置好Apache服务器，可利用它响应HTML（标准通用标记语言下的一个应用）页面的访问请求。实际上Tomcat是Apache服务器的扩展，但运行时它是独立运行的，所以当你运行tomcat时，它实际上作为一个与Apache 独立的进程单独运行的。

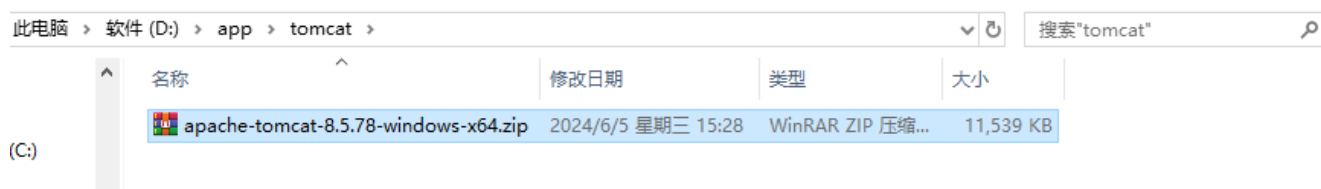
Tomcat是Java中最著名的开源Web容器，简单易用且稳定性良好

1.下载和安装tomcat

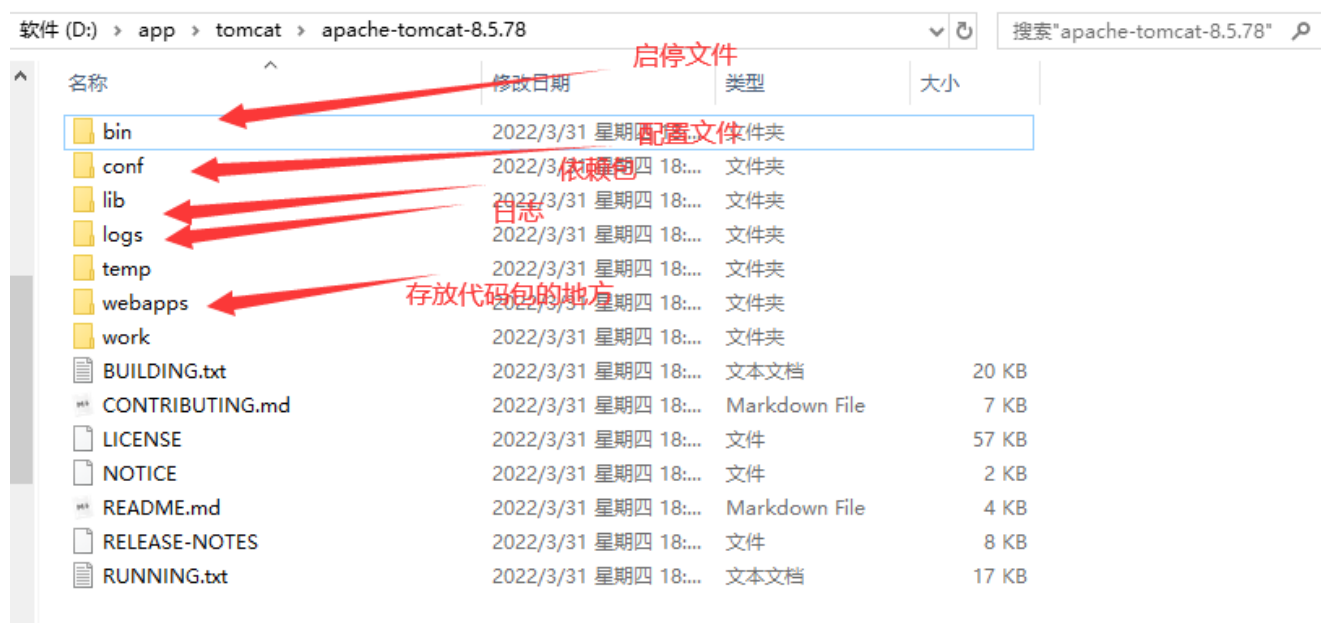
tomcat官网: <https://tomcat.apache.org/>



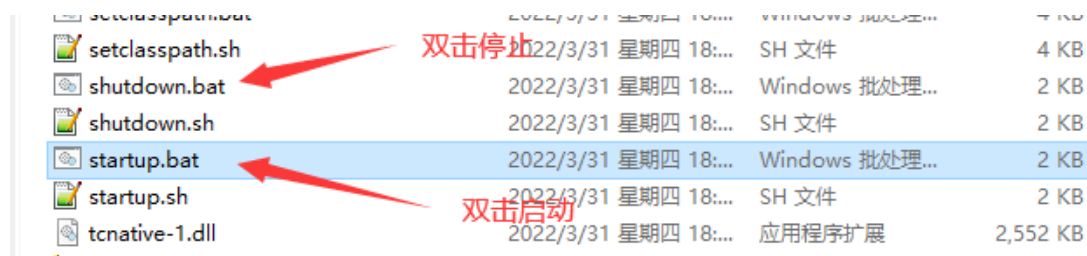
下载后解压即可使用



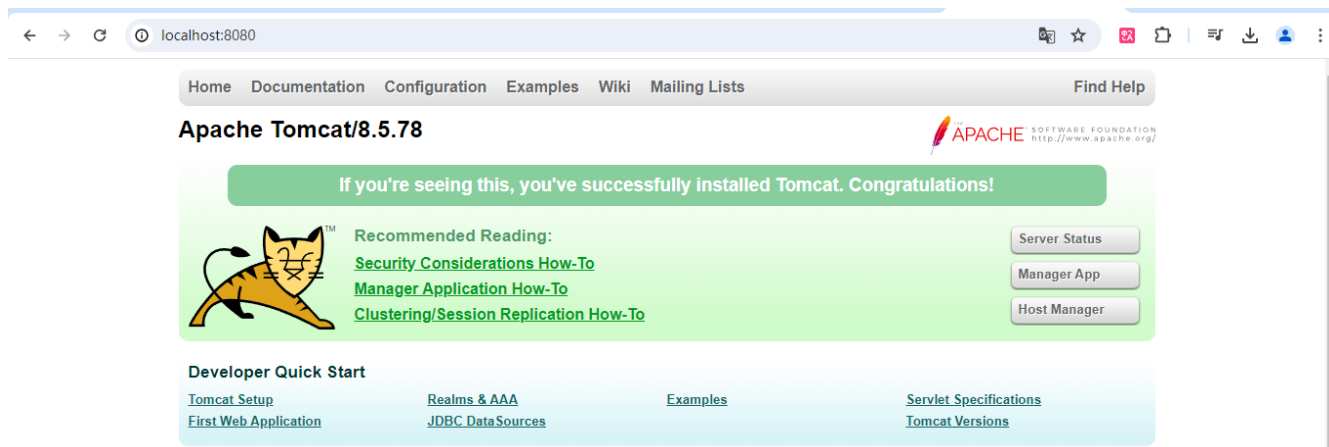
2.目录结构



想要启动和停止tomcat直接进bin文件夹



启动后浏览器访问ip:端口，出现以下界面代表完成



tomcat配置文件



端口修改

```
--> Define a non-SSL/TLS HTTP/1.1 Connector on port 8080
-->
<Connector port="8080" protocol="HTTP/1.1"
           connectionTimeout="20000"
           redirectPort="8443" />
```

tomcat的端口

主机和工作目录修改

```
<Host name="localhost" appBase="webapps"
      unpackWARs="true" autoDeploy="true">
```

主机地址

工作目录

```
<!-- SingleSignOn valve, share authentication between web applications
      Documentation at: /docs/config/valve.html -->
```

3.配置tomcat环境变量

注意：在配置tomcat环境变量时，必须有JAVA_HOME(java)的环境变量

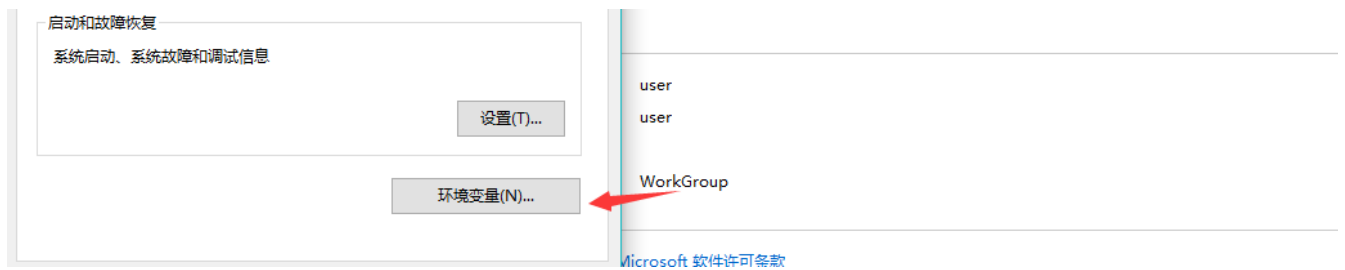
如果不配置环境变量可能会出现：闪退，乱码问题



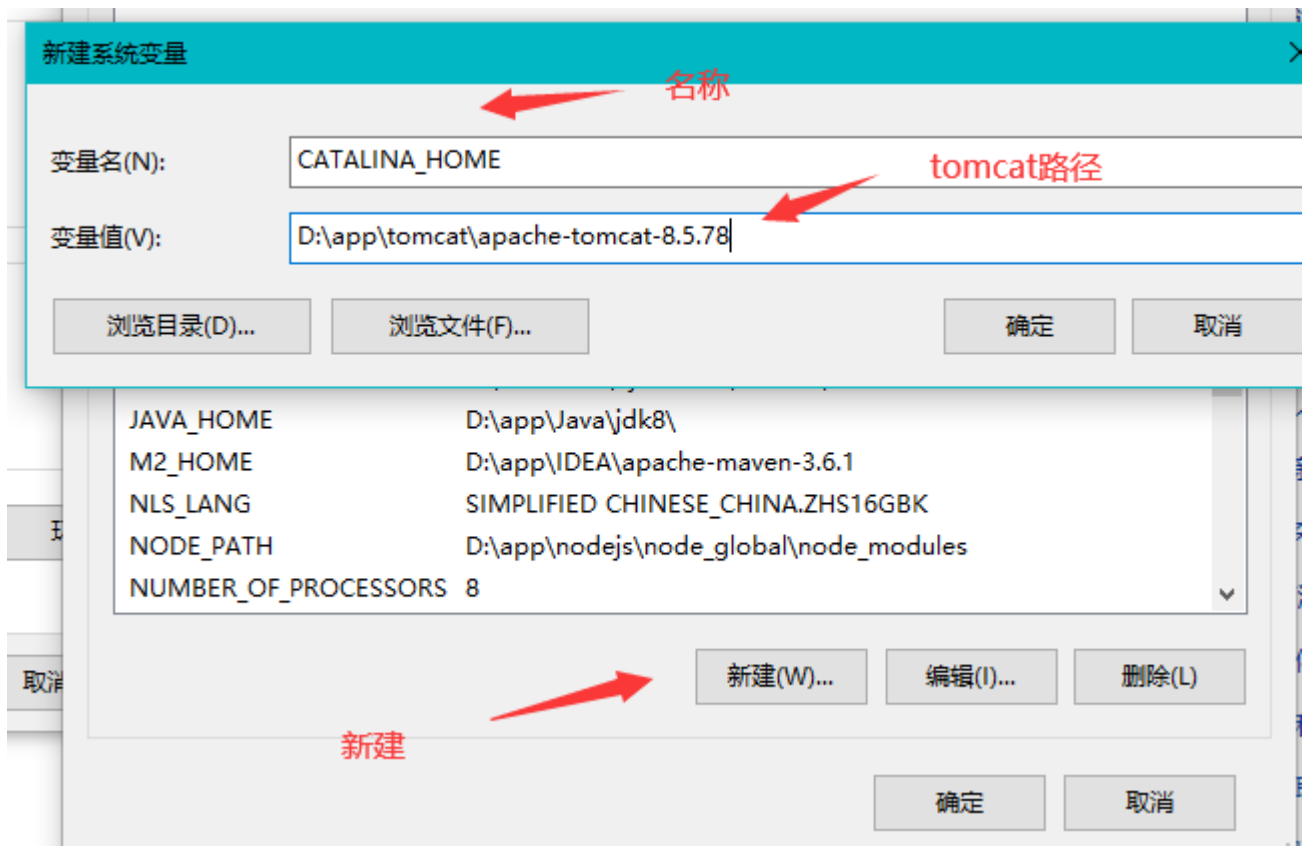
高级系统设置



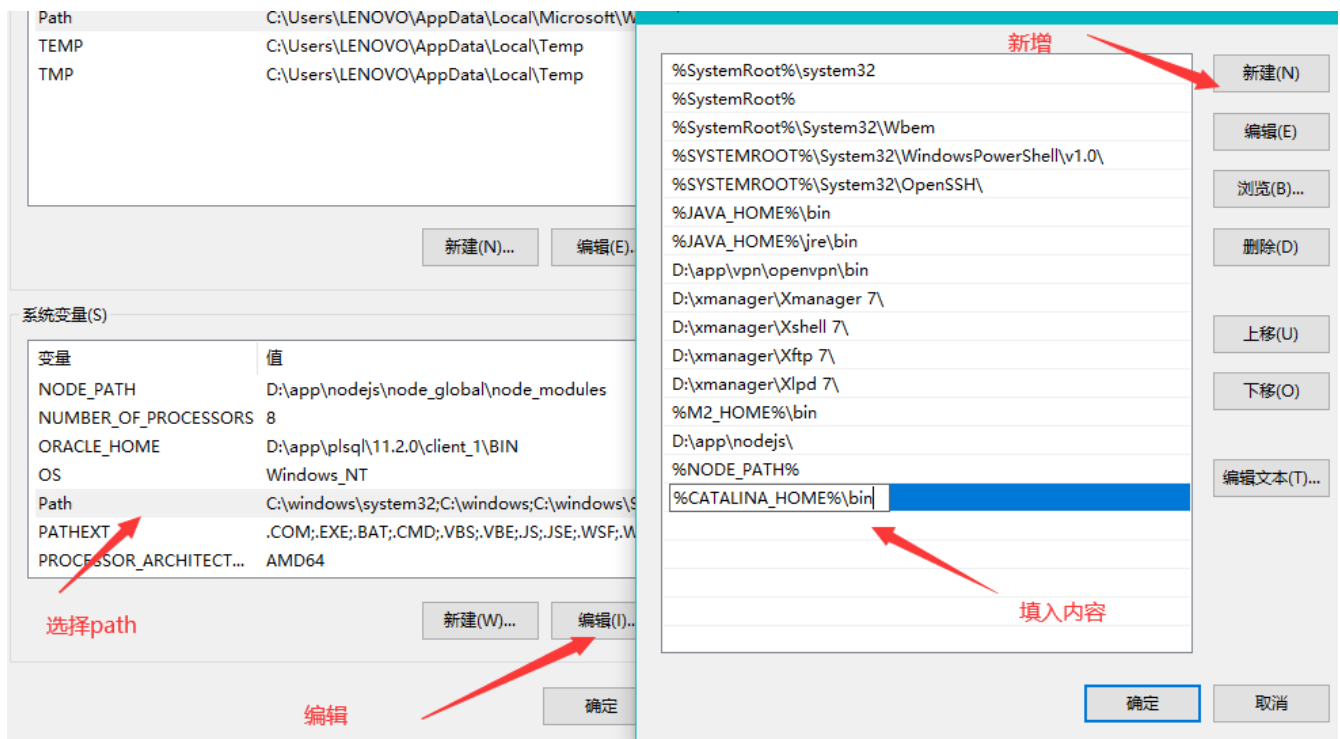
环境变量



新增系统环境变量



新增path

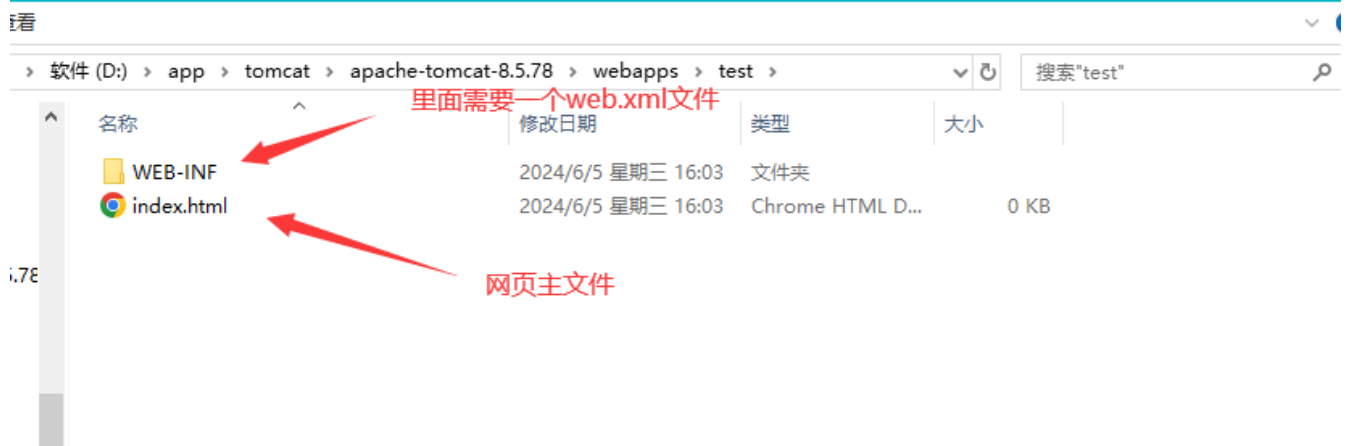


4.发布一个web网站

将自己写的网站，放到服务器（Tomcat）中指定的web应用的文件夹（webapps）下，就可以访问了

网站应该有的结构

```
--webapps : Tomcat服务器的web目录
  -ROOT
  -test : 网站的目录名
    - WEB-INF
      -classes : java程序
      -lib: web应用所依赖的jar包
      -web.xml : 网站配置文件
    - index.html 默认的首页
```



3.Http

(超文本传输协议) 是一个简单的请求-响应协议，它通常运行在TCP之上。

- 文本: html, 字符串, ...
- 超文本: 图片, 音乐, 视频, 定位, 地图.....
- 端口:80

Https:安全的

两个时代

- http1.0
 - HTTP/1.0: 客户端可以与web服务器连接后，只能获得一个web资源，断开连接
- http2.0
 - HTTP/1.1: 客户端可以与web服务器连接后，可以获得多个web资源。

Http请求

客户端-发请求 (Request) -服务器

```
Request URL:https://www.baidu.com/    请求地址
Request Method:GET    get方法/post方法
Status Code:200 OK    状态码: 200
Remote (远程) Address:14.215.177.39:443
Accept:text/html
Accept-Encoding:gzip, deflate, br
Accept-Language:zh-CN,zh;q=0.9    语言
Cache-Control:max-age=0
Connection:keep-alive
```

- 请求行中的请求方式：GET
- 请求方式：Get,Post,HEAD,DELETE,PUT,TRACT....
 - get: 请求能够携带的参数比较少，大小有限制，会在浏览器的URL地址栏显示数据内容，不安全，但高效
 - post:请求能够携带的参数没有限制，大小没有限制，不会在浏览器的URL地址栏显示数据内容，安全，但不高效

消息头:

```
Accept: 告诉浏览器，它所支持的数据类型
Accept-Encoding: 支持哪种编码格式  GBK  UTF-8  GB2312  ISO8859-1
Accept-Language: 告诉浏览器，它的语言环境
Cache-Control: 缓存控制
Connection: 告诉浏览器，请求完成是断开还是保持连接
HOST: 主机..../.

```

响应状态码:

```
200: 请求响应成功200
3xx: 请求重定向·重定向: 你重新到我给你新位置去
4xx: 找不到资源404·资源不存在
5xx: 服务器代码错误 500 502:网关错误

```

三、Maven

1.什么是maven?

- 在Javaweb开发中，需要使用大量的jar包，我们手动去导入
- 如何能够让一个东西自动帮我导入和配置这个jar包，由此，Maven诞生了！Maven会规定好你该如何去编写我们Java代码，必须要按照这个规范来；

2.下载安装Maven

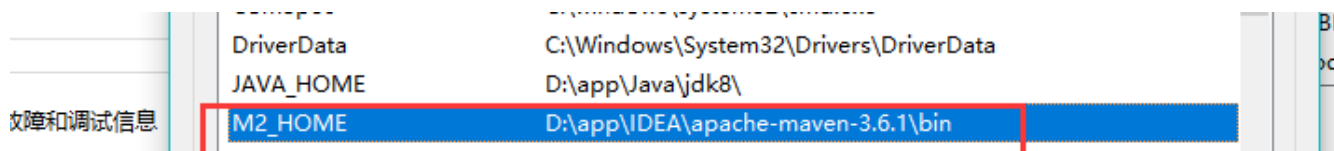
官网地址: <https://maven.apache.org/>

下载后解压即可使用

3.maven环境变量配置

在我们的系统环境变量中配置如下配置:

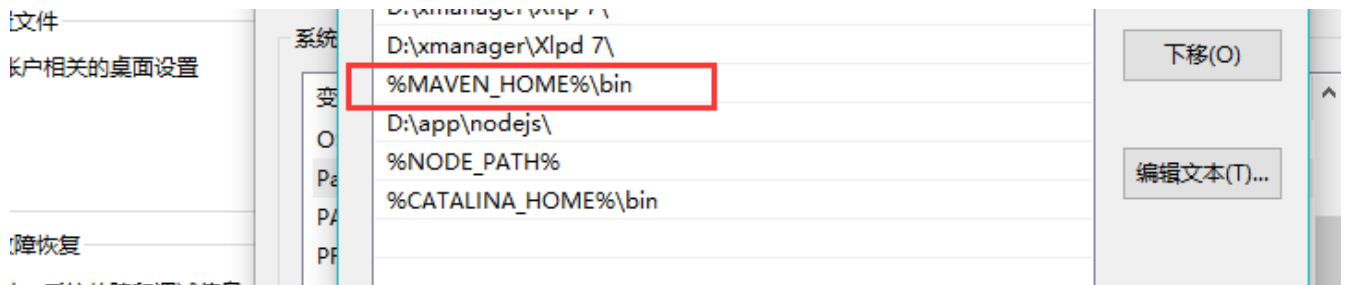
M2_HOME maven目录下的bin目录



MAVEN_HOME maven的目录

ComSpec	C:\windows\system32\cmd.exe
DriverData	C:\Windows\System32\Drivers\DriverData
JAVA_HOME	D:\app\Java\jdk8\
MAVEN_HOME	D:\app\IDEA\apache-maven-3.6.1
NLS_LANG	SIMPLIFIED CHINESE_CHINA.ZHS16GBK
NODE_PATH	D:\app\nodejs\node_global\node_modules

在系统的path中配置%MAVEN_HOME%\bin

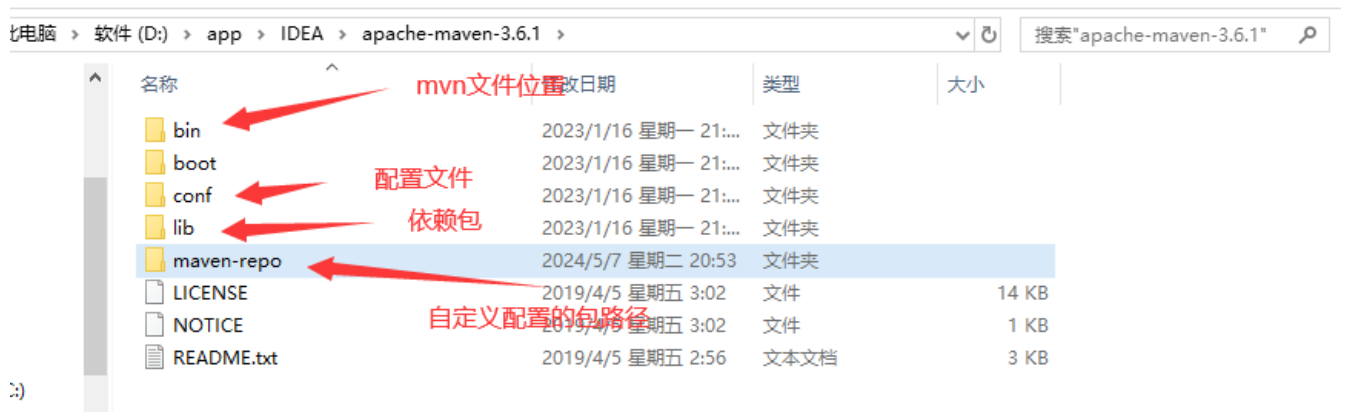


最后测试

```
C:\Users\LENOVO>mvn -version
Apache Maven 3.6.1 (d66c9c0b3152b2e69ee9bac180bb8fcc8e6af555; 2019-04-05T03:00:29+08:00)
Maven home: D:\app\IDEA\apache-maven-3.6.1\bin\..
Java version: 1.8.0_111, vendor: Oracle Corporation, runtime: D:\app\Java\jdk8\jre
Default locale: zh_CN, platform encoding: GBK
OS name: "windows 10", version: "10.0", arch: "amd64", family: "windows"

C:\Users\LENOVO>
```

4.maven目录结构



5.maven配置

配置maven包位置

```
<!-- Default: ${user.home}/.m2/repository -->
<localRepository>path/to/local/repo</localRepository>
-->
<localRepository>D:\app\IDEA\apache-maven-3.6.1\maven-repo</localRepository>
<!-- interactiveMode -->
| This will determine whether maven prompts you when it needs input. If set to false,
| maven will use a sensible default value, perhaps based on some other setting, for
| the parameter in question.
|
| Default: true
```

配置镜像加速

```

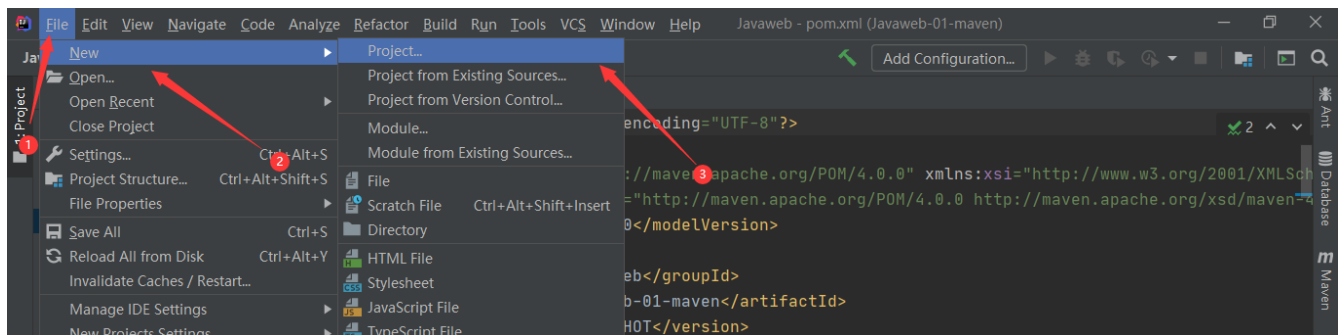
-->
<mirror>
  <id>nexus</id>
  <name>aliyun</name>
  <url>http://maven.aliyun.com/nexus/content/groups/public/</url>
  <mirrorOf>central</mirrorOf>
</mirror>
</mirrors>

```

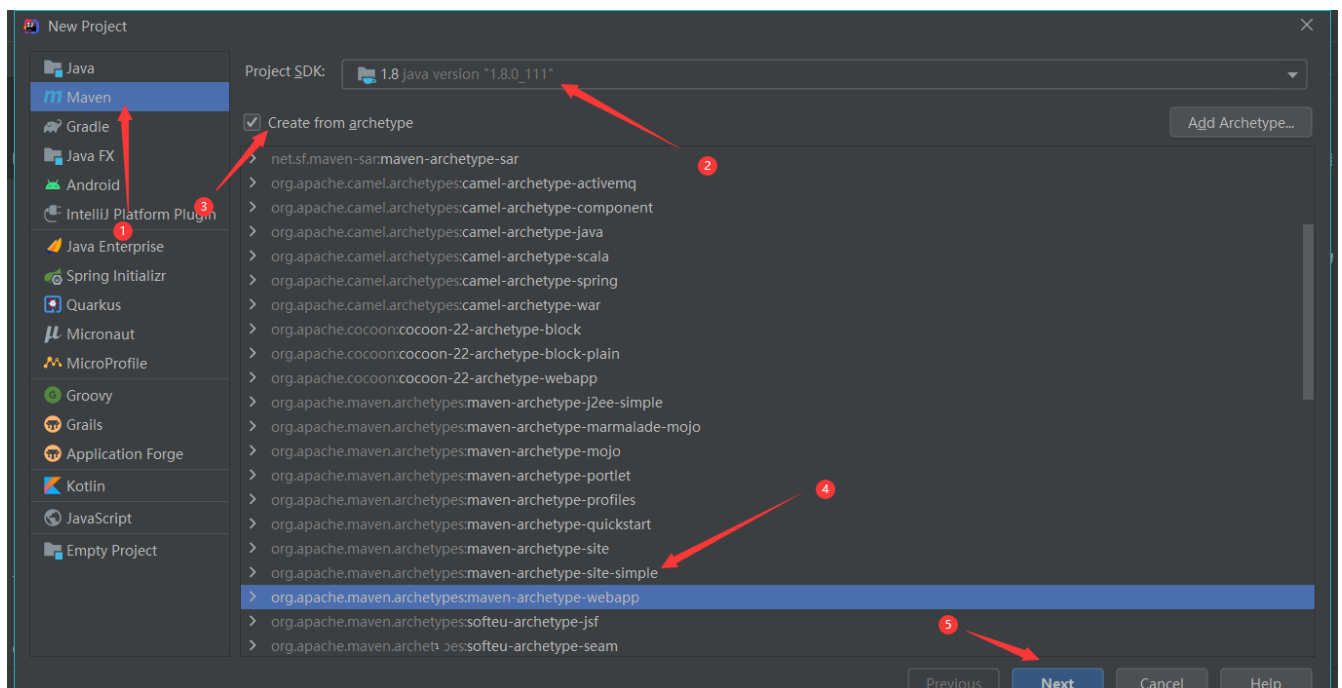
6.在idea中使用maven

创建一个maven项目

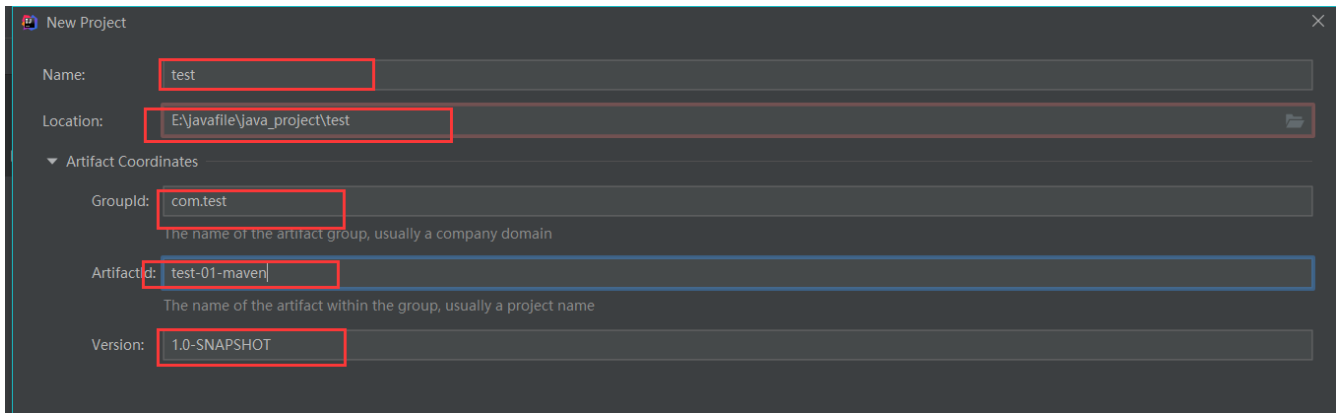
file---->new----->project



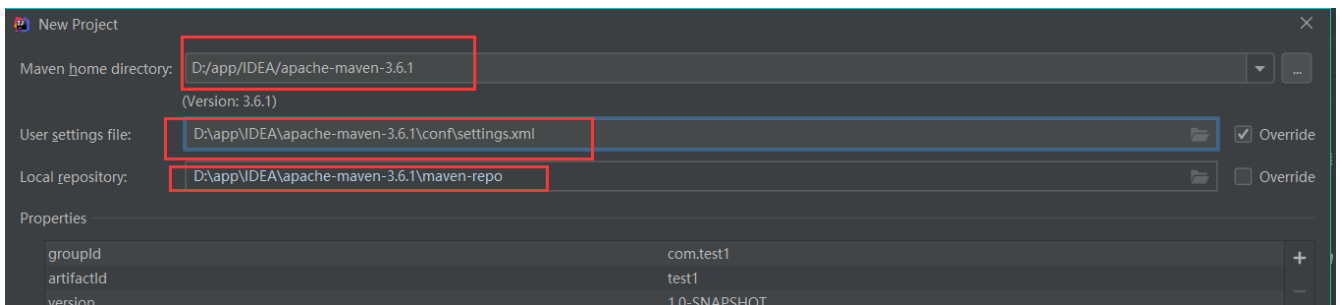
选择maven进行创建



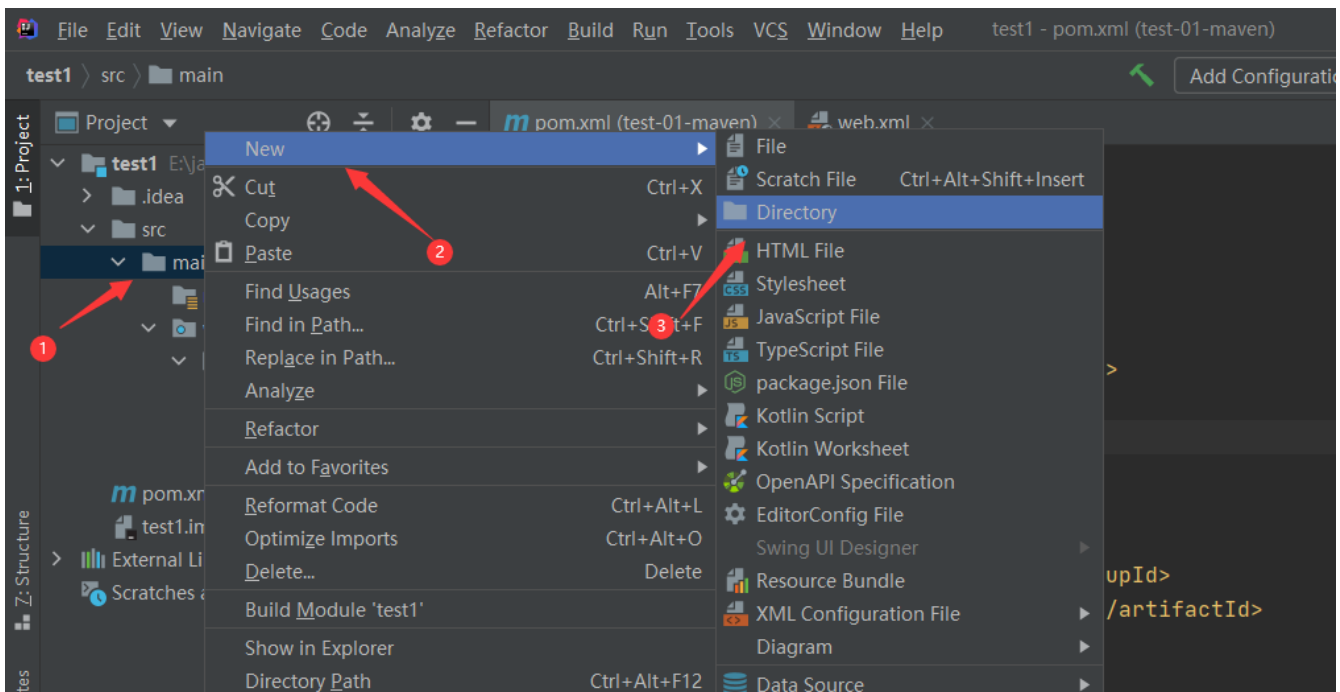
填写项目名称路径等信息



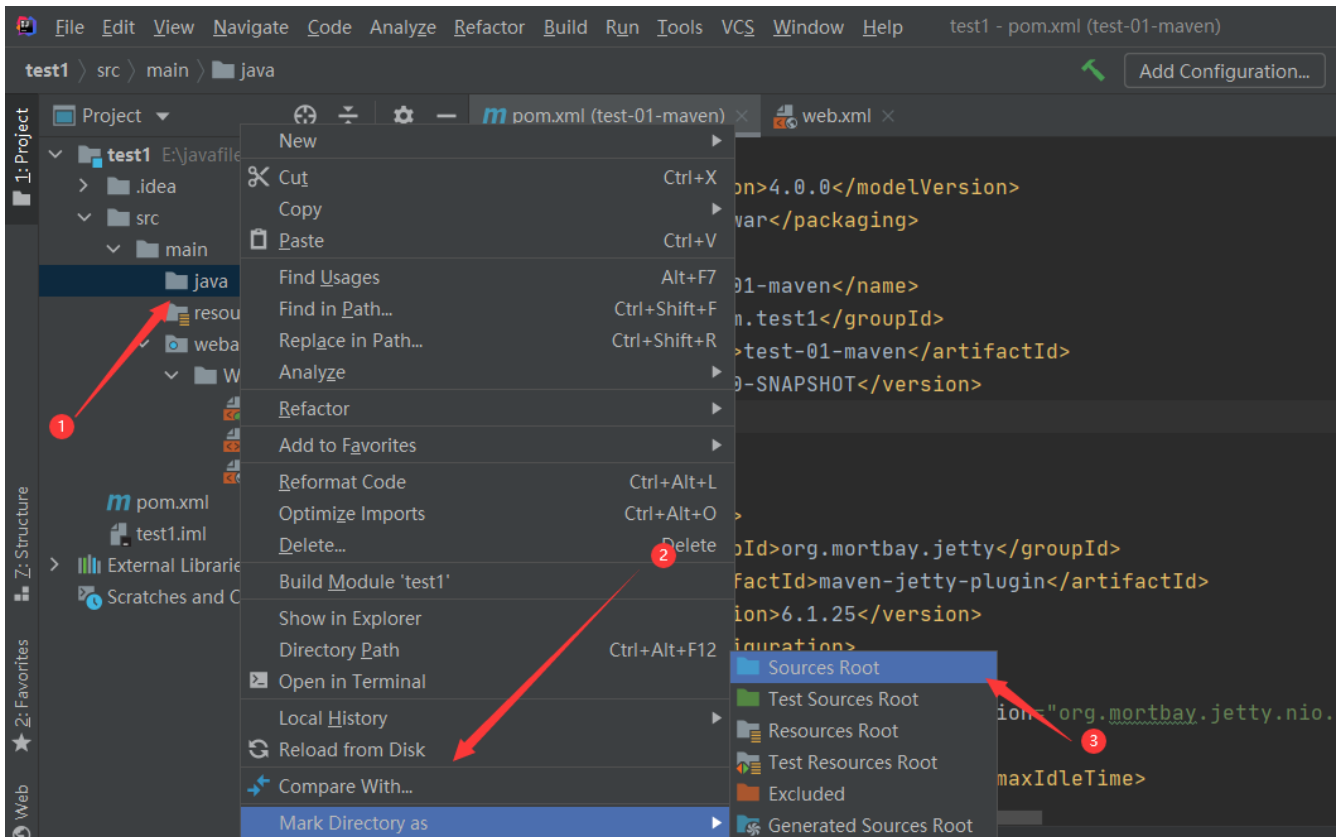
选择maven



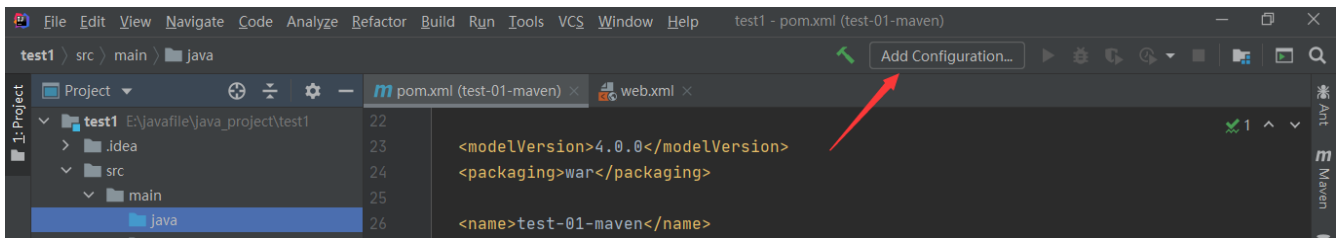
创建java源码目录



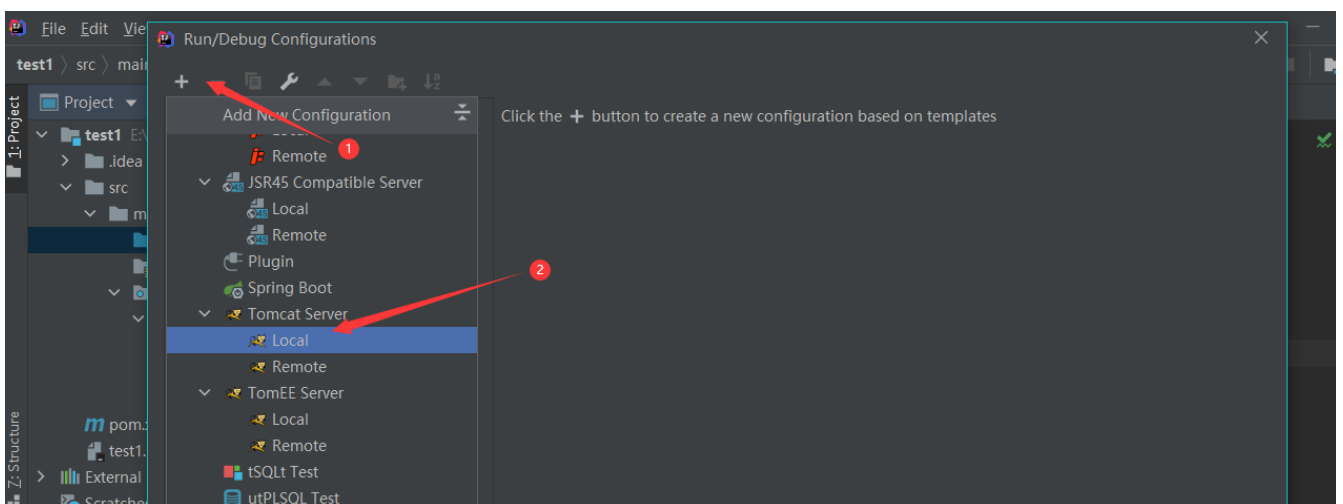
标记为java源码目录



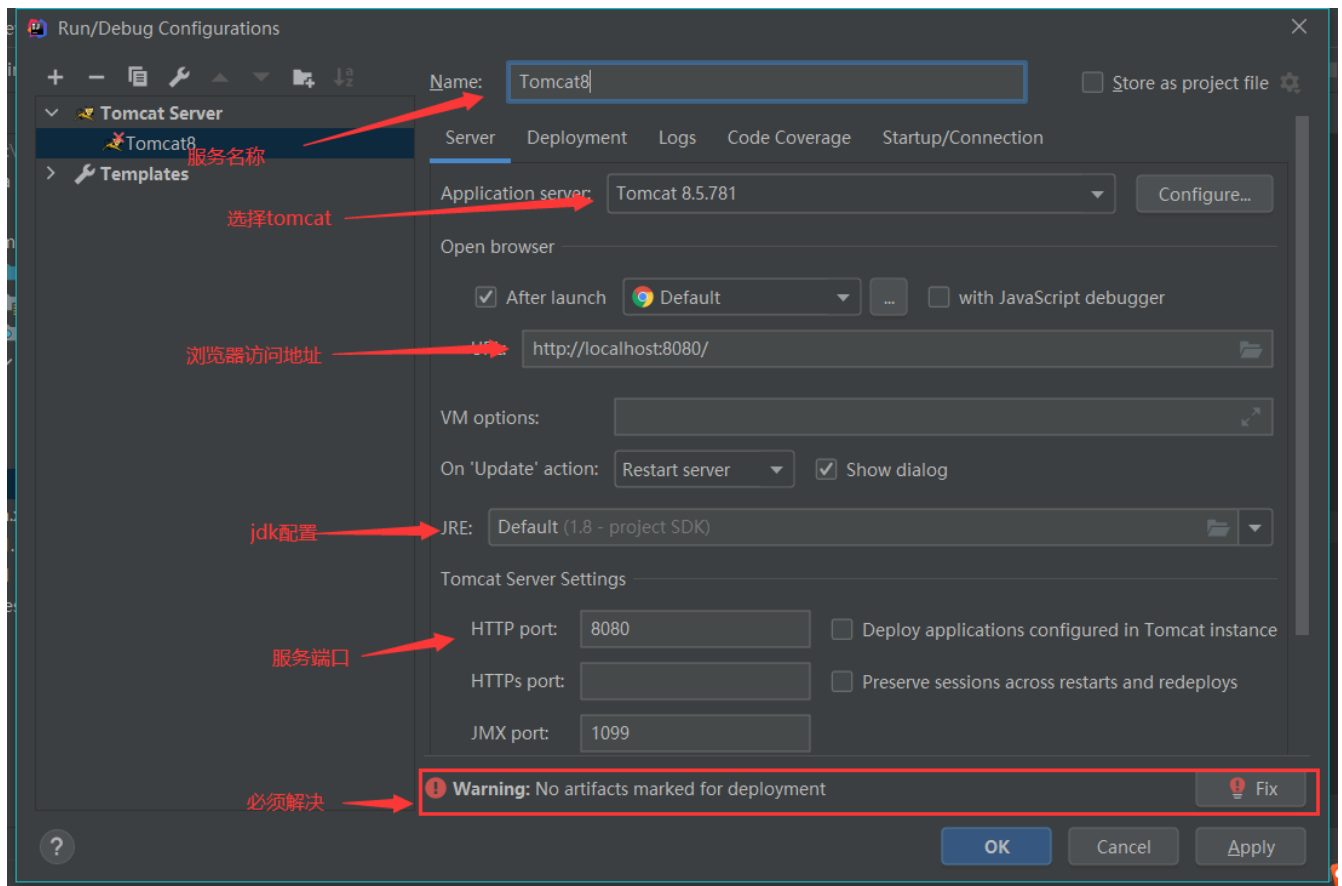
idea中配置tomcat



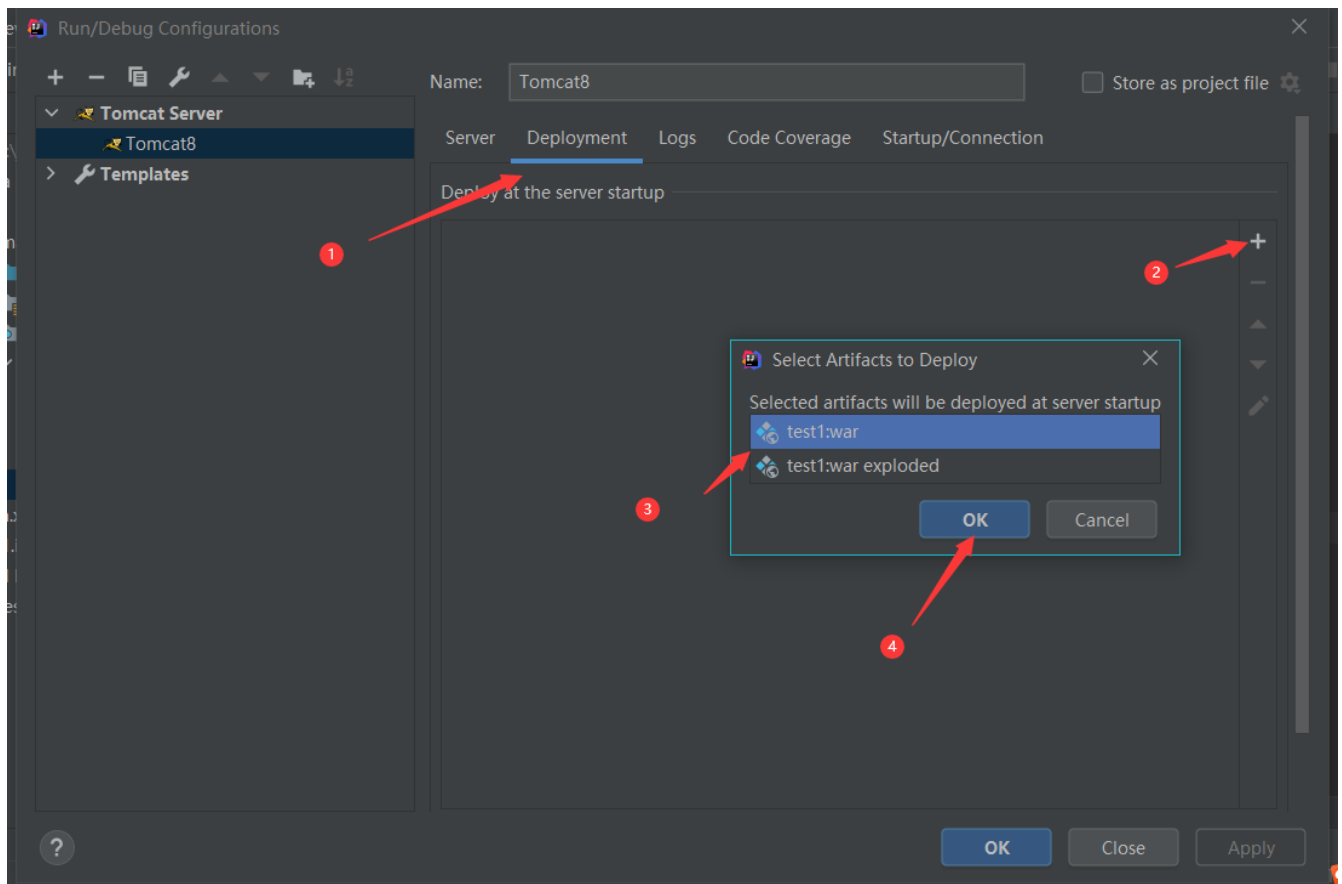
选中tomcat



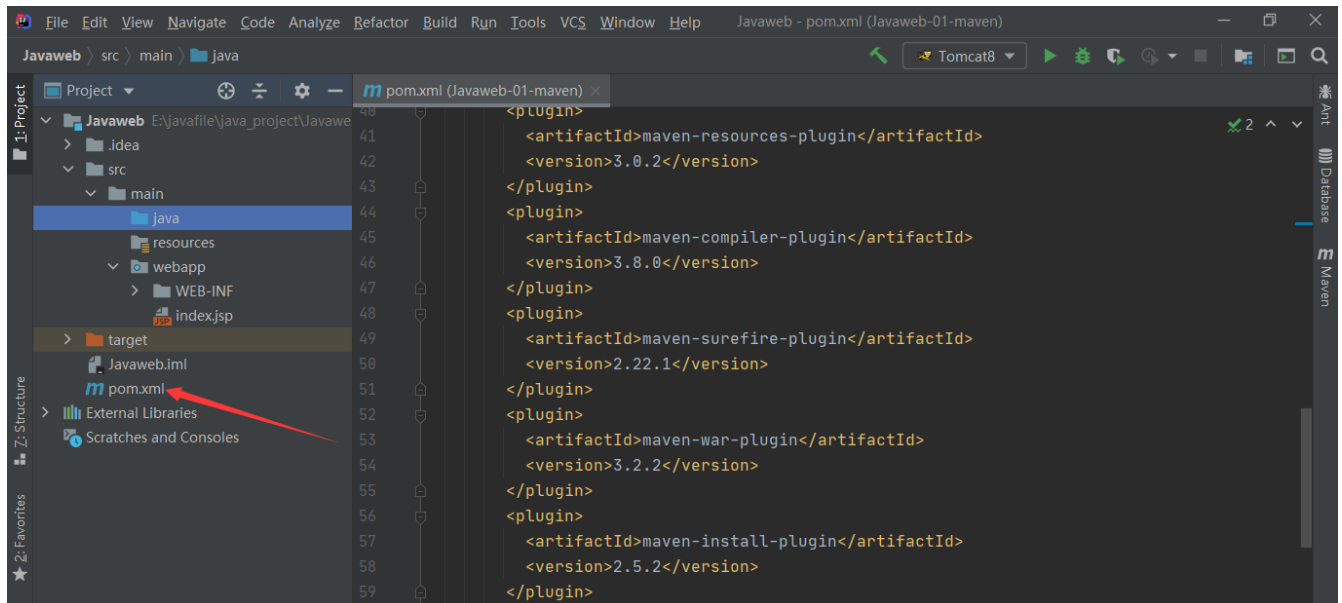
配置tomcat



解决warning, 选择包



pom.xml是maven项目的核心文件



```

<?xml version="1.0" encoding="UTF-8"?>
<!--maven版本和头文件-->
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <!--刚刚配置的GAV-->
  <groupId>com.javaweb</groupId>
  <artifactId>JavaWeb-01-maven</artifactId>
  <version>1.0-SNAPSHOT</version>
  <!-- 打包方式-->
  <packaging>war</packaging>

  <name>JavaWeb-01-maven Maven Webapp</name>
  <!-- FIXME change it to the project's website -->
  <url>http://www.example.com</url>
  <!--配置-->
  <properties>
  <!-- 项目默认构建编码-->
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <!-- 编译版本-->
  <maven.compiler.source>1.8</maven.compiler.source>
  <maven.compiler.target>1.8</maven.compiler.target>
  </properties>
  <!--项目依赖-->
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.11</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
  <!--项目打包构建用-->

```

```

<build>
  <finalName>Javaweb-01-maven</finalName>
  <pluginManagement><!-- lock down plugins versions to avoid using Maven defaults (may be
moved to parent pom) -->
    <plugins>
      <plugin>
        <artifactId>maven-clean-plugin</artifactId>
        <version>3.1.0</version>
      </plugin>
      <!-- see http://maven.apache.org/ref/current/maven-core/default-
bindings.html#Plugin_bindings_for_war_packaging -->
      <plugin>
        <artifactId>maven-resources-plugin</artifactId>
        <version>3.0.2</version>
      </plugin>
      <plugin>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.8.0</version>
      </plugin>
      <plugin>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>2.22.1</version>
      </plugin>
      <plugin>
        <artifactId>maven-war-plugin</artifactId>
        <version>3.2.2</version>
      </plugin>
      <plugin>
        <artifactId>maven-install-plugin</artifactId>
        <version>2.5.2</version>
      </plugin>
      <plugin>
        <artifactId>maven-deploy-plugin</artifactId>
        <version>2.8.2</version>
      </plugin>
    </plugins>
  </pluginManagement>
</build>
</project>

```

maven由于它是约定大于配置，之后可以能遇到写的配置文件不生效的问题。

```

<!--在build中配置resources，来防止我们资源导出失败的问题-->
<build>
  <resources>
    <resource>
      <directory>src/main/resources</directory>
      <includes>
        <include>**/*.properties</include>
        <include>**/*.xml</include>
      </includes>
      <filtering>true</filtering>
    </resource>
  </resources>
</build>

```

```

        <resource>
            <directory>src/main/java</directory>
            <includes>
                <include>**/*.properties</include>
                <include>**/*.xml</include>
            </includes>
            <filtering>true</filtering>
        </resource>
    </resources>
</build>

```

四、servlet

- Servlet就是sun公司开发动态web的一门技术
- Sun在这些API中提供一个接口叫做：Servlet，如果你想开发一个Servlet程序，只需要完成两个小步骤：
 - 编写一个类，实现Servlet接口
 - 把开发好java类部署到web服务器中

把实现了Servlet接口的Java程序叫做，Servlet

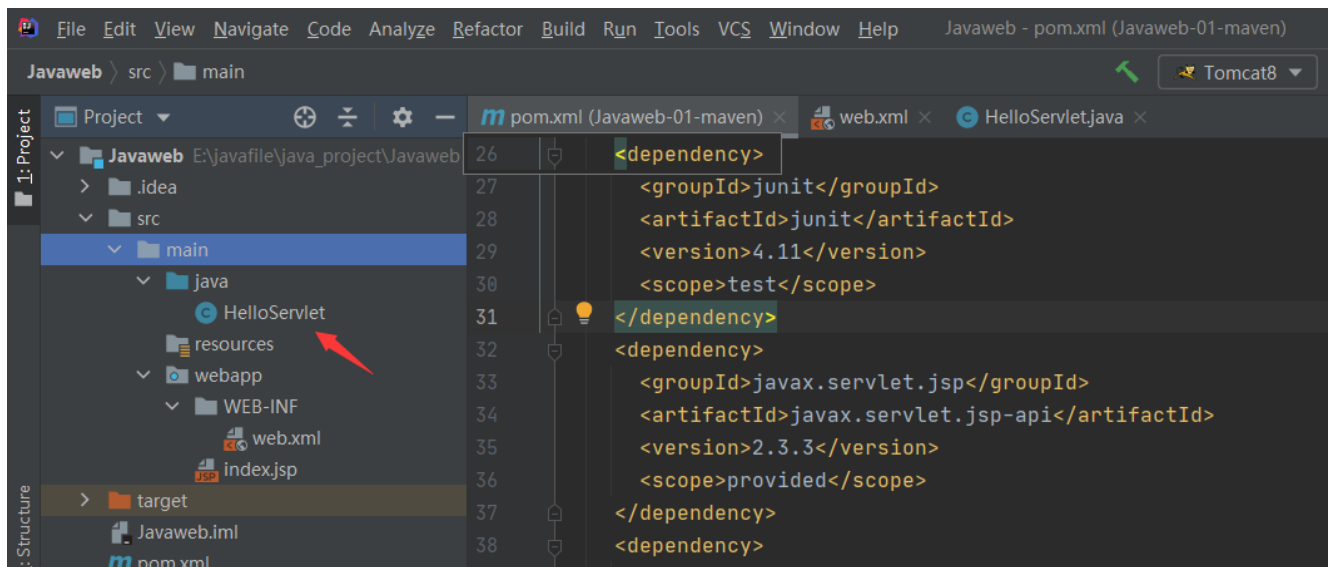
1.servlet需要的maven包

```

<dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>4.0.1</version>
    <scope>provided</scope>
</dependency>
<dependency>
    <groupId>javax.servlet.jsp</groupId>
    <artifactId>javax.servlet.jsp-api</artifactId>
    <version>2.3.3</version>
    <scope>provided</scope>
</dependency>

```

2.创建servlet类



继承HttpServlet，重写doGet和doPost

```
import javax.servlet.http.HttpServlet;
import java.io.IOException;
import java.io.PrintWriter;

/**
 * @author Heber
 * @version 1.0
 */
public class HelloServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws
ServletException, IOException {
        PrintWriter writer = resp.getWriter();
        writer.print("Hello,Servlet");
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws
ServletException, IOException {
        doGet(req, resp);
    }
}
```

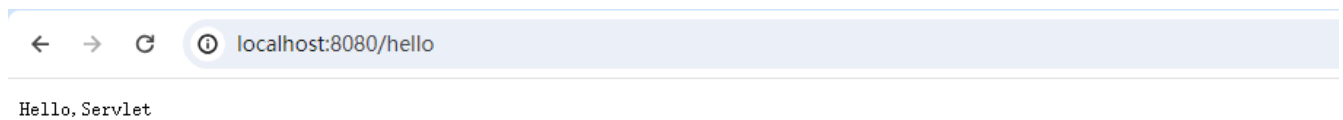
3.注册servlet

需要在web.xml中注册

```
<!--注册servlet-->
<servlet>
    <servlet-name>helloServlet</servlet-name>
    <!--在java下创建com.javaweb.servlet, 并把HelloServlet放进去-->
    <servlet-class>com.javaweb.servlet.HelloServlet</servlet-class>
</servlet>
<!--一个servlet对应一个mapping-->
<servlet-mapping>
    <servlet-name>helloServlet</servlet-name>
    <!--映射的请求路径-->
    <url-pattern>/hello</url-pattern>
</servlet-mapping>
```

4.启动访问

ip:8080/hello



javaweb等用的少，了解一下，直接进入springboot

springboot

一、概念

1.什么是springboot

就是一个javaweb的开发框架，和SpringMVC类似，对比其他javaweb框架的好处，官方说是简化开发，约定大于配置，you can “just run”，能迅速的开发web应用，几行代码开发一个http接口。

Spring Boot 基于 Spring 开发，Spring Boot 本身并不提供 Spring 框架的核心特性以及扩展功能，只是用于快速、敏捷地开发新一代基于 Spring 框架的应用程序。也就是说，它并不是用来替代 Spring 的解决方案，而是和 Spring 框架紧密结合用于提升 Spring 开发者体验的工具。Spring Boot 以**约定大于配置的核心思想**，默认帮我们进行了很多设置，多数 Spring Boot 应用只需要很少的 Spring 配置。同时它集成了大量常用的第三方库配置（例如 Redis、MongoDB、Jpa、RabbitMQ、Quartz 等等），Spring Boot 应用中这些第三方库几乎可以零配置的开箱即用。

2.Spring Boot主要优点

- 为所有Spring开发者更快的入门
- **开箱即用**，提供各种默认配置来简化项目配置
- 内嵌式容器简化Web项目
- 没有冗余代码生成和XML配置的要求

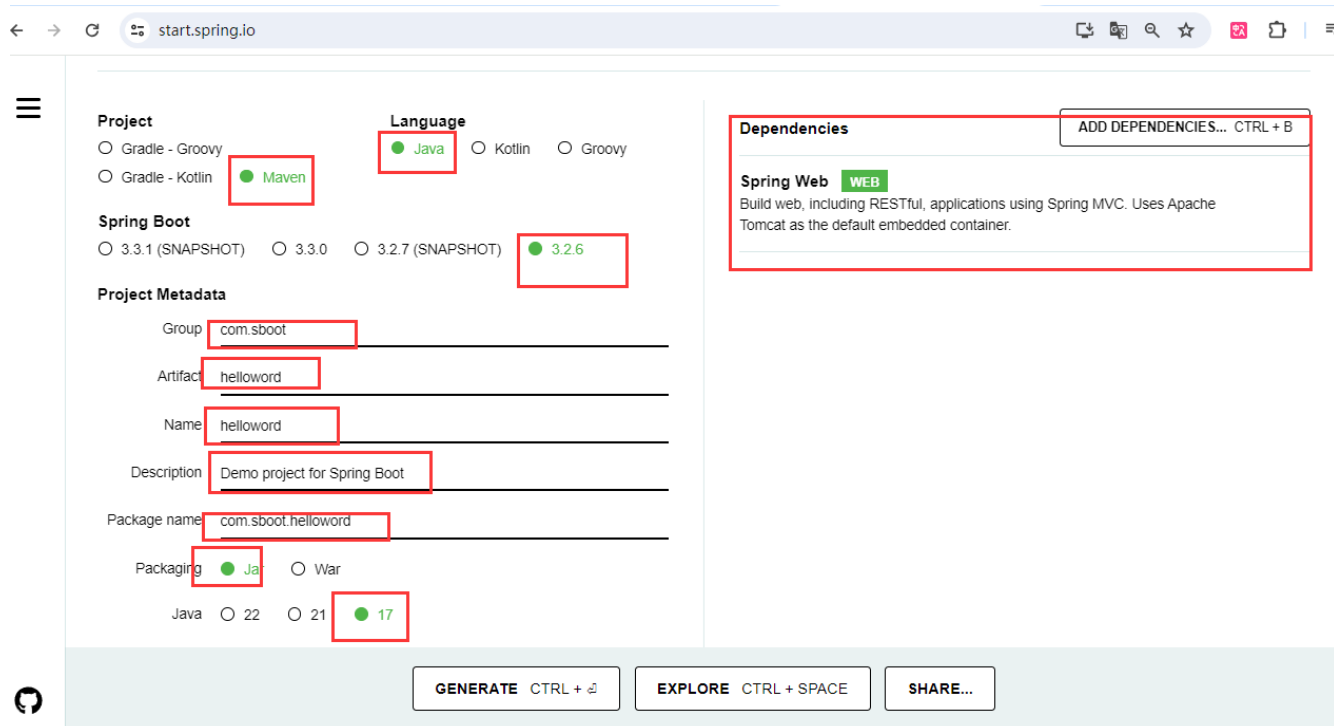
3.什么是微服务

微服务是一种架构风格，它要求我们在开发一个应用的时候，这个应用必须构建成一系列小服务的组合；可以通过http的方式进行互通，要说微服务架构，先得说说过去的单体应用架构。

二、第一个SpringBoot项目

1.创建方式一

浏览器打开<https://start.spring.io/>地址，使用Spring Initializr 的 Web页面创建项目

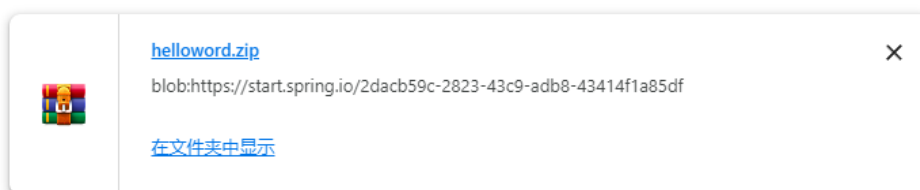


The screenshot shows the Spring Initializr web interface. Red boxes highlight the following elements:

- Project:** ☐ Gradle - Groovy, ☒ Gradle - Kotlin, ☒ Maven
- Language:** ☒ Java, ☐ Kotlin, ☐ Groovy
- Spring Boot:** ☐ 3.3.1 (SNAPSHOT), ☐ 3.3.0, ☐ 3.2.7 (SNAPSHOT), ☒ 3.2.6
- Project Metadata:**
 - Group: com.sboot
 - Artifact: helloworld
 - Name: helloworld
 - Description: Demo project for Spring Boot
 - Package name: com.sboot.helloworld
 - Packaging: ☒ Jar, ☐ War
 - Java: ☐ 22, ☐ 21, ☒ 17
- Dependencies:** A box containing "Spring Web" with a "WEB" tag and a description. A button "ADD DEPENDENCIES... CTRL + B" is at the top right.
- Buttons:** "GENERATE CTRL + G", "EXPLORE CTRL + SPACE", and "SHARE..." are at the bottom.

点击生成之后会出现一个下载的压缩包，我们的springboot项目生成完成，解压项目包，并用IDEA以Maven项目导入，一路下一步即可，直到项目导入完毕。

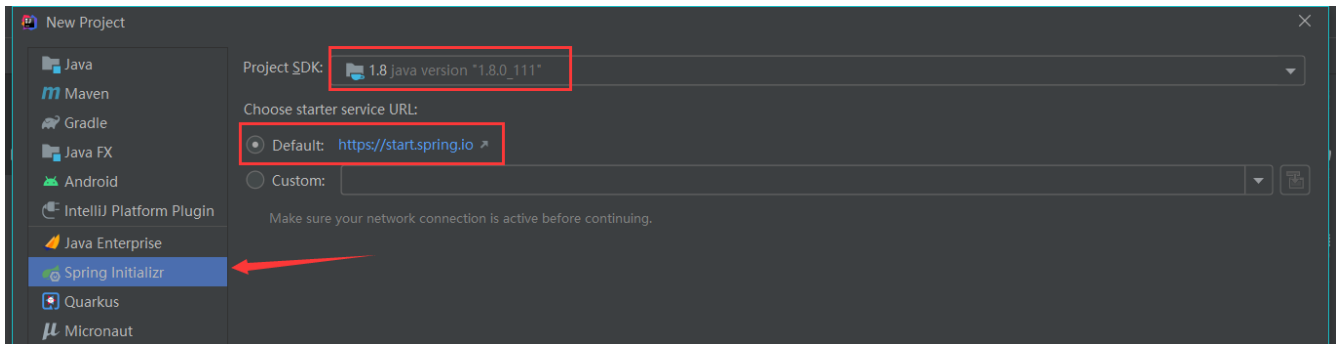
今天



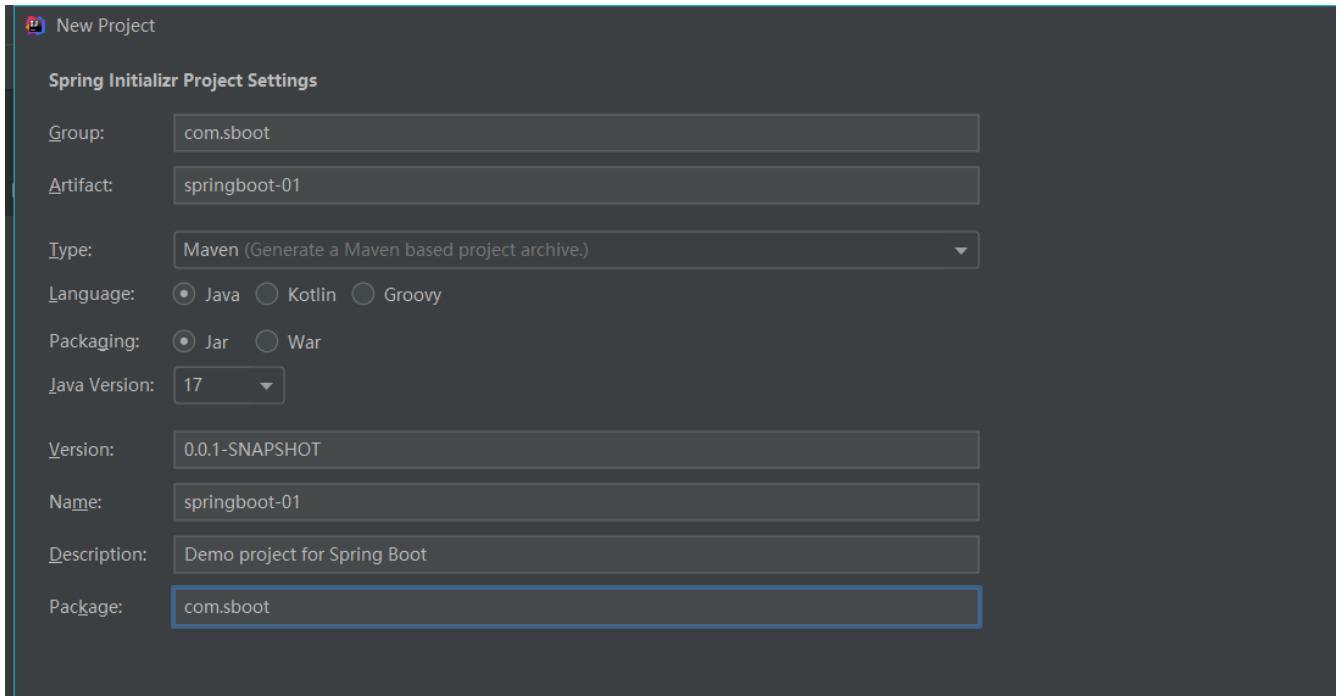
2.创建方式二

使用 IDEA 直接创建项目

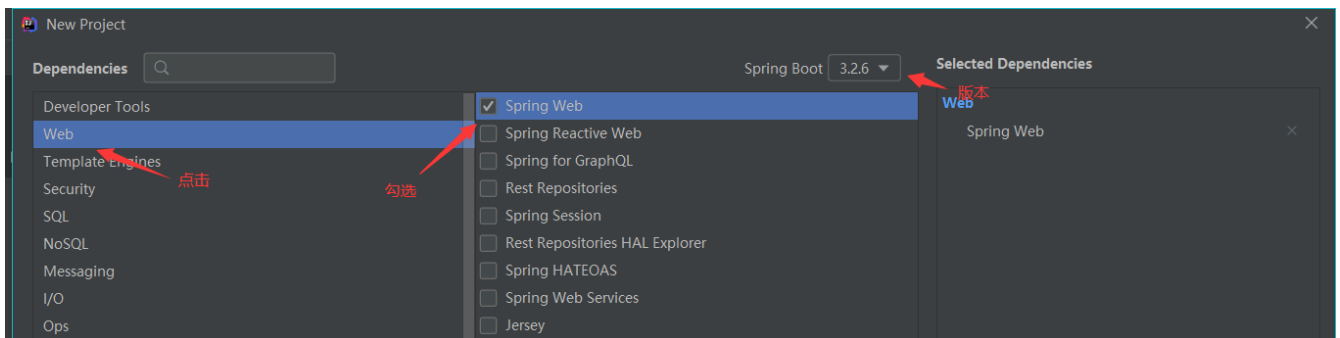
1.创建一个新项目，选择spring initializr



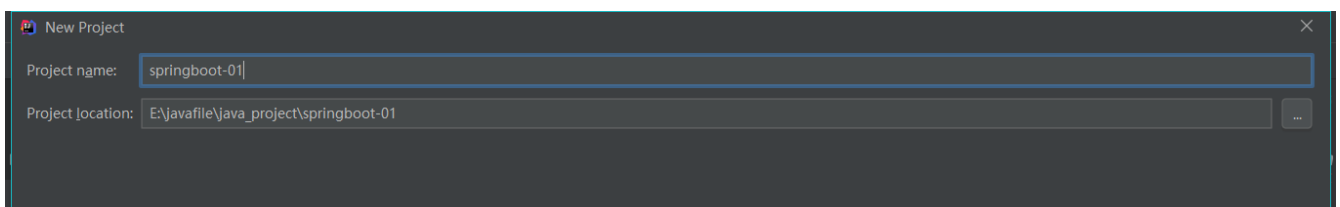
2. 填写项目信息



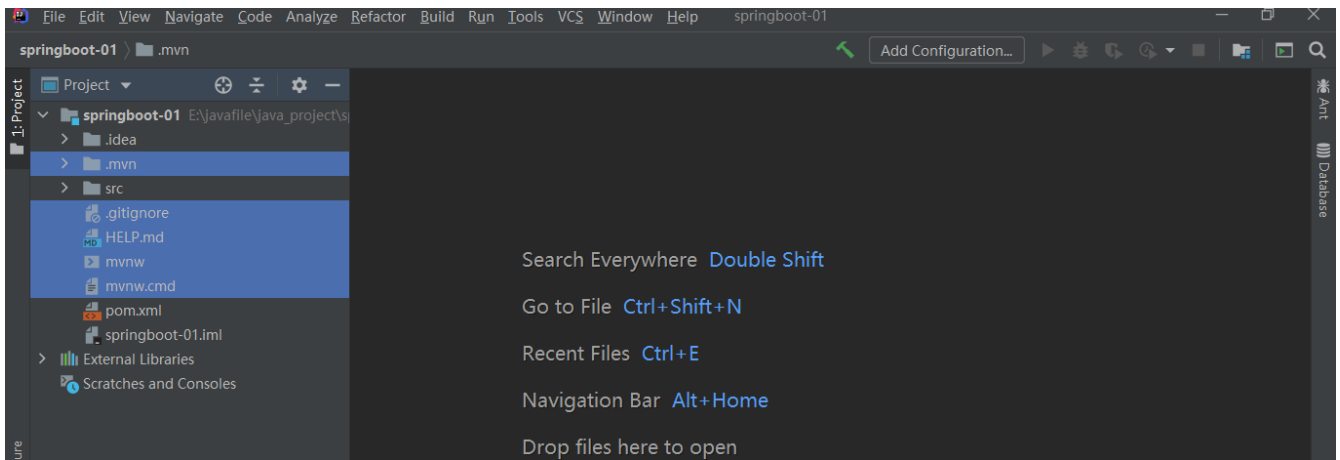
4. 选择初始化的组件 (勾选Spring Web 即可)



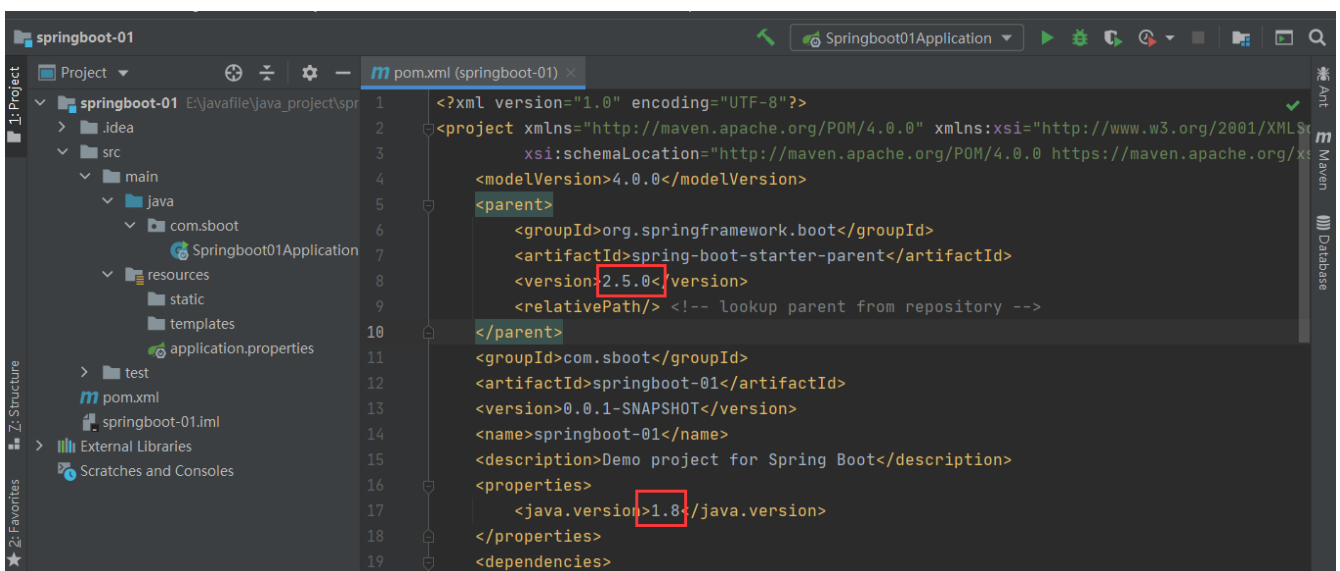
5. 填写项目路径



6. 等待项目构建成功,删除多余文件

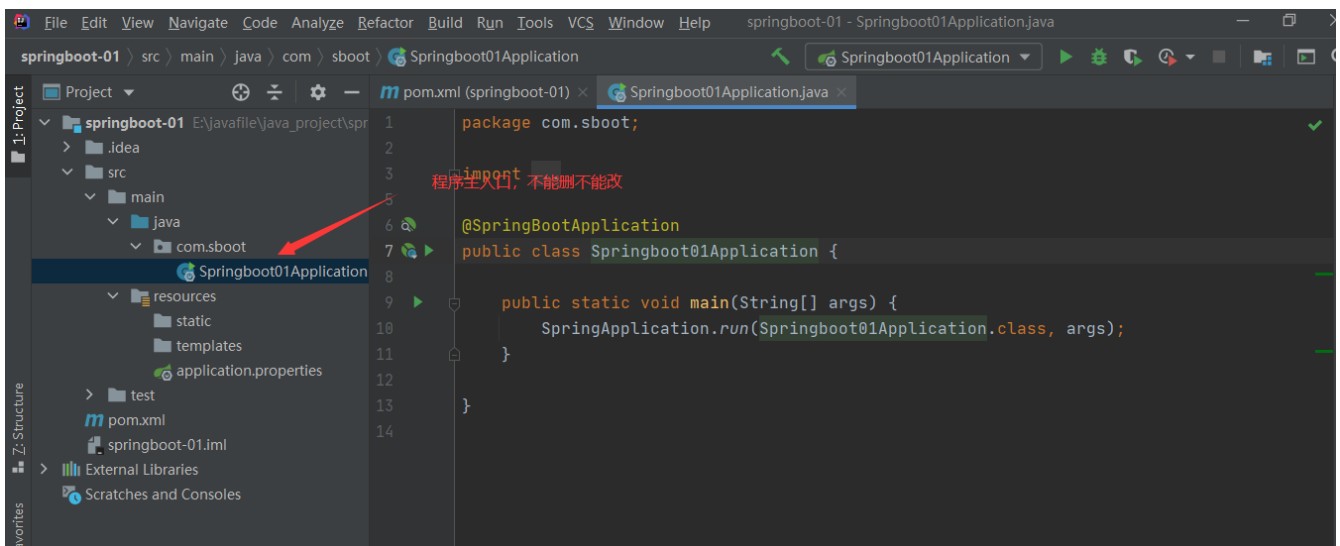


7.由于后面版本的springboot创建时都是3.几版本，不能选择jdk8所以需要自己在pom.xml中修改。同时需要修改springboot版本

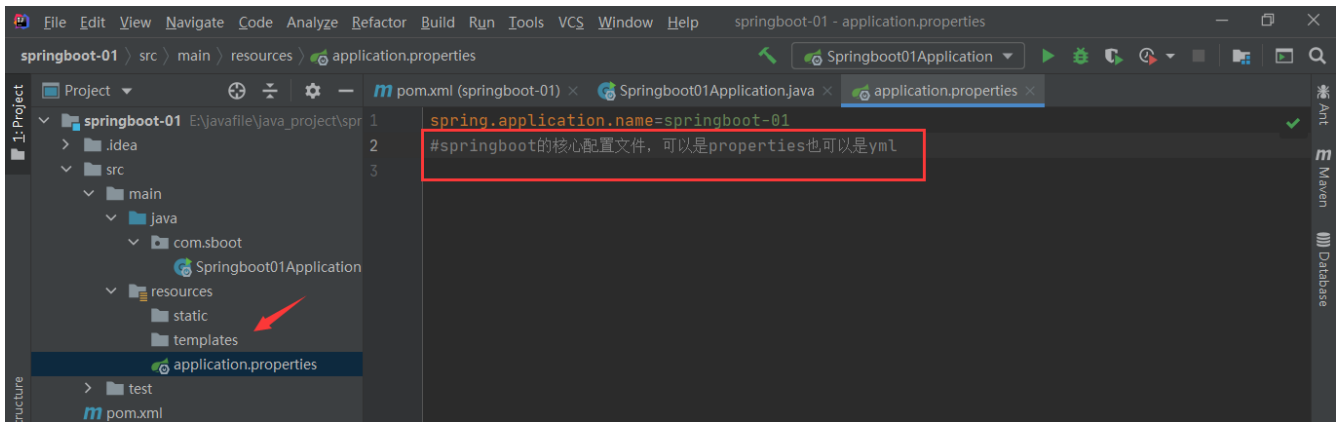


8.结构

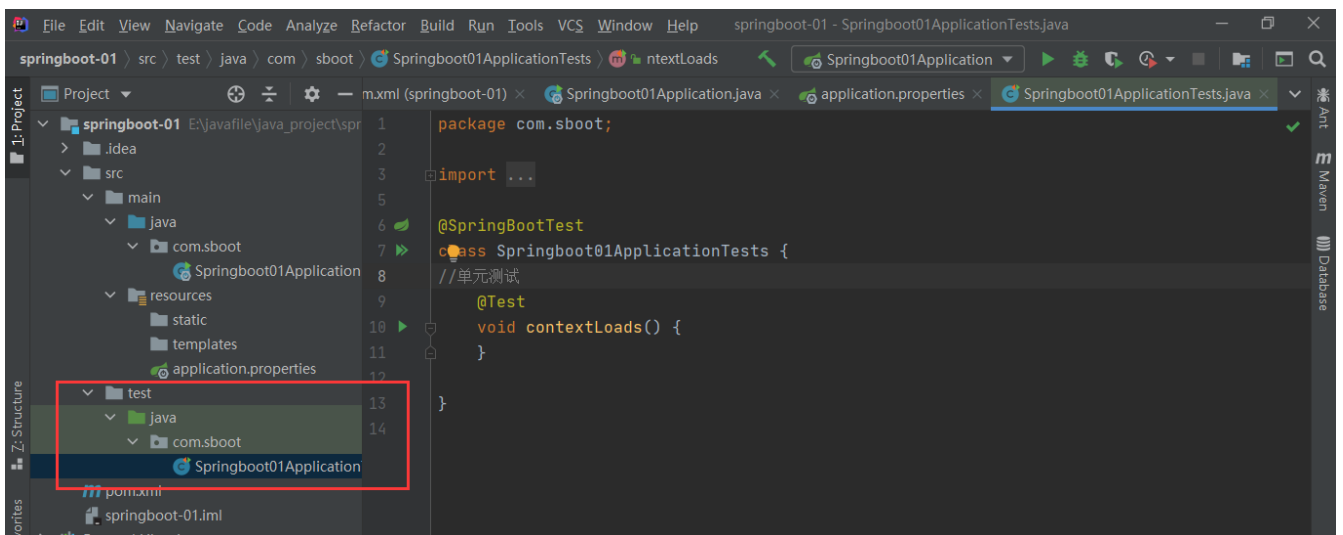
主入口



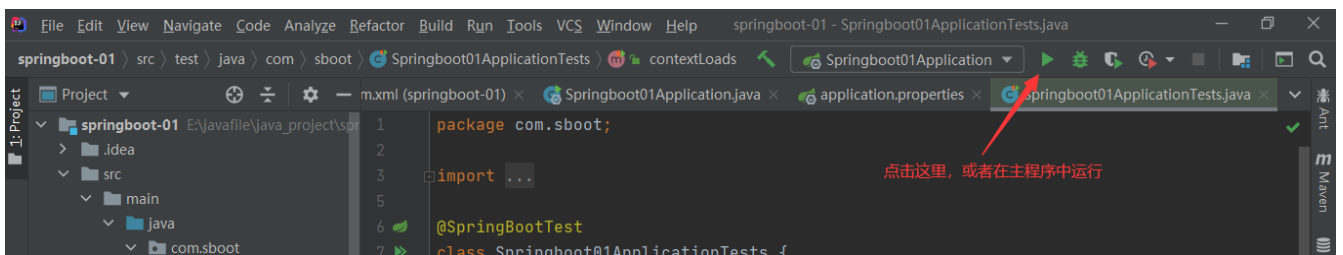
核心配置文件



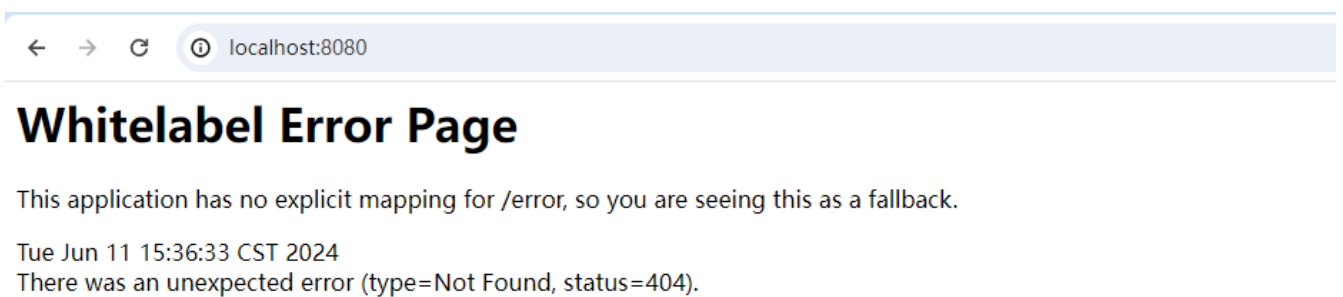
单元测试



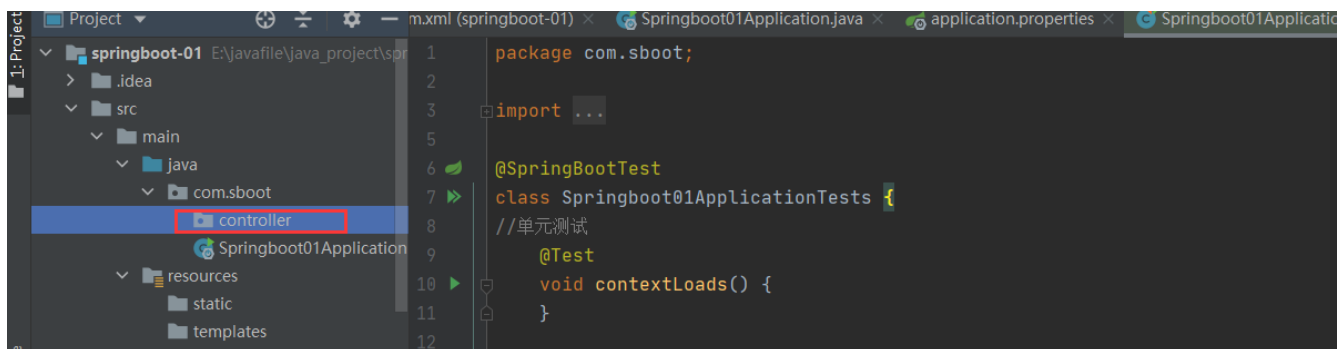
运行springboot



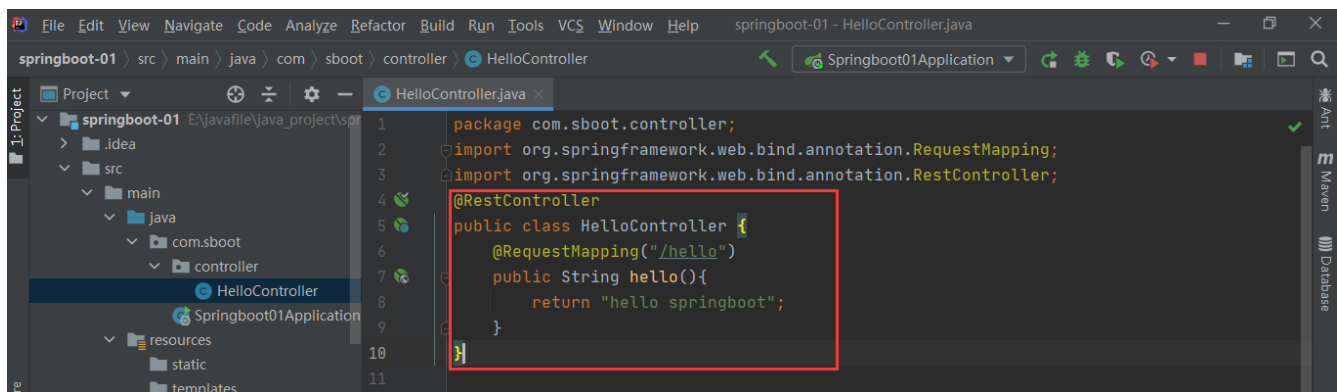
访问，由于新项目什么都没有所以出现以下界面



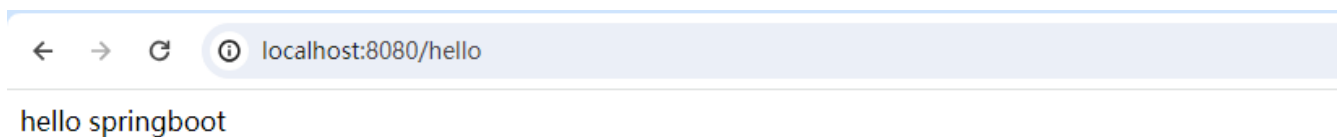
在主程序入口同级目录创建一个controller包



创建一个HelloController，编写一个接口



此时重新启动访问就可以获得返回值



pom.xml介绍

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <!--有一个父项目-->
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.5.0</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>
  <!--坐标配置-->
  <groupId>com.sboot</groupId>
  <artifactId>springboot-01</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>springboot-01</name>
  <description>Demo project for Spring Boot</description>
```

```

<!--java版本-->
<properties>
    <java.version>1.8</java.version>
</properties>
<dependencies>
    <!--启动器，web的依赖，集成了tomcat，servlet，.xml-->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <!--单元测试-->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <!--打jar包插件-->
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>

```

自定义banner

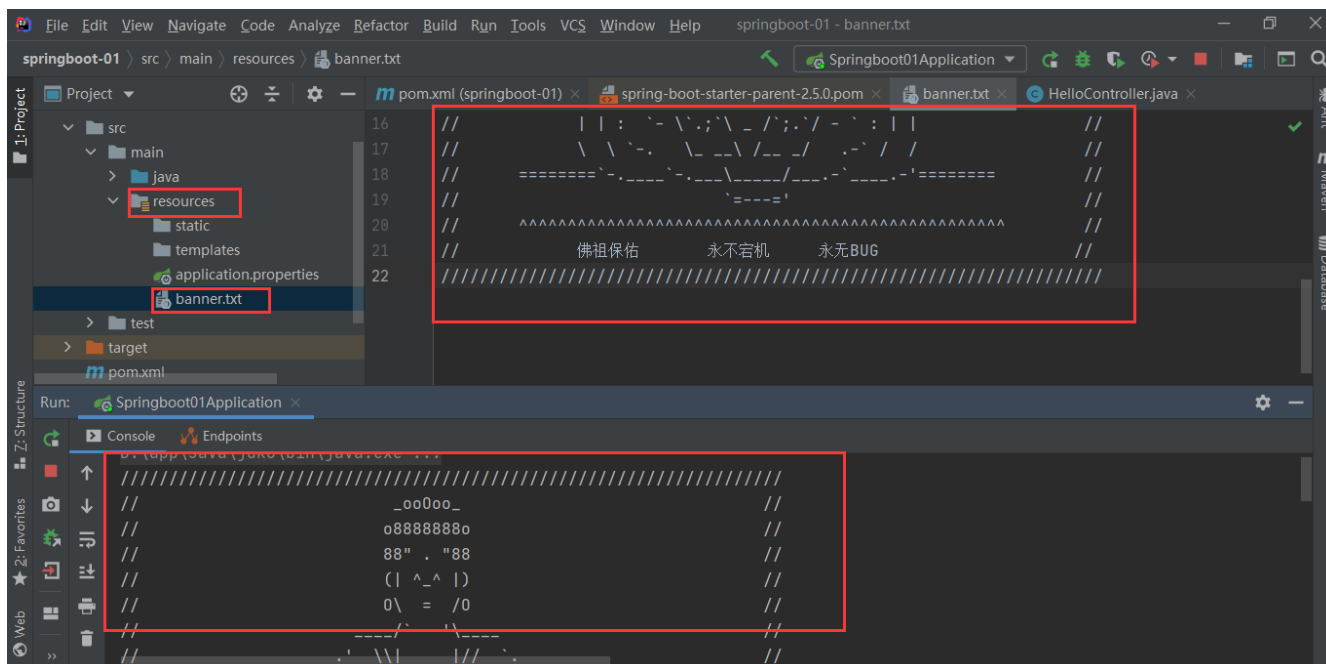
地址: <https://bootschool.net/>搜索需要的图形，比如佛祖



就能得到以下图形



然后在项目resource目录下创建一个banner.txt，把佛祖放进去，启动就会有惊喜



三、springboot自动装配原理

